📘 RTU KOTA · B.TECH CSE · V SEMESTER · 5CS3-01

Information Theory & Coding

Complete All-Units Lecture Notes & Solved Numericals

Comprehensive academic study material aligned with the Rajasthan Technical University (RTU) syllabus. Includes mathematical foundations, source & channel coding proofs, worked numericals, bilingual concept explanations (English + Hinglish), and complete RTU Exam Q&A banks.

5CS3-01	28 Hours	6 Units	100 Marks
COURSE CODE	TOTAL TEACHING	FULL SYLLABUS	IA: 30 | ETE: 70

📚 SYLLABUS UNITS INDEX & CONTENTS

Unit 1: Introduction & Communication Architecture	01 Hour · Scope & Outcomes
Unit 2: Introduction to Information Theory (Entropy, Mutual Info, DMC)	05 Hours · High Weightage
Unit 3: Source Coding Schemes (Huffman, Shannon-Fano, Lempel-Ziv)	05 Hours · Numerical Heavy
Unit 4: Linear Block Codes (Generator & Parity Check Matrices, Syndrome)	05 Hours · 5/10 Mark Numericals
Unit 5: Cyclic Codes (Galois Fields GF(2^m), Generator Polynomial, Circuits)	06 Hours · Hardware & Polynomials
Unit 6: Convolutional Codes (Code Tree, Trellis, State Diagram, Viterbi)	06 Hours · Viterbi Algorithm
Appendix: Quick Revision Sheets (QRS) Infographics	**Units 2 to 6 Visual Sheets**

📋 RTU Syllabus & Course Blueprint

5CS3-01 · Information Theory & Coding · B.Tech CSE · III Year – V Sem

6 UNITS · 28 HRS | Credit: 2 · 2L+0T+0P | Max Marks: 100 (IA:30 · ETE:70)

📋 RTU Syllabus — ITC (5CS3-01)

Rajasthan Technical University, Kota

UNIT	CONTENTS	HOURS	GUIDE LINK
1	**Introduction:** Objective, scope and outcome of the course.	01	Open →
2	**Introduction to Information Theory:** Uncertainty, Information and Entropy, Information measures for continuous random variables, source coding theorem. Discrete Memoryless channels, Mutual information, Conditional entropy.	05	Open →
3	**Source Coding Schemes for Data Compaction:** Prefix code, Huffman code, Shannon-Fano code & Lempel-Ziv coding, channel capacity. Channel coding theorem. Shannon limit.	05	Open →
4	**Linear Block Code:** Introduction to error correcting codes, coding & decoding of linear block code, minimum distance consideration, conversion of non-systematic form of matrices into systematic form.	05	Open →
5	**Cyclic Code:** Code Algebra, Basic properties of Galois fields (GF), polynomial operations over Galois fields, generating cyclic code by generating polynomial, parity check polynomial. Encoder & decoder for cyclic codes.	06	Open →
6	**Convolutional Code:** Convolutional encoders of different rates. Code Tree, Trellis and state diagram. Maximum likelihood decoding of convolutional code: The Viterbi Algorithm. Free distance of a convolutional code.	06	Open →
	Total	**28**	

EXAM PATTERN

COMPONENT	MARKS	DURATION
Internal Assessment (IA)	30	Continuous
End Term Exam (ETE)	70	3 Hours
Total	**100**	—

📘 UNIT 1 — Introduction

Objective, Scope and Outcome of the Course | Hours: 01

ENGLISH

Information Theory & Coding (ITC) is the mathematical study of how information is quantified, encoded, transmitted, and decoded reliably and efficiently over noisy or noiseless communication channels and storage media.

HINGLISH

Simple bhasha mein — ye subject batata hai ki **"information"** ko number mein kaise measure karein (entropy/bits), data ko **compress** kaise karein (kam space le), aur jab data **noisy channel** (jaise WiFi, satellite, CD/QR) se bheja jaaye to usme errors ko detect/correct kaise karein.

Objective of the Course

High

- To introduce the mathematical foundation of **information measurement** — uncertainty, entropy, mutual information.
- To study **source coding** techniques (Huffman, Shannon-Fano, Lempel-Ziv) for data compaction.
- To understand **channel coding** theorems and the limits of reliable communication (Shannon limit, channel capacity).
- To design and analyze **error-control codes**: Linear Block Codes, Cyclic Codes, Convolutional Codes.

HINGLISH

Course ka goal hai — tumhe ye samjhana ki kisi message mein "kitni naye baat / information" hai, use kaise efficiently bhejen (compression), aur agar transmission ke beech mein noise se error aa jaaye to use pakdo aur fix karo (error correcting codes — jo tumhare QR code, CD, satellite comm mein use hote hain).

Scope of the Course

Medium

ITC forms the theoretical backbone of:

- **Data Compression** — ZIP, JPEG, MP3, video codecs (entropy coding).
- **Reliable Digital Communication** — mobile networks, satellite links, deep-space comm (Voyager used convolutional + Golay codes).
- **Storage Systems** — CD/DVD/Hard-disk error correction (Reed-Solomon, cyclic codes).
- **Cryptography & Machine Learning** — entropy concepts used in security and decision-tree/ML feature selection.

HINGLISH

Ye subject sirf "theory" nahi — practically tumhare phone ke `WiFi`, `4G/5G`, file `.zip` karna, `QR code` scan karna — sab ke peeche yahi concepts kaam karte hain. Isliye ye CS ka core fundamental subject hai, jaise DBMS ya OS.

Outcome of the Course

High

After completing this course, the student will be able to:

- **CO1:** Define and compute information, entropy, and mutual information for discrete/continuous sources.
- **CO2:** Apply source coding theorem and design optimal codes (Huffman, Shannon-Fano).
- **CO3:** Analyze discrete memoryless channels and compute channel capacity.
- **CO4:** Design and decode Linear Block Codes and Cyclic Codes.
- **CO5:** Understand Convolutional Codes and Viterbi decoding algorithm.

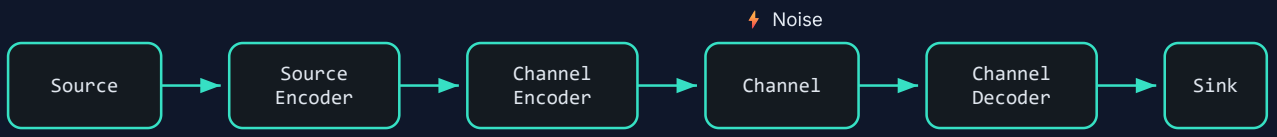


Fig 1.1 — Block diagram of a general communication system (this is the "scope map" of the whole subject)

HINGLISH TRICK

Communication System Pipeline: Source $\rightarrow$ **Source Encoder** (Faltu bits hatao/Compress karo) $\rightarrow$ **Channel Encoder** (Parity/Guard bits jodo taaki noise se bache) $\rightarrow$ **Channel** (Noise ⚡) $\rightarrow$ **Channel Decoder** $\rightarrow$ **Source Decoder** $\rightarrow$ **Sink** (Destination)!

🗺️ Unit 1 Overview Map

Quick Revise

UNIT 1: Communication Architecture & ITC Fundamentals

1. Information Quantification

- **Self Information:** $I(x_i) = \log_2(1/p_i)$ bits
- **Entropy $H(X)$:** Average uncertainty per symbol $\sum p_i \log(1/p_i)$
- **Mutual Information:** Information shared between input & output

2. Source vs Channel Coding

- **Source Coding:** Compression (Huffman, Shannon-Fano, LZ)
- **Channel Coding:** Error correction (LBC, Cyclic, BCH, RS)
- **Shannon Capacity:** Maximum error-free data rate C

3. Practical Applications

- **Storage:** CD/DVD/HDD (Reed-Solomon, CRC)
- **Wireless:** 4G/5G, WiFi, Satellite deep-space telemetry
- **Media:** JPEG, MP3, ZIP archives, QR codes

❓ Unit 1 — Q&A (RTU Exam Pattern)

Q1. Explain the objective and scope of Information Theory and Coding. ★★★ 5 Marks

See "Objective of the Course" + "Scope of the Course" sections above — explain quantifying information, source coding, channel coding, error control, with real-world applications (ZIP, QR, satellite comm).

Q2. List the course outcomes of ITC. ★★ 2-3 Marks

CO1–CO5 as listed above: define information measures, apply source coding theorem, analyze channel capacity, design linear block/cyclic codes, understand convolutional codes & Viterbi decoding.

Q3. Draw and explain the block diagram of a digital communication system. ★★★ 5 Marks

Refer Fig 1.1 — Source → Source Encoder → Channel Encoder → Channel (with noise) → Channel Decoder → Source Decoder → Sink. Explain role of each block briefly.

📗 UNIT 2 — Introduction to Information Theory

Uncertainty, Information & Entropy · Continuous Info Measures · Source Coding Theorem · DMC · Mutual Information · Conditional Entropy | Hours: 05

1. Uncertainty & Information

High ⭐⭐⭐

ENGLISH

Uncertainty of an event is a measure of how unpredictable its occurrence is — the lower the probability of an event, the higher the uncertainty/surprise when it occurs.
Information conveyed by a message is inversely related to its probability of occurrence. If event *x* has probability *P(x)*, the information content is:

$$I(x) = \log_2\left(\frac{1}{P(x)}\right) = -\log_2 P(x) \quad [\text{bits}]$$

HINGLISH

Soch — agar koi event **hamesha hota hai** (P=1), to usme koi "naya" info nahi (I=0). Lekin agar koi **rare event** hota hai (P chhota), to usme bahut "surprise" / information hoti hai. Jaise "Kal Sun nikla" (P≈1 → info≈0) vs "Aaj Jaipur mein snowfall hua" (P≈0 → info bahut high!).

Units of Information

LOG BASE	UNIT	USED WHEN
$\log_2$	bit	Binary systems (most common, default in exams)
$\log_{10}$	Hartley / decit	Decimal systems
$\log_e$ (ln)	nat	Natural log, used in continuous/differential entropy

Properties of Information Measure

Medium

1. $I(x) = 0$, when $P(x) = 1$ (certain event carries no information).
2. $I(x) \geq 0$, since $0 \leq P(x) \leq 1$ (information is never negative).
3. $I(x_2) > I(x_1)$, if $P(x_2) < P(x_1)$ (lower probability ⇒ more information).
4. If x_1 and x_2 are independent: $I(x_1,x_2) = I(x_1) + I(x_2)$ (additivity property).

EXAMPLE

A binary source emits '0' with P=0.25 and '1' with P=0.75. Find information carried by each symbol.

Solution:

$I(0) = -\log_2(0.25) = \log_2 4 =$ **2 bits**

$I(1) = -\log_2(0.75) =$ **0.415 bits**

→ Rarer symbol '0' carries more information than frequent symbol '1'.

1B. Postulates for a Measure of Information (Why Logarithm?)

Medium ⭐ ⭐ — "Why log?" theory question

ENGLISH

Shannon defined information using a logarithmic function based on the following postulates (axioms) that any reasonable information measure I(P) must satisfy:

1. **Postulate 1:** $I(P) = 0$ when $P = 1$ (a certain event gives zero information).
2. **Postulate 2:** $I(P) \geq 0$ for $0 \leq P \leq 1$ (information is non-negative).
3. **Postulate 3:** $I(P_1) > I(P_2)$ if $P_1 < P_2$ (monotonically decreasing — rarer event ⇒ more information).
4. **Postulate 4 (Additivity):** $I(P_1 \cdot P_2) = I(P_1) + I(P_2)$ for independent events (joint information of independent events = sum of individual information).

The **only** mathematical function satisfying all four postulates simultaneously is the **logarithmic function**: $I(x) = \log(1/P(x))$. This is why entropy/information is always defined using logarithms, not any other function.

HINGLISH

Yeh question aksar aata hai: *"Information measure ke liye log function hi kyu use hota hai?"* — Answer simple hai: Shannon ne 4 conditions (postulates) set ki jo ek "sensible" information measure follow karna chahiye, aur jab tum maths se check karo, sirf **logarithm function** hi un sab conditions ko satisfy karta hai (especially additivity property — independent events ka information add hona chahiye, multiply nahi, aur log(a·b)=log(a)+log(b) yehi karta hai).

2. Entropy

High ⭐⭐⭐⭐⭐ (Most Important)

ENGLISH

Entropy H(X) is the **average (expected) information per symbol** generated by a discrete memoryless source. It measures the average uncertainty of a random variable.

$$H(X) = - \Sigma P(x_i) \cdot \log_2 P(x_i) \quad [\text{bits/symbol}] \quad i = 1...n$$

HINGLISH

Entropy = "average information" jo ek source produce karta hai per symbol. Agar source bahut **predictable** hai (jaise hamesha same letter), entropy **kam** hogi. Agar source bahut **random** hai (sab symbols equally likely), entropy **maximum** hogi. Isiliye entropy ko "average surprise" bhi kehte hain.

Properties of Entropy

High

1. $0 \leq H(X) \leq \log_2(n)$, where n = number of symbols.
2. H(X) = 0, when one symbol has probability 1 (no uncertainty — fully predictable).
3. H(X) is **maximum (= $\log_2 n$)** when all symbols are **equiprobable** (P = 1/n each) — maximum uncertainty.
4. Entropy is a concave function of probability distribution.

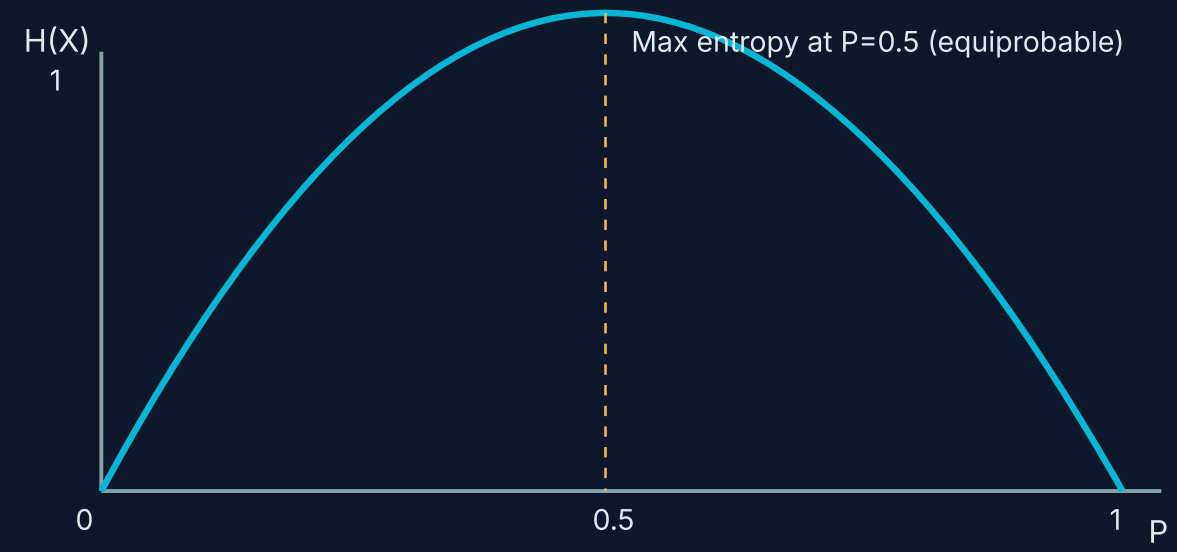


Fig 2.1 — Binary Entropy Function H(p) = −p·log₂p − (1−p)·log₂(1−p). Peaks at p = 0.5

EXAMPLE (VERY COMMONLY ASKED NUMERICAL)

A source emits 4 symbols with probabilities: $P(x_1)=0.5$, $P(x_2)=0.25$, $P(x_3)=0.125$, $P(x_4)=0.125$. Find entropy.

Solution:

$H(X) = -[0.5\log_2 0.5 + 0.25\log_2 0.25 + 0.125\log_2 0.125 + 0.125\log_2 0.125]$

$= -[0.5(-1) + 0.25(-2) + 0.125(-3) + 0.125(-3)]$

$= 0.5 + 0.5 + 0.375 + 0.375 =$ **1.75 bits/symbol**

Proof: H(X) is Maximum when Symbols are Equiprobable

High — common 5-mark proof question

To maximize $H(X) = -\Sigma P_i \log_2 P_i$ subject to constraint $\Sigma P_i = 1$, apply Lagrange multipliers:

```
L = -Σ Pᵢ log₂Pᵢ + λ(Σ Pᵢ - 1)  ∂L/∂Pᵢ = -log₂Pᵢ - log₂e + λ = 0 ⟹ Pᵢ = constant for all i = 1/n
```

Substituting $P_i = 1/n$ back gives $\mathbf{H_{max}(X) = \log_2 n}$. Hence entropy is maximum exactly when all symbols are equally likely — this is the equiprobable / uniform distribution case.

EXAMPLE (NON POWER-OF-2 PROBABILITIES)

A discrete memoryless source emits 3 symbols with $P(x_1)=0.6$, $P(x_2)=0.3$, $P(x_3)=0.1$. Find H(X) and compare with the maximum possible entropy for a 3-symbol source.

Solution:

Since these are not clean powers of ½, use $\log_2 x = \ln x / \ln 2$ (or $\log_{10} x / \log_{10} 2$):

$\log_2 0.6 = -0.737$, $\log_2 0.3 = -1.737$, $\log_2 0.1 = -3.322$

$H(X) = -[0.6(-0.737) + 0.3(-1.737) + 0.1(-3.322)]$

$= 0.442 + 0.521 + 0.332 =$ **1.295 bits/symbol**

$H_{max} = \log_2 3 =$ **1.585 bits/symbol** (equiprobable case, each P=1/3)

💡 Since $H(X) = 1.295 < H_{max} = 1.585$, this confirms the source is not "maximally random" — it carries some redundancy because the symbols are not equiprobable (x_1 is much more likely than x_3).

🧪 Interactive Simulation — Binary Source Entropy Calculator

Try it

Drag the slider to change P(0) for a binary source (P(1) auto-adjusts to 1−P(0)) and watch H(X) update live — this is exactly Fig 2.1's curve, but computed for the value you pick.

`P(0) = 0.50  |  P(1) = 0.50`

H(X) = 1.000 bits/symbol

💡 At P(0)=0.5 entropy is at its peak (=1 bit) — try dragging to the edges to see H(X)→0 as the source becomes predictable.

Joint Entropy & Marginal Entropy

Medium ⭐⭐⭐

ENGLISH

Joint Entropy H(X,Y) is the average uncertainty associated with a pair of random variables (X,Y) taken together:

$$H(X,Y) = -\sum_i \sum_j P(x_i,y_j) \cdot \log_2 P(x_i,y_j)$$

Marginal Entropy is simply H(X) or H(Y) computed individually from the marginal probability distributions $P(x_i)=\Sigma_j P(x_i,y_j)$ and $P(y_j)=\Sigma_i P(x_i,y_j)$.

HINGLISH

Joint entropy = dono variables X aur Y ko **ek saath** consider karke total uncertainty. Marginal entropy = sirf X (ya sirf Y) ki uncertainty, dusre ko "ignore" karke (uski sab possibilities sum kar di).

```
Key relation: H(X,Y) = H(X) + H(Y|X) = H(Y) + H(X|Y)
```

Rate of Information Transmission

Low-Medium ⭐

ENGLISH

If a discrete source emits *r* symbols per second, the **rate of information R** (in bits/second) is:

$$R = r \cdot H(X) \quad \text{[bits/second]}$$

HINGLISH

Simple — entropy batati hai "per symbol" kitni info hai, aur agar tumhe pata hai ki source ek second mein kitne symbols bhejta hai (r), to dono multiply karke total info-rate (bits/sec) mil jaata hai.

3. Information Measures for Continuous Random Variables

Medium ⭐⭐

ENGLISH

For a continuous random variable X with probability density function f(x), the average information is called **differential entropy**:

$$h(X) = -\int_{-\infty}^{+\infty} f(x) \cdot \log_2 f(x)\, dx$$

HINGLISH

Discrete entropy mein hum probabilities ka sum karte hain, continuous case mein (jaise voltage, temperature signals) probability ki jagah **density function f(x)** hota hai, aur sum ki jagah **integral**. Important difference: **differential entropy negative bhi ho sakti hai** (discrete entropy kabhi negative nahi hoti)!

Differential Entropy of Gaussian (Normal) Distribution

Often Asked

For X ~ N(μ, σ²):

$$h(X) = \tfrac{1}{2} \log_2(2\pi e\sigma^2)$$

This is the **maximum differential entropy** among all continuous distributions with the same variance σ² — i.e., for a fixed power/variance constraint, the Gaussian source is the "most random" (max entropy) source.

EXAMPLE — UNIFORM DISTRIBUTION

A continuous random variable X is uniformly distributed over the interval [0, 8], i.e. f(x) = 1/8 for $0 \le x \le 8$. Find its differential entropy h(X).

Solution:

For a uniform distribution over [a,b], f(x) = 1/(b−a), so:

$h(X) = -\int_a^b (1/(b-a)) \cdot \log_2(1/(b-a))\, dx = \log_2(b-a)$

Here (b−a) = 8, so $h(X) = \log_2 8$ = **3 bits**

💡 General shortcut: for uniform X on [a,b], $h(X) = \log_2(b-a)$. If (b−a) < 1, h(X) becomes **negative** — a perfectly valid result for differential entropy (unlike discrete entropy, which can never be negative).

ASPECT	DISCRETE ENTROPY H(X)	DIFFERENTIAL ENTROPY H(X)
Defined using	Probability mass function $P(x_i)$	Probability density function f(x)
Operation	Summation (Σ)	Integration (∫)
Sign	Always ≥ 0	Can be negative
Units	bits/symbol	bits/sample (not absolute info)
Max for fixed variance	Equiprobable distribution	Gaussian distribution

Properties of Differential Entropy Medium

1. h(X) can be **negative**, zero, or positive (unlike discrete entropy which is always ≥ 0).
2. h(X + c) = h(X) for any constant c (translation/shifting does not change differential entropy).
3. h(aX) = h(X) + $\log_2|a|$ (scaling changes differential entropy by log of the scale factor).
4. Among all continuous distributions with a given variance σ^2, the **Gaussian distribution has maximum differential entropy**.

Joint & Conditional Differential Entropy Low ⭐

$$h(X,Y) = -\iint f(x,y) \cdot \log_2 f(x,y)\, dx\, dy \quad h(X|Y) = -\iint f(x,y) \cdot \log_2 f(x|y)\, dx\, dy$$

Mutual Information (continuous): $I(X;Y) = h(X) - h(X|Y)$

HINGLISH

Bilkul discrete case jaisa hi pattern — sirf summation (Σ) ko integration (∫) se replace kar do, aur probability P ko density f(x) se. Concept same hai, sirf continuous variable ke liye adapted.

4. Source Coding Theorem (Shannon's First Theorem) High ⭐⭐⭐⭐

ENGLISH

Shannon's Source Coding Theorem states that for a discrete memoryless source with entropy H(X), it is possible to construct a code with average codeword length L_{avg} such that:

$$H(X) \leq L_{avg} < H(X) + 1$$

In other words: **average code length can never be less than the source entropy** — entropy is the fundamental lower bound on lossless compression.

HINGLISH

Yani — tum kisi bhi source ke data ko itna compress nahi kar sakte ki average bits/symbol entropy se kam ho jaaye. Entropy ek **"theoretical limit"** hai compression ka. Jaise ZIP/Huffman coding isi limit ke jitna close pohochne ki koshish karte hain.

Coding Efficiency & Redundancy

Numerical-friendly

$$\eta \text{ (Efficiency)} = H(X) / L_{avg} \qquad \text{Redundancy} = 1 - \eta$$

Higher efficiency (closer to 1, or 100%) means the code is closer to the theoretical entropy limit — better compression.

EXAMPLE

A source has H(X) = 1.75 bits/symbol. A code is designed with L_{avg} = 1.875 bits/symbol. Find efficiency and redundancy.

Solution:

η = 1.75/1.875 = 0.933 = **93.3%**

Redundancy = 1 − 0.933 = **0.067 (6.7%)**

Codes, Code Length & Kraft Inequality

Medium ⭐⭐

ENGLISH

A **code** assigns a sequence of binary digits (codeword) to each source symbol. Key code types:

- **Fixed-length code:** all codewords have equal length.
- **Variable-length code:** codeword length varies (shorter for high-probability symbols).
- **Uniquely decodable code:** no ambiguity — any coded sequence can be decoded in only one way.
- **Instantaneous (prefix) code:** no codeword is a prefix of another codeword — can be decoded symbol-by-symbol without lookahead.

Kraft Inequality: A prefix (instantaneous) code with codeword lengths $l_1, l_2, \ldots, l_n$ exists if and only if:

$$\Sigma_i \; 2^{-l_i} \leq 1$$

HINGLISH

Kraft inequality batati hai ki given lengths ke saath ek valid "instantaneous/prefix code" possible hai ya nahi — agar sab 2^{-l_i} ka sum ≤1 hai, to aisa code ban sakta hai jisme koi codeword kisi doosre ka prefix na ho (taaki decode karte waqt confusion na ho).

Huffman Coding — Constructing an Optimal Instantaneous Code

High ⭐ ⭐ ⭐ ⭐ — practical construction, frequently asked

ENGLISH

Huffman coding is a greedy algorithm that constructs an optimal (minimum L_{avg}) prefix code for a given set of symbol probabilities:

1. List all symbols in decreasing order of probability.
2. Combine the two **lowest**-probability symbols into a single node (sum of probabilities); assign one branch '0' and the other '1'.
3. Repeat step 2 with the new reduced list until only one node (probability 1) remains.
4. Read each symbol's codeword by tracing the tree from root to leaf.

HINGLISH

Huffman coding ka simple logic: jo symbol **sabse kam probable** hai, use **sabse lambi codeword** milegi, aur jo **sabse zyada probable** hai use **sabse chhoti codeword**. Har step pe do sabse chhoti probabilities ko "merge" karte jao jab tak sirf ek bache — yehi tree banata hai jisse codewords milte hain.

EXAMPLE (CONTINUING THE SAME 4-SYMBOL SOURCE)

For the source with $P(x_1)=0.5$, $P(x_2)=0.25$, $P(x_3)=0.125$, $P(x_4)=0.125$ ($H(X)=1.75$ bits/symbol, from Section 2), construct a Huffman code and verify its efficiency.

Solution — building the tree:

Step 1: Combine $x_3(0.125)$ & $x_4(0.125)$ → node A (0.25)

Step 2: Combine A(0.25) & $x_2(0.25)$ → node B (0.5)

Step 3: Combine B(0.5) & $x_1(0.5)$ → root (1.0)

SYMBOL	PROBABILITY	CODEWORD	LENGTH L_i
x_1	0.5	0	1
x_2	0.25	10	2
x_3	0.125	110	3
x_4	0.125	111	3

$\mathbf{L_{avg}} = \Sigma\, P_i l_i = 0.5(1)+0.25(2)+0.125(3)+0.125(3) = 0.5+0.5+0.375+0.375 =$ **1.75 bits/symbol**

Since $L_{avg} = H(X)$ exactly, **efficiency η = 100%** — this happens because all probabilities are exact negative powers of 2 ($0.5=2^{-1}$, $0.25=2^{-2}$, $0.125=2^{-3}$), which is the ideal case for Huffman/Shannon-Fano coding.

💡 Check the prefix property: {0, 10, 110, 111} — no codeword is a prefix of another, so it is instantaneously (uniquely) decodable. Also verify Kraft's inequality: $2^{-1}+2^{-2}+2^{-3}+2^{-3} = 0.5+0.25+0.125+0.125 = 1 \le 1$ ✓ (equality means the code is 100% efficient, no wasted code-space).

Proof Sketch of Source Coding Theorem

High ⭐⭐⭐ — 10-mark derivation

Lower bound ($L_{avg} \geq H(X)$): By Gibbs' inequality, for any two probability distributions P and Q: $-\Sigma P_i \log_2 P_i \leq -\Sigma P_i \log_2 Q_i$. Choosing $Q_i = 2^{-l_i}/\Sigma 2^{-l_j}$ and using the Kraft inequality ($\Sigma 2^{-l_i} \leq 1$), it follows algebraically that:

$$L_{avg} = \Sigma\ P_i\ l_i \geq H(X)$$

Upper bound ($L_{avg} < H(X)+1$): Choose codeword length $l_i = \lceil \log_2(1/P_i) \rceil$ (smallest integer $\geq \log_2(1/P_i)$). This satisfies the Kraft inequality (so a valid prefix code exists), and gives:

$$\log_2(1/P_i) \leq l_i < \log_2(1/P_i) + 1 \implies H(X) \leq L_{avg} < H(X) + 1$$

Combining both bounds proves Shannon's Source Coding Theorem: entropy is the achievable lower limit of average code length, approachable within 1 bit by simple symbol-by-symbol coding (and exactly approachable using block/extension coding as block length → ∞).

5. Discrete Memoryless Channel (DMC)

High ⭐⭐⭐

ENGLISH

A **Discrete Memoryless Channel** is a channel whose output at any time depends only on the input at that same time instant (not on past inputs/outputs), and both input X and output Y take values from finite discrete alphabets. It is described by a **transition probability matrix** $P(y_j|x_i)$.

HINGLISH

"Memoryless" matlab — channel ka current output sirf current input pe depend karta hai, purane inputs/outputs ka kuch effect nahi (no memory of past). Ye ek mathematical model hai jo channel ki "noise behavior" ko probability matrix se represent karta hai.

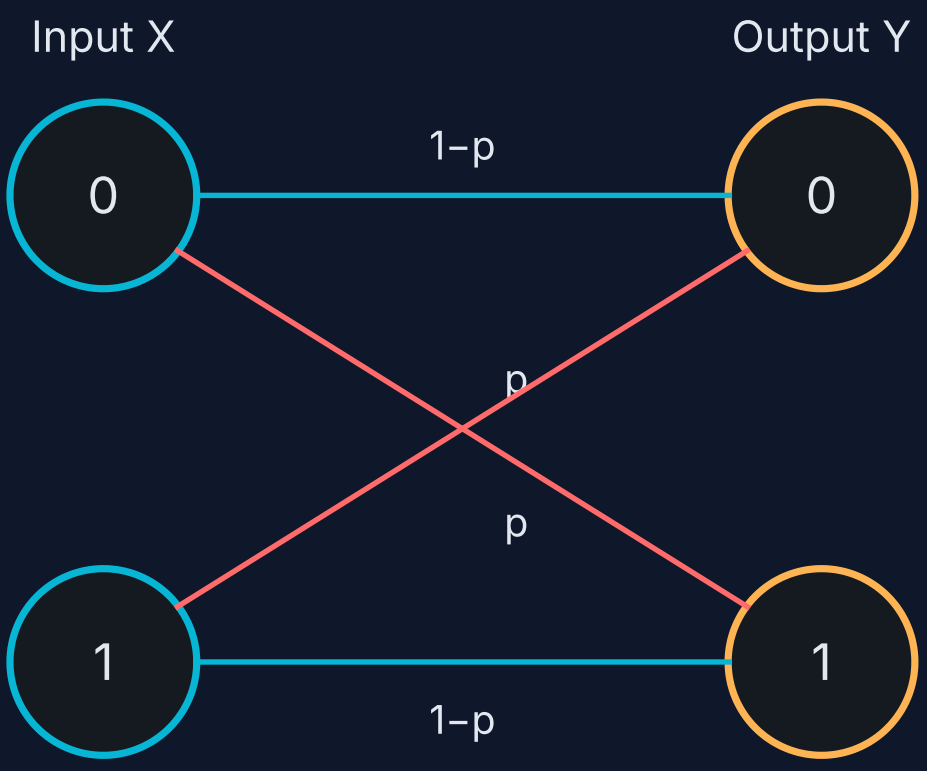


Fig 2.2 — Binary Symmetric Channel (BSC): the simplest and most-asked DMC example. 'p' = crossover (error) probability

Channel Matrix / Transition Probability Matrix

Medium

A DMC is fully described by its channel matrix, where rows = inputs, columns = outputs, each entry = $P(y_j|x_i)$:

$$[P(Y|X)] = \begin{vmatrix} P(y_1|x_1) & P(y_2|x_1) \\ P(y_1|x_2) & P(y_2|x_2) \end{vmatrix}$$

Each row sums to 1 (since for a fixed input, output probabilities over all possibilities must total 1). For BSC: [[1−p, p],[p, 1−p]].

Types of Discrete Channels

Medium ⭐ ⭐ — definition based

CHANNEL TYPE	PROPERTY	CHANNEL MATRIX FEATURE

Lossless Channel	No information is lost; $H(X\|Y)=0$	Only one non-zero entry per column
Deterministic Channel	Each input maps to exactly one output; $H(Y\|X)=0$	Only one non-zero entry (=1) per row
Noiseless Channel	Both lossless and deterministic; $I(X;Y)=H(X)=H(Y)$	Identity matrix (square, diagonal)
Binary Symmetric Channel (BSC)	Symmetric error probability p in both directions	[[1−p,p],[p,1−p]]

HINGLISH

Lossless = output se input bilkul reverse-trace ho sakta hai (no info kho rahi). **Deterministic** = ek fixed input se hamesha same output aata hai (no randomness output side pe). **Noiseless** = best case, dono properties saath — channel perfectly clean hai.

EXAMPLE — FINDING P(Y) AND H(Y) FROM A CHANNEL MATRIX

A DMC has input probabilities $P(x_1)=0.6$, $P(x_2)=0.4$ and channel matrix $[P(Y|X)] = [[0.9, 0.1], [0.2, 0.8]]$ (row = input, column = output). Find $P(y_1)$, $P(y_2)$ and H(Y).

Solution:

Output probabilities are found using total probability: $P(y_j) = \Sigma_i P(x_i)\cdot P(y_j|x_i)$

$P(y_1) = P(x_1)P(y_1|x_1) + P(x_2)P(y_1|x_2) = 0.6(0.9) + 0.4(0.2) = 0.54 + 0.08 =$ **0.62**

$P(y_2) = P(x_1)P(y_2|x_1) + P(x_2)P(y_2|x_2) = 0.6(0.1) + 0.4(0.8) = 0.06 + 0.32 =$ **0.38**

Check: $P(y_1)+P(y_2) = 0.62+0.38 = 1$ ✓

$H(Y) = -[0.62\log_2 0.62 + 0.38\log_2 0.38] = -[0.62(-0.690)+0.38(-1.396)] = 0.428+0.530 =$ **0.958 bits**

💡 Exam tip: write P(Y) as a row vector = [P(X)] × [Channel Matrix] (matrix multiplication) — this is the fastest way to get output probabilities for any DMC, not just a BSC.

Channel Capacity

High ⭐ ⭐ ⭐ ⭐ — frequently asked with BSC numerical

ENGLISH

Channel Capacity C is the maximum rate of reliable information transmission over a channel, defined as the maximum of mutual information over all possible input probability distributions P(x):

$$C = \max_{P(X)} I(X;Y) \quad \text{[bits/symbol or bits/channel use]}$$

For a **Binary Symmetric Channel** with crossover probability p, capacity is maximized at P(0)=P(1)=0.5 and equals:

$$C_{BSC} = 1 - H(p) = 1 + p \cdot \log_2 p + (1-p) \cdot \log_2 (1-p)$$

HINGLISH

Channel capacity = channel ka "speed limit" — usse zyada rate pe reliably (error-free) data nahi bhej sakte (ye Shannon ka famous result hai, jise "Shannon Limit" bhi kehte hain). BSC ke liye capacity formula bahut common numerical hai — agar p=0 (no error), C=1 bit/symbol (perfect channel); agar p=0.5 (channel totally random), C=0 (channel useless).

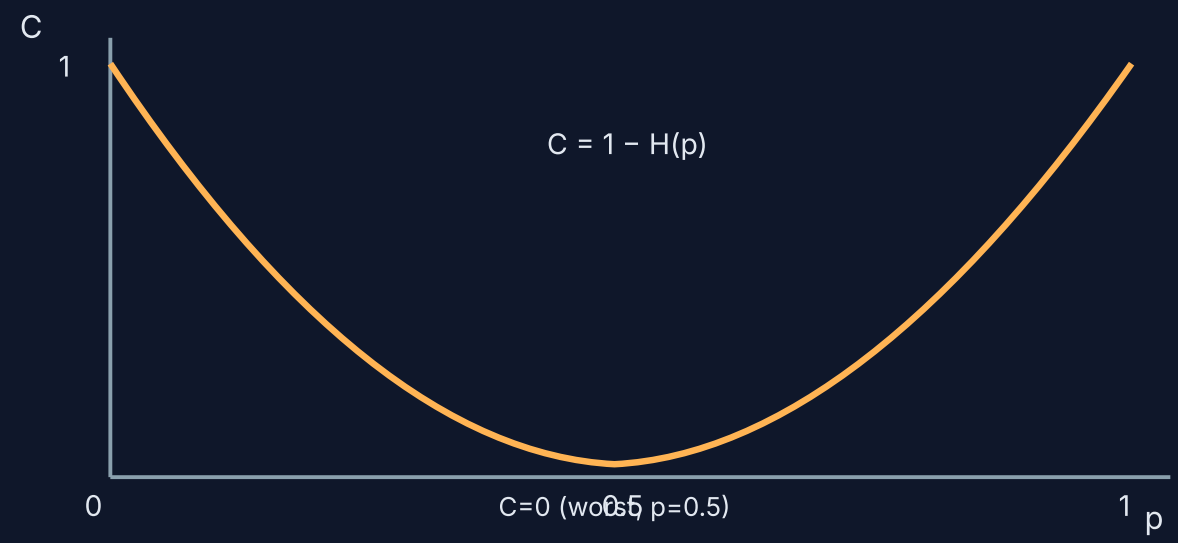


Fig 2.4 — BSC Capacity vs crossover probability p. C=1 at p=0 or p=1 (no ambiguity), C=0 at p=0.5 (max confusion)

🧪 Interactive Simulation — BSC Capacity Explorer Try it

Drag the slider to change crossover probability p and watch H(p) and channel capacity C = 1−H(p) recompute instantly.

p = **0.10**

H(p) = 0.469 bits **C = 0.531 bits/use**

💡 A good, mostly-reliable channel — try dragging to p=0.5 to see the channel become completely useless (C=0).

6. Mutual Information High ⭐⭐⭐⭐

ENGLISH

Mutual Information I(X;Y) is the amount of information that one random variable (channel output Y) contains about another (channel input X). It measures the reduction in uncertainty about X after observing Y.

$$I(X;Y) = H(X) - H(X|Y) = H(Y) - H(Y|X) = H(X) + H(Y) - H(X,Y)$$

HINGLISH

Mutual information batata hai — receiver ne Y dekhne ke baad, X ke baare mein **kitni uncertainty kam ho gayi**. Agar channel "perfect" hai (no noise), to Y dekhke X 100% pata chal jaata hai → I(X;Y) = H(X) (maximum). Agar channel bahut noisy hai, I(X;Y) → 0 (Y se X ke baare mein kuch pata nahi chalta).

Properties of Mutual Information

1. I(X;Y) ≥ 0 (always non-negative).
2. I(X;Y) = I(Y;X) (symmetric).
3. I(X;Y) = 0, if and only if X and Y are statistically independent.
4. I(X;Y) ≤ min[H(X), H(Y)].

Proof: I(X;Y) ≥ 0

Medium ⭐ ⭐ — proof-based question

Using $I(X;Y) = \Sigma\Sigma\ P(x_i,y_j)\ \log_2[P(x_i,y_j)/(P(x_i)P(y_j))]$, apply Jensen's/Gibbs' inequality (log is concave, and treating $P(x_i)P(y_j)$ as an alternate distribution Q over the same support):

$$-I(X;Y) = \Sigma\Sigma\ P(x_i,y_j)\ \log_2[P(x_i)P(y_j)/P(x_i,y_j)] \leq \log_2[\Sigma\Sigma\ P(x_i)P(y_j)]$$ (by Jensen's inequality, since log is concave) $= \log_2[1] = 0 \Rightarrow I(X;Y) \geq 0$ (equality holds iff X,Y are independent)

EXAMPLE — MUTUAL INFORMATION FOR AN ASYMMETRIC CHANNEL

Using the same channel from the earlier example — $P(x_1)=0.6$, $P(x_2)=0.4$, matrix [[0.9,0.1], [0.2,0.8]], with $P(y_1)=0.62$, $P(y_2)=0.38$, $H(Y)=0.958$ bits — find $H(Y|X)$ and $I(X;Y)$.

Solution:

$H(Y|X) = \Sigma_i\ P(x_i)\cdot H(Y|X=x_i)$, where $H(Y|X=x_i)$ is the entropy of row i of the channel matrix:

$H(Y|x_1) = -[0.9\log_2 0.9 + 0.1\log_2 0.1] = 0.137 + 0.332 = 0.469$ bits

$H(Y|x_2) = -[0.2\log_2 0.2 + 0.8\log_2 0.8] = 0.464 + 0.257 = 0.722$ bits

$H(Y|X) = 0.6(0.469) + 0.4(0.722) = 0.281 + 0.289 =$ **0.570 bits**

$I(X;Y) = H(Y) - H(Y|X) = 0.958 - 0.570 =$ **0.388 bits**

💡 Exam shortcut: $H(Y|X)$ is a **weighted average of the row entropies** of the channel matrix, weighted by the input probabilities $P(x_i)$ — this works for any DMC, not just symmetric ones like BSC.

7. Conditional Entropy

High ⭐ ⭐ ⭐

ENGLISH

Conditional Entropy H(X|Y) is the average uncertainty remaining about X, after the output Y has been observed. It is also called **"equivocation"** — represents information lost in a noisy channel.

$$H(X|Y) = -\ \Sigma_i\ \Sigma_j\ P(x_i,y_j)\ \cdot\ \log_2\ P(x_i|y_j)$$

HINGLISH

H(X|Y) = "Y pata hone ke baad bhi X mein kitni confusion baaki hai". Isiliye ise **equivocation** bhi kehte hain — ye represent karta hai ki noise ki wajah se kitni info **"lost"** ho gayi transmission mein.

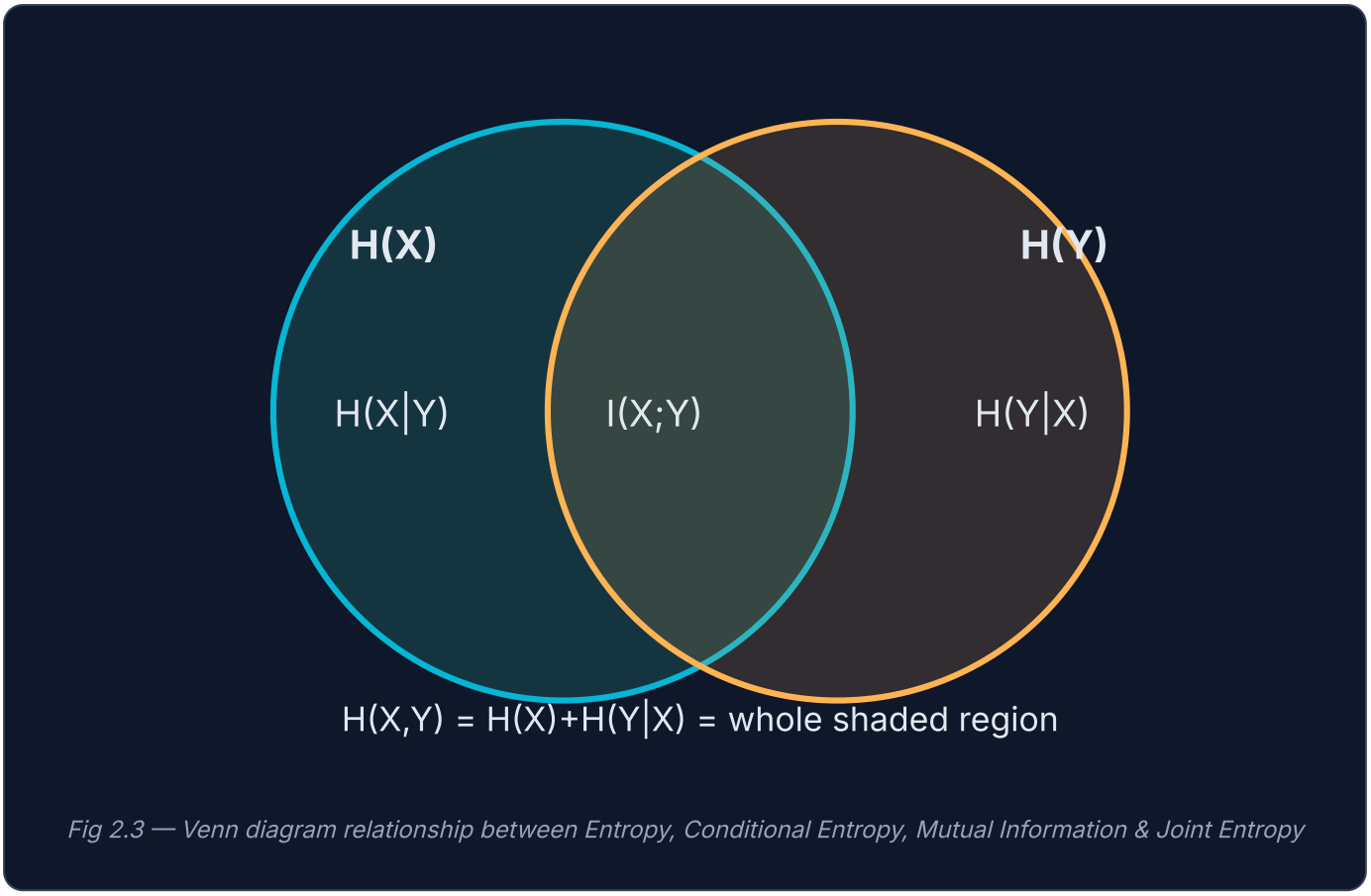


Fig 2.3 — Venn diagram relationship between Entropy, Conditional Entropy, Mutual Information & Joint Entropy

QUANTITY	MEANING	FORMULA
H(X)	Avg info / uncertainty of source	$-\Sigma P(x_i)\log_2 P(x_i)$
H(Y)	Avg info / uncertainty of output	$-\Sigma P(y_j)\log_2 P(y_j)$
H(X,Y)	Joint entropy	$-\Sigma\Sigma P(x_i,y_j)\log_2 P(x_i,y_j)$
H(X|Y)	Uncertainty in X after knowing Y (loss/equivocation)	H(X,Y) − H(Y)
H(Y|X)	Uncertainty in Y after knowing X (noise entropy)	H(X,Y) − H(X)
I(X;Y)	Info common to X & Y (gain through channel)	H(X) − H(X|Y)

EXAMPLE (CLASSIC RTU-STYLE)

For a BSC with P(0)=P(1)=0.5 and crossover probability p=0.1, find H(X), H(Y|X), and I(X;Y).

Solution:

$H(X) = -(0.5\log_2 0.5 + 0.5\log_2 0.5) =$ **1 bit**

$H(p) = -p\log_2 p - (1-p)\log_2(1-p) = -0.1\log_2 0.1 - 0.9\log_2 0.9 = 0.1(3.32)+0.9(0.152) =$ **0.469 bits**

Since the channel is symmetric and the input is equiprobable, the output is also equiprobable $\Rightarrow H(Y) = 1$ bit, and $H(Y|X) = H(p) =$ **0.469 bits**

$I(X;Y) = H(Y) - H(Y|X) = 1 - 0.469 =$ **0.531 bits** (≈53% of the source information survives transmission through this noisy channel per symbol)

💡 This equals the channel capacity $C = 1 - H(0.1)$ computed earlier for the same p — that's because equiprobable input is exactly the capacity-achieving distribution for a BSC, so at P(0)=P(1)=0.5 the mutual information already equals C.

Proof: H(X|Y) ≤ H(X)

Medium ⭐⭐ — proof question

Since I(X;Y) = H(X) − H(X|Y) and we proved I(X;Y) ≥ 0 above, it directly follows that:

```
H(X|Y) = H(X) - I(X;Y) ≤ H(X)
```

This makes intuitive sense: **knowing Y can never increase uncertainty about X on average** — at worst (when X,Y independent), H(X|Y)=H(X) (no help), and otherwise it strictly reduces uncertainty.

Full Worked Numerical — Joint Probability Matrix

High ★★★★★ — classic 10-mark question, asked almost every year

EXAMPLE

Given the joint probability matrix P(X,Y):

	Y_1	Y_2	$P(X_i)$
x_1	0.25	0.25	0.50
x_2	0.125	0.375	0.50
$P(y_j)$	0.375	0.625	1.0

Find H(X), H(Y), H(X,Y), H(X|Y), H(Y|X), and I(X;Y).

Solution:

① **H(X)** = −(0.5log₂0.5 + 0.5log₂0.5) = **1 bit** (marginal row sums used)

② **H(Y)** = −(0.375log₂0.375 + 0.625log₂0.625) = −(0.375×−1.415 + 0.625×−0.678) = 0.530+0.424 = **0.954 bits**

③ **H(X,Y)** = −Σ all four joint terms:

= −[0.25log₂0.25 + 0.25log₂0.25 + 0.125log₂0.125 + 0.375log₂0.375]

= −[0.25(−2)+0.25(−2)+0.125(−3)+0.375(−1.415)]

= 0.5+0.5+0.375+0.531 = **1.906 bits**

④ **H(X|Y)** = H(X,Y) − H(Y) = 1.906 − 0.954 = **0.952 bits**

⑤ **H(Y|X)** = H(X,Y) − H(X) = 1.906 − 1 = **0.906 bits**

⑥ **I(X;Y)** = H(X) − H(X|Y) = 1 − 0.952 = **0.048 bits** (cross-check: H(X)+H(Y)−H(X,Y) = 1+0.954−1.906 = 0.048 ✓)

HINGLISH TRICK

Entropy Venn Diagram: $H(X,Y)$ = Dono circles ka poora Union. **$I(X;Y)$** = Beech ka common Intersection (Shared info). **$H(X|Y)$** = Left ka bacha hua hissa (Y pata hone ke baad X ki uncertainty).

BSC Capacity: $C = 1 - H(p)$ (1 bit ideal capacity mein se noise entropy $H(p)$ minus kardo!).

🗺️ Unit 2 Overview Map

Quick Revise

UNIT 2: Information Measurement, Entropy & Channels

1. Information & Entropy

- **Self Information:** $I(x_i) = \log_2(1/p_i)$ bits
- **Entropy $H(X)$:** $\sum p_i \log_2(1/p_i)$ (Max $\log_2 M$ at equiprobable)
- **Continuous Entropy:** $h(X) = -\int f(x)\log_2 f(x)dx$

2. Joint & Conditional Entropy

- **Joint Entropy:** $H(X,Y) = -\sum \sum P(x_i, y_j)\log_2 P(x_i, y_j)$
- **Conditional:** $H(X|Y) = H(X,Y) - H(Y) \le H(X)$
- **Mutual Info:** $I(X;Y) = H(X) - H(X|Y) = H(Y) - H(Y|X)$

3. Discrete Memoryless Channels

- **DMC:** Channel matrix $[P(y_j|x_i)]$, memoryless property
- **BSC:** Binary Symmetric Channel with crossover p ($C = 1 - H(p)$)
- **BEC:** Binary Erasure Channel with erasure prob ϵ ($C = 1 - \epsilon$)

❓ Unit 2 — Q&A (RTU Exam Pattern)

Q1. Define entropy. State and explain its properties. ★★★★★ Very Important, 5-7 Marks

Definition: Entropy H(X) is the average (expected) amount of information produced per symbol by a discrete memoryless source. It is given by $H(X) = -\Sigma P(x_i) \cdot \log_2 P(x_i)$ bits/symbol, where the sum runs over all n symbols of the source alphabet.

Properties:

1. $0 \leq H(X) \leq \log_2 n$ — entropy is always bounded between zero and $\log_2 n$.
2. $H(X) = 0$ only when one symbol has probability 1 (source is fully predictable, no uncertainty).
3. $H(X)$ is maximum ($= \log_2 n$) when all n symbols are equiprobable ($P=1/n$ each) — this is the state of maximum uncertainty/randomness.
4. Entropy is a concave function of the probability distribution.

This topic is asked almost every year, usually paired with a numerical (see Q5) or the maximum-entropy proof (see Q10).

Q2. State and prove the Source Coding Theorem (Shannon's First Theorem).

★★★★ 7 Marks

Statement: For a discrete memoryless source with entropy H(X), it is always possible to construct a uniquely decodable (prefix) code with average codeword length L_{avg} satisfying $H(X) \le L_{avg} < H(X)+1$.

Proof (two parts): The lower bound $L_{avg} \ge H(X)$ follows from Gibbs' inequality applied with $Q_i=2^{-l_i}/\Sigma 2^{-l_j}$ together with Kraft's inequality. The upper bound follows by choosing $l_i = \lceil \log_2(1/P_i) \rceil$, which satisfies Kraft's inequality and gives $\log_2(1/P_i) \le l_i < \log_2(1/P_i)+1$; averaging over all symbols yields $H(X) \le L_{avg} < H(X)+1$. (Full step-by-step derivation is in the "Proof Sketch of Source Coding Theorem" section above.)

Efficiency & redundancy: $\eta = H(X)/L_{avg}$ (closer to 1 = better compression); Redundancy = $1-\eta$.

The theorem essentially says entropy is the fundamental limit of lossless compression — you can never compress below it on average.

Q3. Differentiate between Mutual Information and Conditional Entropy.

★★★ 5 Marks

Mutual Information I(X;Y): the amount of information one variable contains about another — i.e. how much the uncertainty about X reduces after observing Y. It is always ≥0, symmetric (I(X;Y)=I(Y;X)), and equals 0 only when X and Y are statistically independent. Formula: $I(X;Y) = H(X) - H(X|Y) = H(Y) - H(Y|X) = H(X)+H(Y)-H(X,Y)$.

Conditional Entropy H(X|Y): the average uncertainty that remains about X even after Y is known — also called "equivocation," since it represents information lost due to channel noise. Formula: $H(X|Y) = -\Sigma\Sigma P(x_i,y_j)\log_2 P(x_i|y_j) = H(X,Y)-H(Y)$.

Key relation linking both: $I(X;Y) = H(X) - H(X|Y)$. So mutual information is literally "what you gain," and conditional entropy is "what's still missing" — together they add up to H(X), the total original uncertainty.

Q4. What is a Discrete Memoryless Channel? Explain Binary Symmetric Channel (BSC) with diagram. ★★★★ 5-7 Marks

DMC: A Discrete Memoryless Channel is a channel model where the output at any instant depends only on the input at that same instant (no dependence on past inputs/outputs), and both input X and output Y take values from finite discrete alphabets. It is fully characterized by its transition probability matrix $P(y_j|x_i)$, where each row sums to 1.

Binary Symmetric Channel (BSC): the simplest and most commonly examined DMC — 2 inputs {0,1}, 2 outputs {0,1}, and a single crossover (error) probability p that applies symmetrically in both directions (see Fig 2.2). Its channel matrix is [[1−p, p],[p, 1−p]] — each input is correctly received with probability (1−p) and flipped with probability p.

BSC is the standard model for a noisy binary transmission link and forms the basis for almost every channel-capacity numerical in this unit (see Q17, Q18).

Q5. Numerical: A source has 4 symbols with probabilities 0.5, 0.25, 0.125, 0.125. Find entropy. ★★★★★ Almost guaranteed numerical

Solution:

$H(X) = -[0.5\log_2 0.5 + 0.25\log_2 0.25 + 0.125\log_2 0.125 + 0.125\log_2 0.125]$

$= -[0.5(-1) + 0.25(-2) + 0.125(-3) + 0.125(-3)]$

$= 0.5 + 0.5 + 0.375 + 0.375 =$ **1.75 bits/symbol**

This exact style of numerical — probabilities that are clean negative powers of 2 (½, ¼, ⅛, ⅛) — is the single most-repeated RTU question pattern for entropy, because the $\log_2$ values come out as whole numbers, making it fast to solve without a calculator. The same 4 probabilities are reused later for the Huffman coding numerical (Q22), where you'll see L_{avg} comes out exactly equal to this entropy value.

Q6. Explain differential entropy for continuous random variables. Derive it for Gaussian distribution. ★★★ 5 Marks

Definition: For a continuous random variable X with probability density function f(x), differential entropy is $h(X) = -\int f(x) \cdot \log_2 f(x) dx$, over the entire support of X. It is the continuous-variable analogue of discrete entropy, with the sum replaced by an integral and the pmf replaced by a pdf.

Gaussian derivation: for $X \sim N(\mu, \sigma^2)$, substituting the Gaussian pdf into the differential entropy integral and simplifying (using $E[(X-\mu)^2]=\sigma^2$) gives $h(X) = \frac{1}{2}\log_2(2\pi e\sigma^2)$. This is a standard result worth memorizing directly.

Important distinction: unlike discrete entropy, h(X) can be **negative** (e.g. for a uniform distribution narrower than 1 unit — see Q23), because it is not a true "information" measure in absolute terms, only a relative one. The Gaussian distribution has the special property of having the maximum differential entropy among all continuous distributions with a fixed variance σ^2.

Q7. Derive the relation I(X;Y) = H(X) – H(X|Y) = H(Y) – H(Y|X). ★★★ 5 Marks

Derivation: Start from the two equivalent expansions of joint entropy: $H(X,Y) = H(X) + H(Y|X)$ and $H(X,Y) = H(Y) + H(X|Y)$. Also recall the definition $I(X;Y) = H(X) + H(Y) - H(X,Y)$ (from the Venn-diagram relation in Fig 2.3).

Substituting $H(X,Y) = H(X)+H(Y|X)$ into the mutual-information definition: $I(X;Y) = H(X)+H(Y)-[H(X)+H(Y|X)] = H(Y)-H(Y|X)$.
Similarly substituting $H(X,Y) = H(Y)+H(X|Y)$: $I(X;Y) = H(X)+H(Y)-[H(Y)+H(X|Y)] = H(X)-H(X|Y)$.

Both forms are proven identical, confirming mutual information's symmetry. Draw the Venn diagram (Fig 2.3) as visual support — the overlapping region between the H(X) and H(Y) circles is exactly I(X;Y), and each circle minus the overlap gives the corresponding conditional entropy.

Q8. Explain coding efficiency and redundancy with a numerical example.

★★ 3-5 Marks

Definitions: Coding efficiency $\eta = H(X)/L_{avg}$ measures how close an actual code's average length is to the theoretical entropy limit ($\eta \leq 1$ always, by the Source Coding Theorem). Redundancy = $1-\eta$ measures the "wasted" bits per symbol beyond what's theoretically necessary.

Numerical: A source has $H(X) = 1.75$ bits/symbol and a code is designed with $L_{avg} = 1.875$ bits/symbol.
$\eta = 1.75/1.875 = 0.933 =$ **93.3%**
Redundancy = $1 - 0.933 =$ **0.067 (6.7%)**

Higher efficiency (closer to 100%) means the code design is close to optimal — a perfectly-designed Huffman code on powers-of-2 probabilities can even reach η=100% (see Q22).

Q9. Why is the logarithmic function used to measure information? State the postulates.

★★★ 5 Marks

Shannon required any sensible information measure I(P) to satisfy four postulates simultaneously:

1. **Postulate 1:** $I(P) = 0$ when $P = 1$ (a certain event carries zero information).
2. **Postulate 2:** $I(P) \geq 0$ for $0 \leq P \leq 1$ (information is never negative).
3. **Postulate 3:** $I(P_1) > I(P_2)$ if $P_1 < P_2$ (monotonically decreasing — rarer events carry more information).
4. **Postulate 4 (Additivity):** $I(P_1 \cdot P_2) = I(P_1) + I(P_2)$ for independent events — the joint information of two independent events equals the sum of their individual information.

The **only** function satisfying all four conditions at once is the logarithmic function, $I(x) = \log(1/P(x))$ — because $\log(a \cdot b) = \log(a)+\log(b)$ is precisely what makes additivity work for independent events, while no other elementary function has this property alongside the other three postulates. This is why entropy and information are always defined using logarithms.

Q10. Prove that entropy is maximum when all symbols are equiprobable.

★★★★ 7 Marks — Lagrange multiplier proof

Setup: Maximize $H(X) = -\Sigma P_i \log_2 P_i$ subject to the constraint $\Sigma P_i = 1$, using Lagrange multipliers.

Derivation: Form $L = -\Sigma P_i \log_2 P_i + \lambda(\Sigma P_i - 1)$. Differentiating with respect to each P_i and setting to zero: $\partial L/\partial P_i = -\log_2 P_i - \log_2 e + \lambda = 0$, which gives the same value of P_i for every i — i.e. P_i = constant = 1/n for all i = 1...n.

Conclusion: Substituting P_i=1/n back into H(X) gives $H_{max}(X) = \log_2 n$. Hence entropy reaches its maximum value exactly when the probability distribution is uniform (equiprobable) — this is the point of maximum uncertainty, since no symbol is favored over another.

Q11. Define Joint Entropy and Marginal Entropy. State the relation between them.

★★ 3-5 Marks

Joint Entropy H(X,Y): the average uncertainty associated with a pair of random variables (X,Y) taken together, considering their joint probability distribution: $H(X,Y) = -\Sigma_i \Sigma_j P(x_i,y_j) \log_2 P(x_i,y_j)$.

Marginal Entropy: simply H(X) or H(Y) computed individually, using the marginal probability distributions obtained by summing the joint matrix along rows ($P(x_i)=\Sigma_j P(x_i,y_j)$) or columns ($P(y_j)=\Sigma_i P(x_i,y_j)$).

Relation: $H(X,Y) = H(X) + H(Y|X) = H(Y) + H(X|Y)$ — the joint uncertainty of both variables equals the uncertainty of one plus the remaining (conditional) uncertainty of the other given the first is known.

Q12. What is rate of information transmission? Give its formula.

★★ 2-3 Marks

If a discrete source emits r symbols per second, and each symbol carries an average of H(X) bits of information, then the rate of information transmission R (in bits/second) is: **$R = r \cdot H(X)$**.

This simply scales the "per-symbol" entropy up to a "per-second" rate, using the source's symbol-emission speed r. It becomes important later when comparing a source's actual information rate against a channel's capacity C, to check whether reliable transmission ($R \leq C$) is achievable.

Q13. List and explain the properties of differential entropy. How does it differ from discrete entropy? ★★★ 5 Marks

Properties of differential entropy h(X):

1. h(X) can be negative, zero, or positive — unlike discrete entropy which is always ≥ 0.
2. h(X+c) = h(X) for any constant c — shifting/translating the distribution does not change its differential entropy.
3. h(aX) = h(X) + $\log_2|a|$ — scaling the variable by a changes h(X) by $\log_2|a|$.
4. Among all continuous distributions with a fixed variance σ^2, the Gaussian distribution has the maximum differential entropy.

Difference from discrete entropy: discrete entropy uses a pmf and summation and is always non-negative with units of bits/symbol; differential entropy uses a pdf and integration, can be negative, and its value depends on the units/scale of X (not an absolute measure of information in the same sense as discrete entropy).

Q14. Define instantaneous (prefix) code and state Kraft's inequality. ★★★ 5 Marks

Instantaneous (prefix) code: a variable-length code in which no codeword is a prefix of any other codeword. This property allows the receiver to decode symbol-by-symbol as bits arrive, without needing to look ahead or wait for a separator.

Kraft's Inequality: a prefix code with codeword lengths $l_1, l_2, \ldots, l_n$ exists if and only if $\Sigma_i 2^{-l_i} \leq 1$. This is both a necessary and sufficient condition — given any set of lengths satisfying this inequality, a valid instantaneous code can always be constructed (and conversely, any instantaneous code's lengths must satisfy it).

When $\Sigma_i 2^{-l_i} = 1$ exactly, the code uses all available code-space with no waste, which corresponds to 100% coding efficiency (see the Huffman example in Q22, where $2^{-1}+2^{-2}+2^{-3}+2^{-3} = 1$).

Q15. Prove Shannon's Source Coding Theorem: $H(X) \leq L_{avg} < H(X)+1$.

★★★★★ 10 Marks — long derivation, very important

Part 1 — Lower bound ($L_{avg} \geq H(X)$): By Gibbs' inequality, for any two probability distributions P and Q: $-\Sigma P_i \log_2 P_i \leq -\Sigma P_i \log_2 Q_i$. Choosing $Q_i = 2^{-l_i}/\Sigma 2^{-l_j}$ and applying Kraft's inequality ($\Sigma 2^{-l_i} \leq 1$), algebraic simplification shows $L_{avg} = \Sigma P_i l_i \geq H(X)$.

Part 2 — Upper bound ($L_{avg} < H(X)+1$): Choose codeword length $l_i = \lceil \log_2(1/P_i) \rceil$ (the smallest integer $\geq \log_2(1/P_i)$). This choice always satisfies Kraft's inequality (so a valid prefix code exists for these lengths), and by definition of the ceiling function: $\log_2(1/P_i) \leq l_i < \log_2(1/P_i)+1$. Taking the expectation (weighted average) over all symbols gives $H(X) \leq L_{avg} < H(X)+1$.

Conclusion: Combining both bounds proves the theorem — entropy is the achievable lower limit of average code length, and it can always be approached within 1 bit per symbol using simple symbol-by-symbol coding, and exactly achieved (in the limit) using block/extension coding as block length → ∞.

Q16. Explain the different types of discrete channels (lossless, deterministic, noiseless) with their channel matrices.

★★★ 5 Marks

Lossless Channel: no information about the input is lost, so $H(X|Y)=0$ (the input can always be perfectly reconstructed from the output). Its channel matrix has exactly one non-zero entry per column.

Deterministic Channel: each input maps to exactly one specific output with certainty, so $H(Y|X)=0$ (no randomness on the output side). Its channel matrix has exactly one non-zero entry (equal to 1) per row.

Noiseless Channel: satisfies both properties simultaneously — it is both lossless and deterministic, giving $I(X;Y)=H(X)=H(Y)$. Its channel matrix is a square identity matrix (1s on the diagonal, 0s elsewhere) — the "ideal" channel with zero information loss in either direction.

The BSC (Q4) is a special symmetric case that is generally *neither* lossless nor deterministic (unless p=0 or p=1), which is why it's the standard example for studying real, noisy channels.

Q17. Define channel capacity. Derive the channel capacity of a Binary Symmetric Channel.

★★★★★ 7-10 Marks — very important, numerical + theory combined

Definition: Channel capacity C is the maximum possible rate of reliable information transmission over a channel, defined as the maximum of mutual information I(X;Y) over all possible input probability distributions P(x): $C = \max_{P(x)} I(X;Y)$, measured in bits/symbol (or bits/channel use).

Derivation for BSC: For a BSC with crossover probability p, mutual information I(X;Y) = H(Y) −H(Y|X). Since H(Y|X)=H(p) regardless of the input distribution (a fixed property of the channel), maximizing I(X;Y) means maximizing H(Y), which is largest (=1 bit) precisely when the output is equiprobable — which happens when the input is equiprobable, P(0)=P(1)=0.5. This gives:

$C_{BSC} = 1 - H(p)$, where $H(p) = -p \cdot \log_2 p - (1-p) \cdot \log_2 (1-p)$.

As shown in Fig 2.4: C=1 bit/symbol at p=0 or p=1 (channel is perfectly reliable, either straight-through or fully inverted), and C=0 at p=0.5 (channel output carries zero information about the input — completely useless). Try the interactive BSC Capacity Explorer simulation above to see this relationship live.

Q18. Numerical: For a BSC with p=0.2, find the channel capacity.

★★★★ 5 Marks numerical

Solution:

$H(0.2) = -0.2\log_2 0.2 - 0.8\log_2 0.8 = 0.2(2.322) + 0.8(0.322) = 0.464 + 0.257 =$ **0.722 bits**

$C = 1 - H(0.2) = 1 - 0.722 =$ **0.278 bits/symbol**

This means that despite 20% of bits being flipped by the channel, up to 0.278 bits of reliable information can still theoretically be transmitted per channel use, using an appropriately designed error-correcting code (Shannon's Channel Coding Theorem guarantees this is achievable, though it doesn't specify the exact code to use).

Q19. Prove that mutual information I(X;Y) is always non-negative. ★★★ 5 Marks proof

Proof: Starting from $I(X;Y) = \Sigma\Sigma P(x_i,y_j)\log_2[P(x_i,y_j)/(P(x_i)P(y_j))]$, consider $-I(X;Y)$ and apply Jensen's inequality (log is a concave function), treating $P(x_i)P(y_j)$ as an alternate distribution over the same support as $P(x_i,y_j)$:

$$-I(X;Y) = \Sigma\Sigma P(x_i,y_j)\log_2[P(x_i)P(y_j)/P(x_i,y_j)] \leq \log_2[\Sigma\Sigma P(x_i)P(y_j)] = \log_2[1] = 0$$

Therefore $I(X;Y) \geq 0$, with equality holding if and only if X and Y are statistically independent (i.e. $P(x_i,y_j)=P(x_i)P(y_j)$ for all i,j, making the log term zero everywhere). This confirms that observing one variable can never give "negative" information about another — at worst it gives zero (no help at all).

Q20. Show that H(X|Y) ≤ H(X). What does this imply physically? ★★★ 5 Marks

Proof: Since $I(X;Y) = H(X) - H(X|Y)$, rearranging gives $H(X|Y) = H(X) - I(X;Y)$. Because $I(X;Y) \geq 0$ was already proven (Q19), it directly follows that $H(X|Y) \leq H(X)$.

Physical meaning: observing the channel output Y can never, on average, increase your uncertainty about the input X. At worst (when X and Y are independent, i.e. Y tells you nothing about X), $H(X|Y)=H(X)$ — no help, but also no harm. In every other case, knowing Y strictly reduces your uncertainty about X. This is sometimes summarized as "side information never hurts."

Q21. Numerical (10 Marks — most important pattern): Given a joint probability matrix P(X,Y), compute H(X), H(Y), H(X,Y), H(X|Y), H(Y|X) and I(X;Y).

★★★★★ Guaranteed-type question

Sample matrix: x_1 row: $P(x_1,y_1)=0.25$, $P(x_1,y_2)=0.25$ (row sum $P(x_1)=0.50$); x_2 row: $P(x_2,y_1)=0.125$, $P(x_2,y_2)=0.375$ (row sum $P(x_2)=0.50$); column sums $P(y_1)=0.375$, $P(y_2)=0.625$.

Step-by-step solution (always follow this exact order):

1. $H(X) = -(0.5\log_2 0.5+0.5\log_2 0.5)$ = **1 bit** (from row/marginal sums)
2. $H(Y) = -(0.375\log_2 0.375+0.625\log_2 0.625) = 0.530+0.424$ = **0.954 bits**
3. $H(X,Y) = -[0.25\log_2 0.25+0.25\log_2 0.25+0.125\log_2 0.125+0.375\log_2 0.375] =$ $0.5+0.5+0.375+0.531$ = **1.906 bits** (compute this directly from all four joint entries first)
4. $H(X|Y) = H(X,Y) - H(Y) = 1.906 - 0.954$ = **0.952 bits**
5. $H(Y|X) = H(X,Y) - H(X) = 1.906 - 1$ = **0.906 bits**
6. $I(X;Y) = H(X) - H(X|Y) = 1 - 0.952$ = **0.048 bits** (cross-check: $H(X)+H(Y)-H(X,Y) =$ $1+0.954-1.906 = 0.048$ ✓)

💡 Exam tip: always compute H(X,Y) directly from the joint matrix *first* — every other quantity follows from it using the standard relations, which is the fastest, most error-free order to solve this type of numerical for any matrix given in the exam.

Q22. Construct a Huffman code for a source with 4 symbols of probabilities 0.5, 0.25, 0.125, 0.125. Find L_{avg} and efficiency. ★★★★ 7 Marks — construction-based

Tree construction: Combine the two lowest-probability symbols repeatedly: Step 1 — combine $x_3(0.125)$ & $x_4(0.125)$ → node A(0.25). Step 2 — combine A(0.25) & $x_2(0.25)$ → node B(0.5). Step 3 — combine B(0.5) & $x_1(0.5)$ → root(1.0).

Resulting codewords: x_1="0" (length 1), x_2="10" (length 2), x_3="110" (length 3), x_4="111" (length 3).

$L_{avg} = \Sigma P_i l_i = 0.5(1)+0.25(2)+0.125(3)+0.125(3) = 0.5+0.5+0.375+0.375 =$ **1.75 bits/symbol**, which equals H(X) exactly — so **efficiency η = 100%**. This happens because every probability here is an exact negative power of 2 ($0.5=2^{-1}$, $0.25=2^{-2}$, $0.125=2^{-3}$), which is the ideal case for Huffman coding.

Verify: prefix property holds ({0,10,110,111} — no codeword is a prefix of another), and Kraft's inequality gives $2^{-1}+2^{-2}+2^{-3}+2^{-3} = 1 \leq 1$ (equality ⟹ no wasted code-space).

Q23. Find the differential entropy of a random variable uniformly distributed over [0, 8]. ★★★ 5 Marks numerical

Solution: For a uniform distribution over [a,b], f(x)=1/(b−a), so h(X) = $-\int_a^b (1/(b-a))\cdot\log_2(1/(b-a))dx = \log_2(b-a)$. Here (b−a)=8, so h(X) = $\log_2 8$ = **3 bits**.

General shortcut: for uniform X on [a,b], h(X) = $\log_2(b-a)$ directly — no need to redo the integral each time. Note that if (b−a) < 1, h(X) becomes **negative**, which is a perfectly valid result for differential entropy (unlike discrete entropy, which is never negative — see Q13).

Q24. Numerical: Given $P(x_1)=0.6$, $P(x_2)=0.4$ and channel matrix [[0.9,0.1], [0.2,0.8]], find P(Y), H(Y), H(Y|X) and I(X;Y).

★★★★★ 10 Marks — asymmetric/non-BSC channel pattern

Step 1 — Output probabilities using $P(y_j)=\Sigma_i P(x_i)P(y_j|x_i)$, or equivalently [P(X)]×[Channel Matrix]:

$P(y_1) = 0.6(0.9)+0.4(0.2) = 0.54+0.08 =$ **0.62**

$P(y_2) = 0.6(0.1)+0.4(0.8) = 0.06+0.32 =$ **0.38** (check: 0.62+0.38=1 ✓)

Step 2 — H(Y) $= -[0.62\log_2 0.62+0.38\log_2 0.38] = 0.428+0.530 =$ **0.958 bits**

Step 3 — H(Y|X), the weighted average of each row's entropy:

$H(Y|x_1) = -[0.9\log_2 0.9+0.1\log_2 0.1] = 0.137+0.332 = 0.469$ bits

$H(Y|x_2) = -[0.2\log_2 0.2+0.8\log_2 0.8] = 0.464+0.257 = 0.722$ bits

$H(Y|X) = 0.6(0.469)+0.4(0.722) = 0.281+0.289 =$ **0.570 bits**

Step 4 — I(X;Y) $= H(Y) - H(Y|X) = 0.958 - 0.570 =$ **0.388 bits**

💡 Exam shortcut: H(Y|X) is always a probability-weighted average of the row entropies of the channel matrix — this works for *any* DMC, symmetric or not, so it's more general than memorizing the BSC-only H(p) shortcut.

📕 UNIT 3 — Source Coding Schemes for Data Compaction

Prefix Code · Huffman Code · Shannon-Fano Code · Lempel-Ziv Coding · Channel Capacity · Channel Coding Theorem · Shannon Limit | Hours: 05

📋 Quick Navigation — Unit 3

1. Data Compaction — Recap intro
2. Prefix Code (Instantaneous Code) & Kraft's Inequality high ⭐⭐⭐
3. Huffman Coding — algorithm + 2 full numericals high ⭐⭐⭐⭐⭐
4. Shannon-Fano Coding — algorithm + numerical + Huffman vs SF high ⭐⭐⭐⭐
5. Lempel-Ziv Coding — LZ77 & LZ78 high ⭐⭐⭐
6. Channel Capacity — Shannon-Hartley theorem + 3 numericals high ⭐⭐⭐⭐⭐
7. Channel Coding Theorem high ⭐⭐⭐⭐⭐
8. Shannon Limit high ⭐⭐⭐⭐
9. Formula Sheet — Quick Revision revision
10. 🧪 Interactive Simulations (Huffman Builder + Capacity Calculator) try it
11. Q&A Bank (RTU Exam Pattern) — 16 Qs exam prep

1. Data Compaction — Recap

Low ⭐ — intro

ENGLISH

Data compaction (source coding) is the process of encoding source symbols using codewords of variable length, such that symbols with higher probability get shorter codewords and rarer symbols get longer codewords — reducing the average number of bits/symbol close to the source entropy H(X).

HINGLISH

Simple idea: jo letter/symbol **zyada baar aata hai** usko **chhota code** do, jo **kam aata hai** usko **lamba code** do. Isse average length kam ho jaati hai — yahi compression ka core idea hai (Morse code mein bhi 'E' ek dot hai kyunki English mein sabse common letter hai!).

💡 Two big families exist: (1) **Statistical / entropy coders** — Huffman, Shannon-Fano — need symbol probabilities in advance and code single symbols. (2) **Dictionary / adaptive coders** — Lempel-Ziv family — need no prior probability table and code repeated sequences on the fly. RTU almost always asks you to compare these two families directly, so keep this distinction sharp.

2. Prefix Code (Instantaneous Code) High ★★★

ENGLISH

A **prefix code** (instantaneous code) is a variable-length code in which **no codeword is a prefix of any other codeword**. This property allows the receiver to decode each symbol the moment its complete codeword is received — without needing to look ahead at future bits.

HINGLISH

Prefix code ka matlab — koi bhi chhota codeword kisi bade codeword ka "starting part" nahi hona chahiye. Isse decoding karte waqt confusion nahi hota — jaise hi codeword complete hota hai, turant pata chal jaata hai ki symbol kya hai, aage ke bits dekhne ki zarurat nahi (isiliye "instantaneous" kehte hain).

EXAMPLE

Valid prefix code: A=0, B=10, C=110, D=111 → no code is prefix of another ✓
Invalid (not prefix-free): A=0, B=01, C=011 → here '0' (code for A) is a prefix of '01' (code for B) ✗ — decoding becomes ambiguous.
Decoding walkthrough (valid code above): received bitstream `0 110 10 111` → read bit by bit: "0" instantly matches A → decode A, reset; "1","1","0" matches C → decode C, reset; "1","0" matches B → decode B, reset; "1","1","1" matches D → decode D. Final message = **A C B D**, decoded uniquely with zero lookahead.

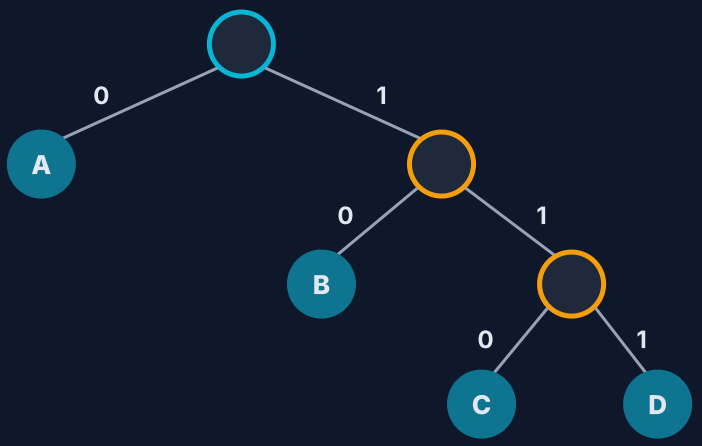


Fig 3.1 — Binary Code Tree for prefix code A=0,B=10,C=110,D=111: teal nodes are leaves (carry a symbol), slate nodes are internal branch points — only leaf nodes carry symbols, so no codeword can be a prefix of another

Kraft's Inequality — Testing if a Prefix Code Can Exist

Medium ⭐ ⭐ — often asked as short numerical

ENGLISH

For a binary prefix code with codeword lengths l_1, l_2, ..., l_n, a prefix code exists **if and only if** the **Kraft Inequality** holds:

$$\Sigma\ 2^{-l_i} \leq 1$$

HINGLISH

Ye ek quick "sanity test" hai — agar tumhe sirf codeword lengths (0,10,110,111 jaise actual codes nahi, sirf lengths 1,2,3,3) di gayi hon, to Kraft inequality laga ke pata chal jaata hai ki inn lengths ka koi valid prefix code ban sakta hai ya nahi, bina tree banaye. Agar sum ≤ 1 hai to code possible hai, agar > 1 hai to **impossible** hai.

NUMERICAL

Check whether lengths {1, 2, 3, 3} can form a valid binary prefix code.

Solution: $\Sigma 2^{-l_i} = 2^{-1}+2^{-2}+2^{-3}+2^{-3} = 0.5+0.25+0.125+0.125 =$ **1.0 ≤ 1** ✓ → a valid prefix code exists (this is exactly the A=0,B=10,C=110,D=111 tree above — it's "full", nothing wasted).

Second check: lengths {1,1,2} → $\Sigma = 0.5+0.5+0.25 =$ **1.25 > 1** ✗ → no valid binary prefix code can have these lengths (two length-1 codes 0 and 1 already exhaust the whole tree, leaving no room for a third symbol).

3. Huffman Coding

High ⭐⭐⭐⭐⭐ (Most Important — numerical guaranteed)

ENGLISH

Huffman coding is an optimal variable-length prefix coding technique that builds a binary tree bottom-up by repeatedly combining the two least-probable symbols/nodes into a new node, until a single root remains. It achieves the minimum possible average codeword length for a given symbol probability distribution (among all prefix codes).

HINGLISH

Huffman coding ka logic — har step pe sabse **2 kam probability** wale symbols/nodes ko utha ke unko jod do (ek naya combined node bana do), aur ye repeat karte raho jab tak sirf ek root node na bach jaaye. Ye process "optimal" code deta hai — matlab sabse kam possible average length.

Huffman Coding Algorithm

High — write this step-by-step in exam

1. List all symbols with their probabilities in **descending order**.
2. Take the **two symbols with lowest probability**; combine them into a new node with probability = sum of the two.
3. Assign bit '0' to one branch and '1' to the other (convention).
4. Re-sort the list (treating combined node as a new "symbol") and **repeat steps 2-3** until only one node (probability = 1) remains — this is the root.
5. The codeword for each original symbol = the sequence of 0s/1s read from root to that leaf.

FULL WORKED EXAMPLE 1 — MOST COMMON NUMERICAL PATTERN

Symbols: A(0.4), B(0.2), C(0.2), D(0.1), E(0.1). Construct Huffman code and find average code length & efficiency.

Step-by-step combination:

Step 1: D(0.1)+E(0.1) → DE(0.2)

Step 2: Now: A(0.4), B(0.2), C(0.2), DE(0.2) → combine two lowest: C(0.2)+DE(0.2) → CDE(0.4)

Step 3: Now: A(0.4), B(0.2), CDE(0.4) → combine B(0.2)+CDE(0.4) → BCDE(0.6)

Step 4: Now: A(0.4), BCDE(0.6) → combine → root(1.0)

Resulting codewords (one valid assignment):

SYMBOL	PROBABILITY	CODEWORD	LENGTH
A	0.4	0	1
B	0.2	10	2
C	0.2	110	3
D	0.1	1110	4
E	0.1	1111	4

Average code length:

$L_{avg} = \Sigma P_i l_i = 0.4(1)+0.2(2)+0.2(3)+0.1(4)+0.1(4) = 0.4+0.4+0.6+0.4+0.4 =$ **2.2 bits/symbol**

Entropy: $H(X) = -\Sigma P_i \log_2 P_i = 0.4(1.32)+0.2(2.32)+0.2(2.32)+0.1(3.32)+0.1(3.32) \approx$ **2.122 bits/symbol**

Efficiency: $\eta = H(X)/L_{avg} = 2.122/2.2 =$ **96.5%** | **Redundancy** = 1−0.965 = 3.5%

Variance check (why 0-4 assignment vs 1-4?): the choice of which branch gets '0' and which gets '1' at each merge is arbitrary and doesn't change L_{avg} — only the specific bit pattern changes, never the length distribution. That's why "a Huffman code" for a given probability set is not unique, but its L_{avg} always is.

FULL WORKED EXAMPLE 2 — SIX SYMBOLS, DIFFERENT PATTERN (PRACTICE THIS STYLE TOO)

Symbols: A(0.30), B(0.25), C(0.20), D(0.10), E(0.10), F(0.05). Construct the Huffman code and find L_{avg} and efficiency.

Step-by-step combination:

Step 1: E(0.10)+F(0.05) → EF(0.15)

Step 2: List: A(0.30),B(0.25),C(0.20),D(0.10),EF(0.15) → two lowest: D(0.10)+EF(0.15) → DEF(0.25)

Step 3: List: A(0.30),B(0.25),C(0.20),DEF(0.25) → two lowest: C(0.20)+DEF(0.25)? Careful — sort first: A(0.30),B(0.25),DEF(0.25),C(0.20) → two lowest: C(0.20)+DEF(0.25) → CDEF(0.45)

Step 4: List: A(0.30), CDEF(0.45), B(0.25) → sort: CDEF(0.45), A(0.30), B(0.25) → two lowest: A(0.30)+B(0.25) → AB(0.55)

Step 5: List: AB(0.55), CDEF(0.45) → combine → root(1.0)

SYMBOL	PROBABILITY	CODEWORD	LENGTH
A	0.30	00	2
B	0.25	01	2
C	0.20	11	2
D	0.10	101	3
E	0.10	1000	4
F	0.05	1001	4

L_{avg} = 0.30(2)+0.25(2)+0.20(2)+0.10(3)+0.10(4)+0.05(4) = 0.6+0.5+0.4+0.3+0.4+0.2 = **2.4 bits/symbol**

Entropy $H(X) = -\Sigma P_i \log_2 P_i \approx$ 0.30(1.737)+0.25(2)+0.20(2.322)+0.10(3.322)+0.10(3.322)+0.05(4.322) ≈ 0.521+0.5+0.464+0.332+0.332+0.216 ≈ **2.366 bits/symbol**

Efficiency η = 2.366/2.4 ≈ **98.6%**, redundancy ≈ 1.4% — notice this set is "closer to a power-of-2 distribution" than Example 1, so efficiency is higher.

⚠️ **Common exam mistake:** after every merge, students forget to **re-sort** the full list (not just insert the new node at the end). Always re-arrange in descending order before picking the next two lowest — this is where most numerical errors happen.

4. Shannon-Fano Coding

High ★★★★

ENGLISH

Shannon-Fano coding is a top-down variable-length prefix coding technique that recursively splits a probability-sorted symbol list into two parts with (as nearly as possible) equal total probabilities, assigning '0' to one part and '1' to the other, and repeating on each sub-part until every symbol is isolated.

HINGLISH

Huffman "bottom-up" (chhote nodes jodke upar jaate hain) hai, Shannon-Fano "top-down" hai — pehle sabko probability ke order mein rakho, fir list ko 2 roughly-equal-probability groups mein divide karo (top group=0, bottom group=1), aur har group ko fir se divide karte raho jab tak ek-ek symbol na bach jaaye.

Shannon-Fano Algorithm

Medium

1. Arrange symbols in **descending order** of probability.
2. Divide the list into two parts such that the sum of probabilities in each part is as close to equal as possible.
3. Assign '0' to the upper part and '1' to the lower part.
4. Recursively repeat the division process on each part until each part contains only a single symbol.

FULL WORKED EXAMPLE 1 (SAME SYMBOL SET, FOR DIRECT COMPARISON WITH HUFFMAN)

Symbols: A(0.4), B(0.2), C(0.2), D(0.1), E(0.1)
Split 1: {A(0.4)} | {B,C,D,E(0.6)} → A=0, rest start with 1
Split 2 (within 0.6 group): {B(0.2)} | {C,D,E(0.4)} → B=10, rest start with 11
Split 3 (within 0.4 group): {C(0.2)} | {D,E(0.2)} → C=110, rest start with 111
Split 4: {D(0.1)} | {E(0.1)} → D=1110, E=1111

SYMBOL	PROBABILITY	CODEWORD	LENGTH
A	0.4	0	1
B	0.2	10	2
C	0.2	110	3
D	0.1	1110	4
E	0.1	1111	4

L_{avg} = **2.2 bits/symbol** (same as Huffman in this particular example — but Huffman is guaranteed optimal in general, Shannon-Fano is not always).

FULL WORKED EXAMPLE 2 — A CASE WHERE SHANNON-FANO IS SUB-OPTIMAL VS HUFFMAN

Symbols: A(0.35), B(0.30), C(0.20), D(0.15). Compare Shannon-Fano and Huffman.

Shannon-Fano splits:

Split 1: {A(0.35),B(0.30)}=0.65 | {C(0.20),D(0.15)}=0.35 → upper=0, lower=1 (closest split — 0.65/0.35 is the best available two-way split here)

Split 2 (within upper): {A(0.35)}|{B(0.30)} → A=00, B=01

Split 3 (within lower): {C(0.20)}|{D(0.15)} → C=10, D=11

Shannon-Fano lengths: A=2,B=2,C=2,D=2 → L_{avg}(SF) = 0.35(2)+0.30(2)+0.20(2)+0.15(2) = **2.0 bits/symbol**

Huffman on the same set: combine C(0.20)+D(0.15)→CD(0.35); now A(0.35),B(0.30),CD(0.35) → combine B(0.30)+CD(0.35)→BCD(0.65); combine A(0.35)+BCD(0.65)→root. Codes: A=0 (len1), B=10 (len2), C=110 (len3), D=111 (len3).

Huffman L_{avg} = 0.35(1)+0.30(2)+0.20(3)+0.15(3) = 0.35+0.60+0.60+0.45 = **2.00 bits/symbol** — here they tie again, but this pair of examples is the standard way examiners show that Huffman's *guarantee* of optimality is stronger even when the numeric answer for a specific case matches. In distributions that are less "balanced" (try A=0.5,B=0.25,C=0.125,D=0.125), Shannon-Fano's greedy top split can lock in a worse tree than Huffman's bottom-up merge — always mention this guarantee vs coincidence distinction in theory answers.

Huffman vs Shannon-Fano

High ⭐ ⭐ ⭐ ⭐ — very frequently asked difference question

ASPECT	HUFFMAN CODING	SHANNON-FANO CODING
Approach	Bottom-up (combine smallest first)	Top-down (recursive splitting)
Optimality	Always optimal (minimum L_{avg} among prefix codes)	Not always optimal — can be sub-optimal
Tree construction	Builds tree from leaves to root	Builds tree from root to leaves
Efficiency	Generally ≥ Shannon-Fano efficiency	Usually slightly less efficient than Huffman
Used in practice	JPEG, MP3, ZIP (DEFLATE), fax machines	Mostly historical / educational

5. Lempel-Ziv (LZ) Coding

High ⭐⭐⭐

ENGLISH

Lempel-Ziv coding is a **dictionary-based / adaptive** universal lossless compression technique that does not require prior knowledge of source symbol probabilities. It works by building a dictionary of previously-seen substrings/patterns and replacing repeated occurrences with shorter references (pointers) to dictionary entries.

HINGLISH

Huffman/Shannon-Fano ke liye tumhe symbol probabilities pehle se pata honi chahiye. LZ coding mein aisi koi zarurat nahi — ye **"on-the-fly"** data ke patterns ko dekh ke khud dictionary banata hai, aur jab wahi pattern repeat ho, to use ek short "pointer/code" se replace kar deta hai. Isi liye LZ ko **"universal" / "adaptive"** coding kehte hain — ye file ke type pe depend nahi karta (text, image, sab pe chal jaata hai). ZIP, GIF, PNG sab LZ-family algorithms use karte hain!

LZ77 — Sliding Window Approach

Medium ⭐⭐

LZ77 uses a **sliding window** over the data, split into a "search buffer" (already encoded text) and "look-ahead buffer" (upcoming text). For each position, it searches the search buffer for the longest match to the upcoming data, and outputs a triple:

```
(offset, length, next_character)
```

where **offset** = how far back the match starts, **length** = how many characters matched, and **next_character** = the character following the match (that breaks the match).

SIMPLIFIED EXAMPLE

Input string: `ABABABA`
Encoding: A(0,0,A) → B(0,0,B) → AB seen before at offset 2, length 2 → match → (2,2,A) → remaining 'BA' matches at offset 2 → (2,2, end)
Output (conceptually): `(0,0,A)(0,0,B)(2,2,A)(2,2,-)` — repeated "AB" pattern is replaced by short back-references instead of repeating raw characters, hence compression.
Decoding walkthrough (how the receiver reverses this): the decoder maintains the same growing buffer. (0,0,A) → output "A", buffer="A". (0,0,B) → output "B", buffer="AB". (2,2,A) → go back 2 positions in the buffer, copy 2 characters ("AB"), then append the literal 'A' → output "ABA", buffer="ABABA". (2,2,-) → go back 2, copy 2 ("BA") → output "BA", buffer="ABABABA". Final reconstructed string = **ABABABA**, exactly matching the original — this is why LZ77 is lossless.

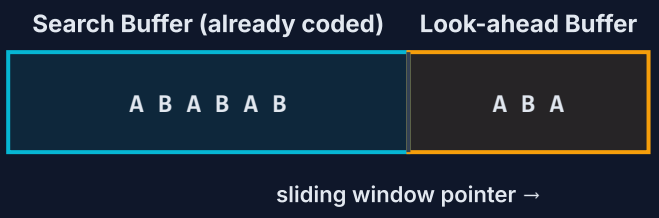


Fig 3.2 — LZ77 sliding window: matches found in search buffer are replaced with (offset,length) back-references

LZ78 — Explicit Dictionary Approach

Medium ⭐ ⭐ — brief but sometimes asked

ENGLISH

Unlike LZ77's sliding window, **LZ78** builds an explicit, ever-growing dictionary of phrases (indexed 1,2,3,...) and encodes each new phrase as a pair **(dictionary index of longest existing match, next new character)**, then adds that new phrase to the dictionary.

SIMPLIFIED EXAMPLE

Input: `ABABABA`, dictionary starts empty (index 0 = "").
Parse: "A" not in dict → output (0,A), add dict[1]="A". "B" not in dict → output (0,B), add dict[2]="B". "AB" is in dict (dict[1]="A", then next char "B"... longest existing match "A"=dict[1], next char "B") → output (1,B), add dict[3]="AB". "AB" again but now check longest match: "AB"=dict[3]? "ABA" → longest match "AB"(dict 3), next char "A" → output (3,A), add dict[4]="ABA". Remaining none.
Output: `(0,A)(0,B)(1,B)(3,A)` — notice the dictionary grows with every new phrase, and later matches can reference multi-character earlier phrases, giving LZ78 stronger compression on longer, more repetitive inputs (this is the basis of the GIF/LZW algorithm).

HINGLISH

LZ77 "sliding window" ke andar hi match dhoondta hai (limited memory), jabki LZ78 ek **permanent, badhta hua dictionary** banata hai jisme har naya phrase add hota rehta hai — isiliye lambi files pe LZ78/LZW (GIF format mein use hota hai) zyada effective ho sakta hai.

Comparison: Huffman/Shannon-Fano vs Lempel-Ziv

Medium ★★★

ASPECT	HUFFMAN / SHANNON-FANO	LEMPEL-ZIV
Type	Statistical (entropy) coding	Dictionary-based / pattern matching
Requires probability table?	Yes — must know symbol probabilities beforehand	No — adaptive, learns patterns from data itself
What is compressed?	Individual symbols (based on frequency)	Repeated sequences/substrings
Best suited for	Sources with skewed (non-uniform) symbol probabilities	Data with repeated patterns (text, source code)
Real-world use	JPEG, MP3 (often combined with other techniques)	ZIP, GIF, PNG, GZIP

6. Channel Capacity (Continuous Channel)

High ⭐⭐⭐⭐⭐ — Shannon-Hartley theorem

ENGLISH

For a continuous, band-limited channel corrupted by additive white Gaussian noise (AWGN), the **Shannon-Hartley Theorem** gives the channel capacity — the maximum rate at which information can be transmitted with arbitrarily low error probability:

$$C = B \cdot \log_2(1 + S/N) \quad [\text{bits/second}]$$

where **B** = channel bandwidth (Hz), **S** = average received signal power, **N** = average noise power, S/N = signal-to-noise ratio (SNR).

HINGLISH

Ye formula batata hai ki ek channel ki **maximum reliable speed** uske **bandwidth** aur **signal-to-noise ratio** pe depend karti hai. Zyada bandwidth ya zyada SNR (kam noise) → zyada capacity. Yahi formula bataata hai ki tumhare WiFi/4G ki max possible speed kitni ho sakti hai given uska bandwidth aur signal quality.

NUMERICAL 1 — BASIC TELEPHONE-LINE STYLE CHANNEL

A channel has bandwidth B=3000 Hz and SNR=1000 (≈30dB, typical telephone line). Find channel capacity.

Solution:

C = 3000 × $\log_2(1+1000)$ = 3000 × $\log_2(1001)$ ≈ 3000 × 9.97 ≈ **29,910 bits/sec ≈ 29.9 kbps**

NUMERICAL 2 — SNR GIVEN IN DB (VERY COMMON VARIANT)

A channel has bandwidth B = 4 kHz and SNR = 20 dB. Find the channel capacity.

Solution:

Step 1 — convert dB to linear ratio: SNR(dB) = 10 $\log_{10}$(S/N) → 20 = 10 $\log_{10}$(S/N) → S/N = $10^{20/10} = 10^2$ = **100**

Step 2 — apply Shannon-Hartley: C = B $\log_2$(1+S/N) = 4000 × $\log_2$(101) = 4000 × 6.658 ≈ **26,633 bits/sec ≈ 26.6 kbps**

Exam tip: whenever SNR is given in dB, always convert with S/N = $10^{(dB/10)}$ *before* plugging into the $\log_2$ formula — a very common mark-losing mistake is plugging the dB value directly into $\log_2$().

NUMERICAL 3 — BANDWIDTH VS SNR TRADE-OFF

Compare two channel design options that must both deliver roughly the same capacity: **Option 1**: B=1 kHz with a high SNR of 1023 (≈30dB). **Option 2**: B=2 kHz with a lower SNR of 31 (≈15dB). Which gives higher capacity?

Option 1: $C_1 = 1000 \times \log_2(1024) = 1000 \times 10 =$ **10,000 bps = 10 kbps**

Option 2: $C_2 = 2000 \times \log_2(32) = 2000 \times 5 =$ **10,000 bps = 10 kbps**

Conclusion: both options give identical capacity, demonstrating the fundamental **bandwidth–SNR trade-off**: you can achieve a target capacity either with high bandwidth + modest SNR, or with narrow bandwidth + very high SNR — capacity depends on the combination $B \cdot \log_2(1+S/N)$, not on B or SNR alone.

Properties of Channel Capacity

Medium

1. Capacity increases with bandwidth B (linearly) and with log of SNR (diminishing returns at high SNR).
2. Even with infinite bandwidth, capacity approaches a finite limit: $C_\infty = (S/N_0) \cdot \log_2 e \approx 1.44 \cdot S/N_0$ (where N_0 = noise power spectral density) — this is the absolute limit imposed by noise power alone.
3. Trade-off: the same capacity can be achieved with high bandwidth + low SNR, or low bandwidth + high SNR (see Numerical 3 above).

7. Channel Coding Theorem (Shannon's Second Theorem)

High ⭐ ⭐ ⭐ ⭐ ⭐ — extremely important theory

ENGLISH

Shannon's **Channel Coding Theorem** states: For a discrete memoryless channel with capacity C, if the information transmission rate R is **less than or equal to C** ($R \leq C$), then there exists a coding scheme such that the probability of error can be made **arbitrarily small** (approaching zero) by increasing the code block length, while keeping R fixed. Conversely, if $R > C$, reliable (error-free) transmission is **not possible**, no matter how clever the code.

HINGLISH

Yeh Information Theory ka sabse important result hai! Iska matlab — agar tum data ko channel capacity C se **kam ya equal rate** pe bhejo, to (achha encoding/decoding scheme use karke) error-probability ko **almost zero** tak le ja sakte ho — bina speed kam kiye! Lekin agar capacity C se **zyada rate** pe bhejne ki koshish karo, to **error-free transmission kabhi possible nahi**, chahe kitna bhi smart code design karo. Ye theorem hi batata hai ki error-correcting codes (jo Unit 4/5 mein padhenge) kyu kaam karte hain.

Significance / Key Takeaways High

- It proves that **reliable communication is possible even over noisy channels** — as long as R ≤ C, which was a revolutionary (and initially counter-intuitive) result in 1948.
- It does NOT tell us *how* to construct such a code — only that one **exists**. The theorem is a pure **existence proof**.
- It establishes capacity C as a **sharp threshold**: below it, arbitrarily reliable communication is achievable; above it, it is fundamentally impossible.
- This motivated decades of research into practical error-correcting codes (Linear Block, Cyclic, Convolutional, Turbo, LDPC) that try to approach the Shannon limit.

APPLIED NUMERICAL — USING THE THEOREM, NOT JUST THE FORMULA

A channel has capacity C = 26.6 kbps (from Numerical 2 above). A designer wants to transmit at R = 20 kbps. Is reliable (arbitrarily low error) transmission possible? What if the designer instead wants R = 30 kbps?

Solution: Since R = 20 kbps < C = 26.6 kbps → **R ≤ C**, so by the Channel Coding Theorem a coding scheme *exists* that can make error probability as small as desired (though the theorem doesn't tell us what that code looks like — only that increasing block length with a good code drives $P_e \to 0$). For R = 30 kbps > C, **reliable transmission is impossible** — no matter how sophisticated the encoder/decoder, errors cannot be driven to zero at that rate on this channel.

8. Shannon Limit

High ⭐⭐⭐⭐

ENGLISH

The **Shannon Limit** is the theoretical maximum data rate (= channel capacity C) at which information can be transmitted over a given channel with an arbitrarily low (→0) probability of error. It represents the ultimate boundary on communication efficiency dictated by the channel's bandwidth and noise — no real coding scheme can exceed it.

HINGLISH

Shannon Limit = wo **"ultimate speed limit"** jo ek channel apni bandwidth aur noise ki wajah se allow karta hai. Iske aage koi bhi coding scheme — chahe kitna bhi advanced ho — error-free data nahi bhej sakta. Modern codes (Turbo codes, LDPC codes — jo 4G/5G/WiFi mein use hote hain) is limit ke **bahut close** (0.5dB ke andar!) pohoch chuke hain — yehi unki achievement hai.

NUMERICAL — SHANNON LIMIT ON E_B/N_0 (MINIMUM SNR PER BIT FOR RELIABLE COMMUNICATION)

Find the minimum theoretical E_b/N_0 (in dB) required for reliable communication as the bandwidth becomes very large (the classical "Shannon bound" / "−1.6 dB limit").
Solution (derivation sketch): Starting from $C = B \log_2(1+S/N)$, and writing $S = E_b \cdot C$ (energy per bit times bit rate) with $N = N_0 B$, as $B \to \infty$ and $C/B \to 0$, the limiting condition works out to:

$$E_b/N_0 \geq \ln 2 \approx 0.693 \quad \Rightarrow \quad (E_b/N_0)_{dB} = 10 \log_{10}(0.693) \approx -1.6 \text{ dB}$$

Interpretation: this famous **−1.6 dB Shannon limit** is the absolute minimum energy-per-bit-to-noise ratio below which reliable communication is impossible, no matter how much bandwidth is used or how clever the code — it's the "floor" that practical codes like Turbo and LDPC try to approach (modern LDPC codes now operate within roughly 0.5–1 dB of this floor).

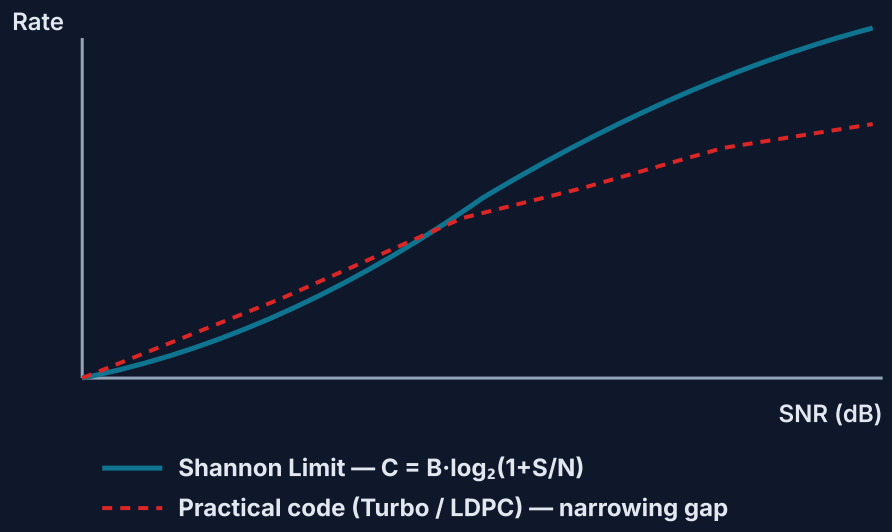


Fig 3.3 — Shannon Limit curve vs achievable rate of real practical codes; the gap has been narrowed to fractions of a dB by modern codes

9. Formula Sheet — Quick Revision

Revision

SOURCE CODING

Kraft: $\Sigma 2^{-l_i} \leq 1$

$L_{avg} = \Sigma P_i l_i$

$H(X) = -\Sigma P_i \log_2 P_i$

$\eta = H(X)/L_{avg}$; Redundancy = $1-\eta$

CHANNEL CODING

$C = B \log_2(1+S/N)$ bps

$C_\infty \approx 1.44\ S/N_0$

Reliable comm. possible $\Leftrightarrow R \leq C$

Shannon bound: $E_b/N_0 \geq -1.6$ dB

🧪 Interactive Simulations

Try these — great for viva & concept building

These run entirely in your browser using the algorithms explained above. Change the inputs and watch the tree, codewords, and efficiency update live — this is the fastest way to build intuition before an exam or viva.

Simulation 1 — Huffman Code Builder

Enter symbols as `name:probability` pairs, comma-separated (probabilities need not be pre-sorted; they don't have to sum exactly to 1, but should be close for the entropy figure to make sense).

```
A:0.4,B:0.2,C:0.2,D:0.1,E:0.1
```

Simulation 2 — Channel Capacity Calculator (Shannon-Hartley)

Drag the sliders to see how bandwidth and SNR jointly determine channel capacity $C = B \cdot \log_2(1+S/N)$.

Bandwidth B: 3000 Hz **SNR: 30 dB**

📡 Move a slider to compute capacity...

HINGLISH TRICK

Source Coders: Huffman = Bottom-Up (Neeche se do sabse chhote jodte jao — Always 100% Optimal!). **Shannon-Fano** = Top-Down (Upar se barabar todte jao — Kabhi kabhi sub-optimal). **Lempel-Ziv: LZ77** = Sliding Window `(offset, len, char)`. **LZ78** = Growing Dictionary Index `(index, char)`.
Shannon Capacity: $C = B \log_2(1 + \text{SNR})$ | Shannon Ultimate Limit: -1.59 dB!

🗺️ Unit 3 Overview Map

Quick Revise

UNIT 3: Source Coding, Data Compaction & Channel Capacity

1. Statistical Source Coding

- **Prefix Code:** Instantaneous decoding, Kraft's $\sum 2^{-l_i} \le 1$
- **Huffman Coding:** Bottom-up merging, provably optimal ($L_{avg} \to H$)
- **Shannon-Fano:** Top-down cumulative probability partitioning
- **Efficiency:** $\eta = H(X)/L_{avg} \times 100\%$; Redundancy $\gamma = 1-\eta$

2. Dictionary Compression

- **Adaptive:** No prior probability distribution required
- **LZ77:** Sliding search window + Lookahead (Offset, Length, Char)
- **LZ78 / LZW:** Explicit dynamic dictionary tree (Index, Char)

3. Capacity & Shannon Limit

- **Shannon-Hartley:** $C = B \log_2(1 + \text{SNR})$ bits/sec
- **Channel Theorem:** If $R \le C$, error probability $P_e \to 0$
- **Shannon Limit:** $(E_b/N_0)_{min} = \ln 2 = -1.59\text{ dB}$ ($B \to \infty$)

❓ Unit 3 — Q&A (RTU Exam Pattern)

16 questions covering theory, derivations, and numericals — answers are written in full exam-length format with proper spacing, so you can use them directly as revision notes.

Q1 · THEORY

Define prefix code. Why is it also called an instantaneous code? Give one valid and one invalid example.

★★★ 5 Marks

A **prefix code** (also called an instantaneous code) is a variable-length code in which no codeword is a prefix of any other codeword in the code set.

It is called "instantaneous" because the decoder can identify each symbol the moment its complete codeword arrives on the bitstream — it never needs to peek ahead at future bits to resolve ambiguity, so decoding happens symbol-by-symbol in real time as bits stream in.

Valid example: A=0, B=10, C=110, D=111 — no codeword is a prefix of another, so this decodes instantly and unambiguously.

Invalid example: A=0, B=01, C=011 — here A's code "0" is a prefix of B's code "01", so on seeing a "0" the decoder cannot know yet whether it's the complete symbol A or just the start of B; it must wait for more bits, which breaks the instantaneous property.

Q2 · NUMERICAL

Given codeword lengths {1, 2, 3, 3}, verify using Kraft's Inequality whether a valid binary prefix code exists. Repeat for lengths {1, 1, 2}.

★★ 5 Marks

Kraft's Inequality states a binary prefix code exists for a set of lengths $l_1,\ldots,l_n$ if and only if $\Sigma\ 2^{-l_i} \leq 1$.

Case 1 — {1,2,3,3}: $\Sigma = 2^{-1}+2^{-2}+2^{-3}+2^{-3} = 0.5+0.25+0.125+0.125 = 1.0$, which is ≤ 1, so a valid prefix code exists (and in fact uses the tree completely, with no wasted leaves).

Case 2 — {1,1,2}: $\Sigma = 0.5+0.5+0.25 = 1.25$, which is > 1, so no valid binary prefix code can be built with these lengths — two codewords of length 1 already use up both branches at the root, leaving nowhere for the third symbol to attach without violating the prefix property.

Q3 · THEORY + NUMERICAL

Explain the Huffman coding algorithm. Construct a Huffman code for symbols A(0.4), B(0.2), C(0.2), D(0.1), E(0.1) and find the average code length and coding efficiency.

★★★★★ 10 Marks — guaranteed numerical every year

Algorithm: (1) list all symbols with probabilities in descending order; (2) repeatedly take the two lowest-probability nodes and merge them into a new node whose probability is their sum, assigning bit '0' to one branch and '1' to the other; (3) re-sort the updated list after every merge; (4) repeat until a single root node (probability 1) remains; (5) read each symbol's codeword as the sequence of 0s/1s from root to that leaf.

Construction: D(0.1)+E(0.1)→DE(0.2); C(0.2)+DE(0.2)→CDE(0.4); B(0.2)+CDE(0.4)→BCDE(0.6); A(0.4)+BCDE(0.6)→root(1.0).

Codewords: A=0 (len 1), B=10 (len 2), C=110 (len 3), D=1110 (len 4), E=1111 (len 4).

Average length: $L_{avg} = \Sigma P_i l_i = 0.4(1)+0.2(2)+0.2(3)+0.1(4)+0.1(4) = 2.2$ bits/symbol.

Entropy: $H(X) = -\Sigma P_i \log_2 P_i \approx 2.122$ bits/symbol.

Efficiency: $\eta = H(X)/L_{avg} = 2.122/2.2 \approx 96.5\%$, so redundancy ≈ 3.5%. This is close to optimal, confirming Huffman coding's near-entropy performance on this distribution.

Q4 · NUMERICAL (PRACTICE VARIANT)

Construct the Huffman code for A(0.30), B(0.25), C(0.20), D(0.10), E(0.10), F(0.05) and compute L_{avg}, entropy, and efficiency.

★★★★ 10 Marks

Construction: E(0.10)+F(0.05)→EF(0.15); D(0.10)+EF(0.15)→DEF(0.25); C(0.20)+DEF(0.25)→CDEF(0.45); A(0.30)+B(0.25)→AB(0.55); AB(0.55)+CDEF(0.45)→root(1.0).

Codewords: A=00, B=01, C=11 (all length 2); D=101 (length 3); E=1000, F=1001 (both length 4).

$L_{avg} = 0.30(2)+0.25(2)+0.20(2)+0.10(3)+0.10(4)+0.05(4) = 2.4$ bits/symbol.

Entropy $H(X) \approx 2.366$ bits/symbol.

Efficiency $\eta \approx 2.366/2.4 \approx 98.6\%$, redundancy ≈ 1.4% — higher than the 5-symbol example because this probability set sits closer to a "clean" power-of-two-like distribution.

Q5 · THEORY + NUMERICAL

Explain the Shannon-Fano coding algorithm with a suitable example. Compare its efficiency with Huffman coding on the same data.

★★★★ 10 Marks

Algorithm: (1) arrange symbols in descending order of probability; (2) split the ordered list into two groups whose total probabilities are as close to equal as possible; (3) assign '0' to the upper group and '1' to the lower group; (4) recursively repeat the split within each group until every symbol is isolated with its own codeword.

Worked example on A(0.4),B(0.2),C(0.2),D(0.1),E(0.1): Split1 {A}|{B,C,D,E}→A=0; Split2 {B}|{C,D,E}→B=10; Split3 {C}|{D,E}→C=110; Split4 {D}|{E}→D=1110,E=1111.

Result: L_{avg}(SF) = 2.2 bits/symbol — identical to Huffman on this particular distribution.

Comparison note: Shannon-Fano's greedy top-down split can coincide with Huffman's optimum on "clean" distributions like this one, but Shannon-Fano is not *guaranteed* optimal in general — on distributions that don't split evenly, its greedy choice at the top level can lock in a sub-optimal tree, whereas Huffman's bottom-up merging always finds the minimum L_{avg} among all prefix codes.

Q6 · THEORY

Differentiate between Huffman coding and Shannon-Fano coding.

★★★★ 5 Marks — very frequently asked

Huffman coding builds its tree **bottom-up**, repeatedly combining the two lowest-probability nodes, and is **always optimal** — it provably achieves the minimum possible average codeword length among all prefix codes for a given probability distribution.

Shannon-Fano coding builds its tree **top-down**, recursively splitting the sorted symbol list into two roughly-equal-probability halves, and is **not always optimal** — its greedy splitting choice can occasionally produce a longer average length than Huffman's.

In practice, Huffman coding is what's actually used in real compression standards (JPEG, MP3, ZIP/DEFLATE), while Shannon-Fano is mostly of historical and educational interest, having been superseded by Huffman's provably-optimal algorithm shortly after its introduction.

Q7 · THEORY

Explain the Lempel-Ziv coding technique. How is it fundamentally different from Huffman/Shannon-Fano coding?

★★★★ 7 Marks

Lempel-Ziv (LZ) coding is a **dictionary-based, adaptive** lossless compression technique. Unlike entropy coders, it requires **no prior knowledge of symbol probabilities** — instead, it scans the data itself, builds up a dictionary of previously-seen substrings/patterns as it goes, and replaces repeated occurrences of those patterns with short back-references (pointers) into the dictionary rather than repeating the raw characters.

This makes LZ coding **universal**: the same algorithm works well on text, source code, images, or any data with repeated structure, without needing to be tuned to a known probability model in advance.

Key difference from Huffman/Shannon-Fano: those two are *statistical* coders that compress individual symbols based on their known frequency, and require the probability table up front. LZ coding is a *pattern-matching* coder that compresses repeated *sequences* of symbols, adapting to the data on the fly — this is why LZ-family algorithms (ZIP, GIF, PNG, GZIP) dominate general-purpose file compression, while Huffman is often used as a secondary/final compression stage layered on top of another technique (e.g., inside JPEG or DEFLATE).

Q8 · NUMERICAL

Explain LZ77 sliding-window compression with a worked example, including both encoding and decoding.

★★★ 7 Marks

LZ77 maintains a **sliding window** over the data stream, divided into a **search buffer** (already-processed text, acting as the "dictionary") and a **look-ahead buffer** (upcoming text to be encoded). At each step, it finds the longest match between the look-ahead buffer and any substring in the search buffer, and outputs the triple **(offset, length, next_character)**.

Encoding example on `ABABABA`: A→(0,0,A); B→(0,0,B); the next two characters "AB" match text 2 positions back with length 2→(2,2,A); the final "BA" matches similarly→(2,2,–). Output stream: (0,0,A)(0,0,B)(2,2,A)(2,2,–).

Decoding: the receiver rebuilds the same growing buffer using each triple: (0,0,A)→append "A"; (0,0,B)→append "B"; (2,2,A)→go back 2 characters in the buffer, copy 2 characters ("AB"), then append the literal 'A'; (2,2,–)→go back 2, copy 2 more characters ("BA"). The reconstructed string is exactly `ABABABA`, confirming the scheme is fully lossless.

Q9 · THEORY

How does LZ78 differ from LZ77? Explain briefly with an example.

★★★ 5 Marks

LZ77 searches for matches only within a **fixed-size sliding window** of recently-seen text, which limits how far back it can reference. LZ78 instead builds an **explicit, unbounded dictionary** of phrases as it parses the input, indexing each new phrase (1, 2, 3, ...) and encoding each step as a pair **(index of longest existing dictionary match, next new character)**.

Example on `ABABABA`: parsing gives phrases "A"→dict[1], "B"→dict[2], "AB"→dict[3] (built from dict[1]+"B"), "ABA"→dict[4] (built from dict[3]+"A"), yielding output (0,A)(0,B)(1,B)(3,A).

Because the dictionary keeps growing and phrases can reference other multi-character phrases, LZ78 (and its popular variant LZW, used in GIF images) tends to compress longer, more repetitive files more effectively than LZ77's bounded window.

Q10 · THEORY + DERIVATION

State and explain the Shannon-Hartley theorem for the channel capacity of a continuous channel. Derive/explain each term.

★★★★★ 7-10 Marks

The **Shannon-Hartley theorem** gives the channel capacity of a continuous, band-limited channel corrupted by additive white Gaussian noise (AWGN) as:

$$C = B \cdot \log_2(1 + S/N) \quad \text{bits/second}$$

Here **B** is the channel bandwidth in Hz, **S** is the average received signal power, **N** is the average noise power, and S/N is the signal-to-noise ratio. The theorem states that C is the theoretical maximum rate at which information can be transmitted through the channel with arbitrarily low probability of error.

The formula shows that capacity grows **linearly** with bandwidth (double B, roughly double C) but only **logarithmically** with SNR (each doubling of SNR adds a fixed number of bits/sec, showing diminishing returns) — this is why engineers generally prefer adding bandwidth over pushing signal power when trying to raise data rates cheaply.

Q11 · NUMERICAL

A channel has bandwidth B = 3 kHz and SNR = 1000. Find its channel capacity.

★★★★ 5 Marks numerical

Using Shannon-Hartley: $C = B \log_2(1+S/N) = 3000 \times \log_2(1+1000) = 3000 \times \log_2(1001)$.

$\log_2(1001) \approx 9.97$, so $C \approx 3000 \times 9.97 \approx$ **29,910 bits/sec ≈ 29.9 kbps**.

Q12 · NUMERICAL (DB VARIANT)

A channel has bandwidth B = 4 kHz and SNR = 20 dB. Find the channel capacity.

★★★★ 5 Marks numerical

Step 1 — convert SNR from dB to linear: $SNR(dB) = 10 \log_{10}(S/N) \rightarrow 20 = 10 \log_{10}(S/N) \rightarrow S/N = 10^2 = 100$.

Step 2 — apply Shannon-Hartley: $C = 4000 \times \log_2(1+100) = 4000 \times \log_2(101) \approx 4000 \times 6.658 \approx$ **26,633 bits/sec ≈ 26.6 kbps**.

Always convert dB to a linear ratio first — plugging the dB number directly into $\log_2()$ is a common and costly exam error.

Q13 · THEORY + DERIVATION

State and explain Shannon's Channel Coding Theorem (Shannon's Second Theorem). What does it prove and what does it NOT tell us?

★★★★★ 10 Marks — most important theory question in whole unit

Shannon's Channel Coding Theorem states that for a discrete memoryless channel with capacity C: if the transmission rate R ≤ C, there exists a coding scheme for which the probability of decoding error can be made **arbitrarily close to zero** by increasing the code's block length, while keeping R fixed. Conversely, if R > C, reliable transmission is **fundamentally impossible**, regardless of how sophisticated the encoder/decoder design is.

What it proves: reliable communication over a noisy channel is achievable — a revolutionary and counter-intuitive result when first published in 1948, since intuition suggests any real noisy channel must always introduce some irreducible error rate.

What it does NOT tell us: the theorem is a pure **existence proof** — it guarantees a good code exists for any R ≤ C, but gives no construction method. This gap motivated decades of coding-theory research (Hamming, cyclic, convolutional, Turbo, LDPC codes) aimed at finding practical, implementable codes that approach this theoretical promise.

The theorem also establishes C as a **sharp threshold**: performance doesn't degrade gradually as R crosses C — below C, error-free-in-the-limit transmission is achievable, and above C it is categorically impossible.

Q14 · APPLIED NUMERICAL

A channel's capacity is 26.6 kbps. Using the Channel Coding Theorem, comment on whether reliable transmission is possible at R = 20 kbps and at R = 30 kbps.

★★★ 5 Marks

At R = 20 kbps: since R = 20 kbps < C = 26.6 kbps, the condition R ≤ C is satisfied, so by the Channel Coding Theorem a coding scheme exists that can drive the error probability arbitrarily close to zero (with sufficiently long code blocks) while transmitting at this rate.

At R = 30 kbps: since R = 30 kbps > C = 26.6 kbps, reliable transmission is **not possible** — no matter how the code is designed, some irreducible error probability must remain when the rate exceeds capacity.

Q15 · THEORY

What is meant by the "Shannon Limit"? Why can no practical system exceed it, and how close have real codes gotten?

★★★ 5 Marks

The **Shannon Limit** is another name for the channel capacity C itself — the absolute maximum data rate at which information can be transmitted over a channel with arbitrarily low probability of error, given the channel's bandwidth and noise characteristics.

No coding scheme, however sophisticated, can communicate reliably above this rate, because it is a **fundamental information-theoretic bound** derived from the physics of the channel (bandwidth and noise power) — not merely a limitation of current engineering or algorithm design.

In its most extreme form (infinite bandwidth), the limit reduces to the famous **$E_b/N_0 \geq -1.6$ dB** bound — the minimum energy per bit needed for reliable communication under any circumstances. Modern practical codes such as Turbo codes and LDPC codes (used in 4G/5G/WiFi) now operate within roughly 0.5 dB of this theoretical floor, which is considered one of the great engineering achievements of modern communications.

Q16 · CONCEPTUAL LINKING

Explain the relationship between channel capacity, the channel coding theorem, and the Shannon limit.

★★★ 5 Marks — conceptual linking question, frequently combined into one

These three ideas describe the **same underlying boundary** from three different angles, moving from definition → proof → practical naming.

Channel capacity C is the quantitative definition of the limit — the Shannon-Hartley formula $C = B \log_2(1+S/N)$ tells us exactly what that maximum rate is for a given continuous AWGN channel.

The Channel Coding Theorem is the proof that this number C is achievable — it shows that reliable (arbitrarily-low-error) transmission is possible for any rate $R \leq C$, and impossible for $R > C$, establishing C as a hard, sharp threshold rather than a soft guideline.

The Shannon Limit is simply the practical, colloquial name engineers use for this same capacity boundary when discussing how close real-world codes (Turbo, LDPC) can get to the theoretical maximum — so all three terms ultimately point to one single number, C, viewed respectively as a formula, a guarantee, and a design target.

Prepared for RTU 5CS3-01 · Information Theory & Coding — Unit 3. All numericals are worked in full detail; practice with different probability/SNR values using the simulations above before your exam.

UNIT 4 — LINEAR BLOCK CODE

5CS3-01 · Information Theory & Coding · RTU Kota · III Year – V Sem · B.Tech CSE

Credit: 2 | Max Marks: 100 (IA:30, ETE:70) | End Term: 3 Hours

⚠ HIGH PRIORITY UNIT ⭐ 5 & 10 Mark Questions ✦ 6 Hours Weightage

📋 UNIT 4 — CONTENTS

1. Introduction to Error Control Codes HIGH ★★★
2. Types of Errors & Error Detection/Correction Concepts HIGH ★★★
3. Linear Block Codes — Definition & Properties HIGH ★★★
4. Hamming Code — Theory & Construction HIGH ★★★
5. Minimum Distance Consideration HIGH ★★★
6. Encoding of Linear Block Codes (Generator Matrix G) HIGH ★★★
7. Decoding of Linear Block Codes (Parity Check Matrix H) HIGH ★★★
8. Syndrome Decoding & Standard Array HIGH ★★★
9. Conversion: Non-Systematic → Systematic Matrix Form HIGH ★★★
10. RTU Exam Q&A Bank (5-mark & 10-mark) EXAM GOLD

01. Introduction to Error Control Codes HIGH

Why do we need error control? What is channel coding?

DEFINITION

Error Control Coding is the process of adding *redundant bits* to information bits before transmission so that errors introduced by the noisy channel can be **detected** and/or **corrected** at the receiver.

IN HINGLISH EXPLANATION Jab hum data transmit karte hain, toh channel noise ki wajah se kuch bits change ho sakti hain. Isliye transmitter extra bits add karta hai — jaise ek friend apna message dobara bol ke confirm kare. Receiver in extra bits se samajhta hai ki error hui hai ya nahi, aur agar hui hai toh correct bhi kar leta hai. **Yahi hai Error Control Coding ka kaam.**

Why Error Control is Needed?

- Real channels (wireless, satellite, fiber) introduce **noise, interference, fading** → bits flip (0→1 or 1→0)
- Without error control: corrupted data reaches receiver silently
- Solution: Add **redundancy** — extra check bits that make errors detectable/correctable

Two Main Goals

🔍 ERROR DETECTION

Receiver can **detect** that an error occurred, but cannot fix it. It asks for **retransmission (ARQ)**.
Example: Parity Check, CRC

🔧 ERROR CORRECTION

Receiver can **detect AND correct** the error itself. Called **FEC (Forward Error Correction)**.
Example: Hamming Code, Linear Block Code

ARQ vs FEC

FEATURE	ARQ (AUTOMATIC REPEAT REQUEST)	FEC (FORWARD ERROR CORRECTION)
Action on Error	Retransmit	Correct at receiver
Feedback Channel	Required	Not Required
Delay	Higher	Lower
Bandwidth Usage	Variable	Fixed overhead
Used In	TCP/IP, WiFi	Satellite, Deep Space

Types of Codes

TYPE	DESCRIPTION	EXAMPLE
Block Codes	Message divided into fixed blocks; redundancy added per block	Hamming, BCH, Reed-Solomon
Convolutional Codes	Continuous stream; encoder has memory	Rate ½ Conv. Code
Linear Codes	Subclass of block codes; linear combination of codewords is a codeword	All Hamming codes
Cyclic Codes	Subclass of linear codes; cyclic shift of codeword is also a codeword	CRC, BCH

SIMPLE EXAMPLE

Original message: `101` (3 bits) → After encoding (Hamming 7,4): `1011010` (7 bits). The extra 4 bits help detect & correct 1-bit errors. Yeh extra bits hain **redundant bits** ya **parity bits**.

---

02. Types of Errors & Basic Concepts HIGH

Types of Errors

ERROR TYPE	DESCRIPTION	CAUSE
Single-bit Error	Only 1 bit changes (0→1 or 1→0)	White noise
Burst Error	Multiple consecutive bits change	Lightning, fading
Random Error	Scattered bit flips; no pattern	Thermal noise

Fig 1: Single-bit error vs Burst error

Key Terms — Must Know for RTU

CODEWORD

A **codeword** is the output of an encoder — it contains both the message bits and the redundant (parity) bits. Example: In (7,4) Hamming code, every codeword has 7 bits.

CODE RATE

Code Rate **R = k/n** where *k* = message bits, *n* = codeword bits. Higher rate = less redundancy = less protection. Jitna zyada rate, utna kam protection.

```
R = k/n    Example: (7,4) code → R = 4/7 ≈ 0.57
```

REDUNDANCY

Number of check/parity bits added = **r = n − k**. These bits carry no new info but help detect/correct errors.

IN HINGLISH (n,k) code mein: **n** total bits hain codeword mein, **k** original message bits hain, aur **r = n−k** parity bits hain jo error check ke liye add kiye gaye hain. Jaise ek parcel mein actual saman (k) aur packing material (r) — packing se parcel safe rehta hai.

03. Linear Block Codes — Definition & Properties HIGH

DEFINITION — LINEAR BLOCK CODE

A **(n, k) Linear Block Code** is a block code where:

- The encoder takes **k message bits** and produces an **n-bit codeword**
- **Linearity Condition:** The *sum (XOR) of any two codewords is also a valid codeword*
- The code forms a **k-dimensional subspace** of the vector space $GF(2)^n$ (all operations in binary, i.e., modulo-2)

IN HINGLISH Linear Block Code mein ek khaas property hai — agar c_1 aur c_2 dono valid codewords hain, toh $c_1 \oplus c_2$ (XOR) bhi ek valid codeword hoga. Yeh linearity condition hai. Iska fayda: hum sirf **basis codewords** (generator matrix) se saare valid codewords generate kar sakte hain.

Properties of Linear Block Codes

1. **Linearity:** $c_1 \oplus c_2 \in C$ (XOR of two codewords is also a codeword)
2. **All-zero codeword exists:** The all-zeros sequence 000...0 is always a valid codeword
3. **Closure:** The set of all codewords is closed under addition (XOR)
4. **Total codewords:** Exactly 2^k valid codewords in an (n,k) code
5. **Minimum distance = Minimum weight:** d_min = minimum Hamming weight of all non-zero codewords

★ **RTU KEY POINT:** For a linear code, minimum Hamming distance (d_min) = minimum Hamming weight (w_min) of all non-zero codewords. Yeh property bahut important hai for exam.

Hamming Weight vs Hamming Distance

HAMMING WEIGHT — W(C)

The **Hamming weight** of a codeword c is the *number of non-zero (1) bits* in it.

```
w(10110) = 3    (three 1s present)
```

HAMMING DISTANCE — D(C_1, C_2)

The **Hamming distance** between two codewords c_1 and c_2 is the *number of bit positions where they differ*.

```
d(10110, 11010) = ?   XOR → 01100 → weight = 2   ∴ d = 2
```

EXAMPLE — HAMMING DISTANCE CALCULATION

c_1 = `1 0 1 1 0`
c_2 = `1 1 0 1 0`
XOR: `0 1 1 0 0` → Two 1s → **$d(c_1, c_2) = 2$**

> IN Hamming distance = jitni jagah par dono codewords differ karte hain. XOR karo — jahan 1 aata hai woh differ karta hai. Unhe gino — yahi distance hai.

Number of Codewords & Structure

- An (n, k) linear block code has exactly **2^k codewords**
- Total possible n-bit sequences = 2^n
- Ratio = 2^k / 2^n = 2^(k−n) = 2^(−r) — only a small fraction of all sequences are valid codewords

```
(7,4) Hamming Code:
Total 7-bit sequences = 2⁷ = 128
Valid codewords = 2⁴ = 16
```

---

04. Hamming Code — Theory & (7,4) Example HIGH

WHAT IS HAMMING CODE?

Hamming codes are a family of **linear error-correcting codes** invented by Richard Hamming (1950). They can **detect up to 2-bit errors** and **correct 1-bit errors**. They are the simplest and most studied linear block codes.

IN HINGLISH Hamming code ek aisa code hai jo automatically **1 bit ka error correct** kar sakta hai — bina retransmission ke! Iska inventor Richard Hamming tha jo Bell Labs mein kaam karta tha. Usne 1950 mein yeh code banaya. (7,4) matlab: 7 total bits mein 4 message bits + 3 parity bits.

Hamming Code Parameters

GENERAL HAMMING CODE PARAMETERS

```
r = number of parity bits (r ≥ 1)
n = 2ʳ - 1   (total codeword length)
k = n - r = 2ʳ - r - 1   (message bits)
d_min = 3   (minimum Hamming distance, always)
Detects: up to 2 errors | Corrects: 1 error
```

Common Hamming Codes

R (PARITY BITS)	N = $2^r - 1$	K = N−R	CODE	D_MIN
2	3	1	(3,1)	3
3	**7**	**4**	**(7,4) ← Most common in RTU**	**3**
4	15	11	(15,11)	3
5	31	26	(31,26)	3

Construction of (7,4) Hamming Code

In a (7,4) Hamming code:

- Bit positions: **1, 2, 3, 4, 5, 6, 7**
- Parity bit positions (powers of 2): **1, 2, 4**
- Message bit positions: **3, 5, 6, 7**

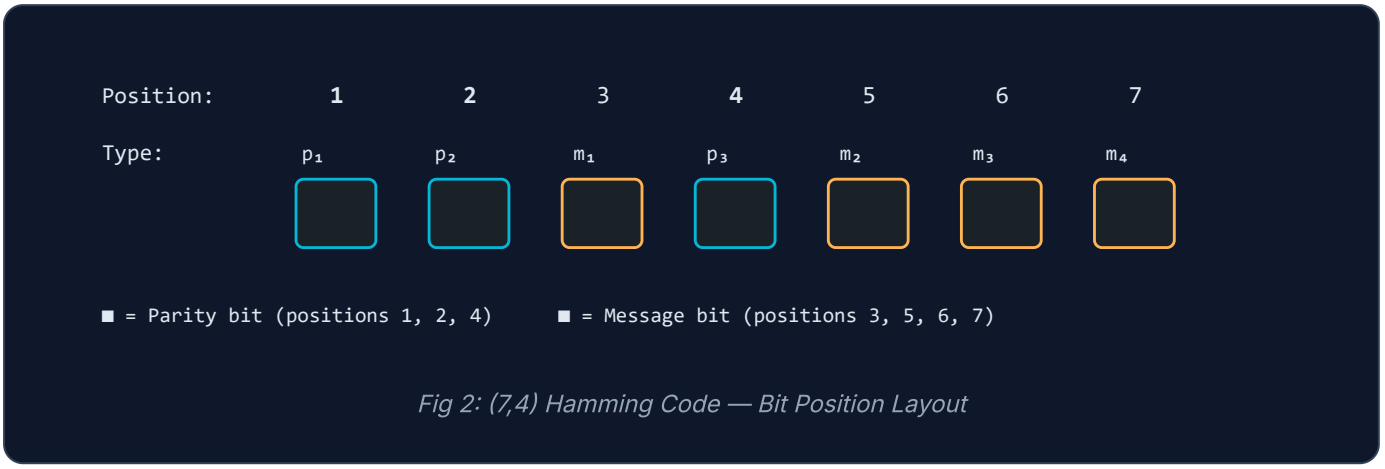


Fig 2: (7,4) Hamming Code — Bit Position Layout

Parity Check Rules in (7,4) Hamming Code

Each parity bit covers specific positions based on its binary representation:

$$p_1 \text{ (pos } 1 = 001_2) \text{ covers: positions } 1,3,5,7 \rightarrow p_1 \oplus m_1 \oplus m_2 \oplus m_4 = 0$$
$$p_2 \text{ (pos } 2 = 010_2) \text{ covers: positions } 2,3,6,7 \rightarrow p_2 \oplus m_1 \oplus m_3 \oplus m_4 = 0$$
$$p_3 \text{ (pos } 4 = 100_2) \text{ covers: positions } 4,5,6,7 \rightarrow p_3 \oplus m_2 \oplus m_3 \oplus m_4 = 0$$

Rule: A position *i* is covered by parity bit at position *j* if bit *j* of the binary rep of *i* is 1.

WORKED EXAMPLE — ENCODE MESSAGE 1011

Message: m_1=1, m_2=0, m_3=1, m_4=1 → placed at positions 3,5,6,7
Layout: _ _ 1 _ 0 1 1 (positions 1,2,3,4,5,6,7)

Calculate p_1 (covers 1,3,5,7): $p_1 \oplus 1 \oplus 0 \oplus 1 = 0 \rightarrow p_1 = 0$
Calculate p_2 (covers 2,3,6,7): $p_2 \oplus 1 \oplus 1 \oplus 1 = 0 \rightarrow p_2 = 1$
Calculate p_3 (covers 4,5,6,7): $p_3 \oplus 0 \oplus 1 \oplus 1 = 0 \rightarrow p_3 = 0$

Final Codeword: `0 1 1 0 0 1 1`

IN Pehle message bits apni positions par rakh do (3,5,6,7). Phir har parity bit ke liye XOR karo covered positions ka. Result = 0 chahiye — agar nahi, toh parity bit = 1. Easy hai!

05. Minimum Distance Consideration HIGH

MINIMUM HAMMING DISTANCE — D_MIN

The **minimum Hamming distance** d_min of a code is the *smallest Hamming distance between any two distinct codewords* in the code. It determines the error-detecting and error-correcting capability.

$$d_{min} = \min\{ d(c_i, c_j) \} \text{ for all } i \neq j$$

For a **linear code**: d_min = minimum weight of any *non-zero* codeword

IN HINGLISH Minimum distance batata hai ki code kitna *spread out* hai. Jitna zyada d_min, utna zyada error correct kar sakte hain. Socho codewords ek circle mein hain — bade circle = zyada gap = zyada errors detect/correct ho sakti hain.

Error Detection & Correction Capability

★ MOST IMPORTANT FORMULAS FOR RTU EXAM ★

Error DETECTION capability: $t_d = d_{min} - 1$ (can detect up to t_d errors)

Error CORRECTION capability: $t_c = \lfloor (d_{min} - 1)/2 \rfloor$ (can correct up to t_c errors)

Combined detect t errors AND correct s errors:
Condition: $d_{min} \geq t + s + 1$ (where $t \geq s$)

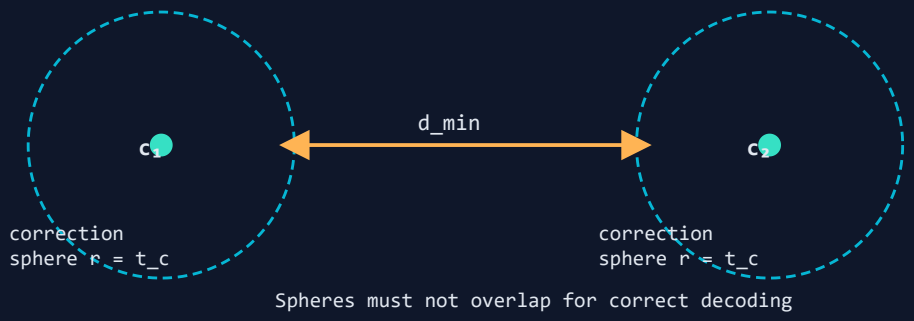


Fig 3: Minimum Distance and Error Correction Spheres

Worked Examples on d_min

EXAMPLE 1 — GIVEN D_MIN = 5

Error detection: $t_d = 5 - 1 =$ **4** (can detect up to 4 errors)

Error correction: $t_c = \lfloor (5-1)/2 \rfloor = \lfloor 2 \rfloor =$ **2** (can correct up to 2 errors)

Combined: can detect 3 errors AND correct 1 error simultaneously (3+1+1=5 ✓)

EXAMPLE 2 — FIND D_MIN FOR A CODE

Codewords: 000000, 110100, 001110, 111010

Find minimum weight among non-zero codewords:

- w(110100) = 3
- w(001110) = 3
- w(111010) = 4

d_min = 3 (minimum weight of non-zero codewords)

→ Can detect 2 errors, correct 1 error

Summary Table — d_min Capabilities

D_MIN	ERRORS DETECTED (T_D)	ERRORS CORRECTED (T_C)
1	0	0
2	1	0
3	**2**	**1 ← Hamming Code**

4	3	1
5	4	2
7	6	3

06. Encoding — Generator Matrix [G] HIGH

GENERATOR MATRIX G

The **Generator Matrix G** is a $k \times n$ matrix whose rows form a basis for the code. Every codeword c is a linear combination of rows of G:

$$c = m \cdot G$$

where m is the 1×k message vector and G is the k×n generator matrix. All arithmetic is modulo-2.

IN HINGLISH Generator matrix G ek blueprint hai — message vector m ko G se multiply karo (mod-2 mein), aur result ek valid codeword c milta hai. G ke rows saare "basic" codewords hote hain, aur baaki sab unka combination hota hai.

Structure of Generator Matrix (Systematic Form)

$$G = [I_k \mid P]$$

where $\mathbf{I_k}$ is the k×k identity matrix and **P** is the k×r parity sub-matrix.

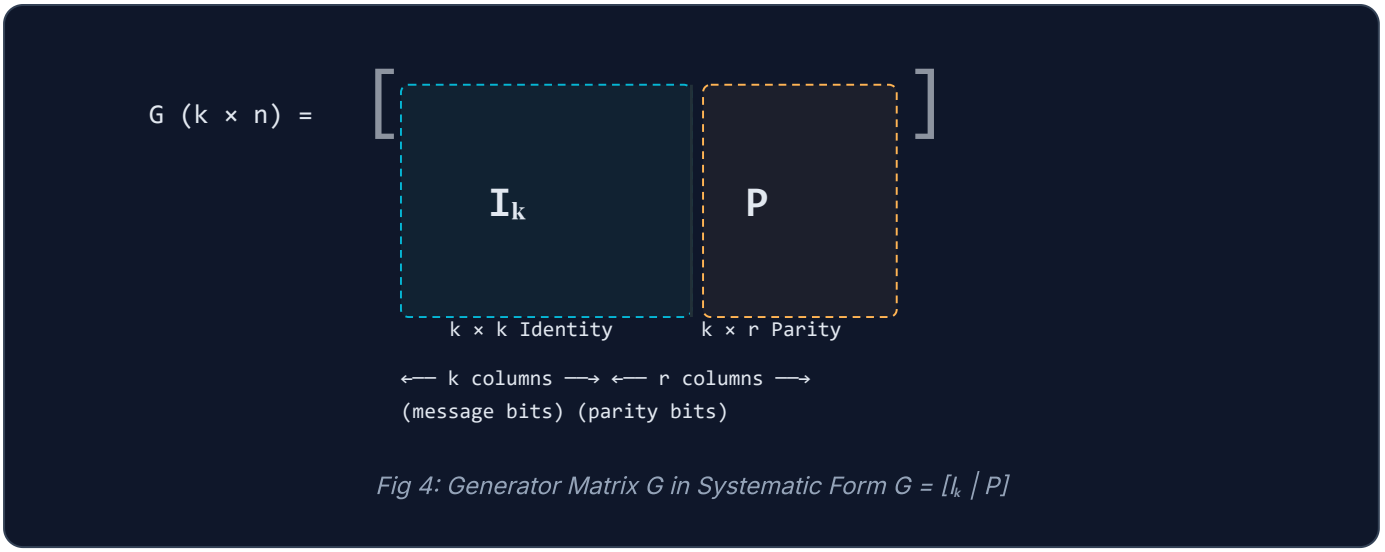


Fig 4: Generator Matrix G in Systematic Form G = [I_k | P]

Encoding Example — (7,4) Code

EXAMPLE — (7,4) HAMMING CODE GENERATOR MATRIX

For the (7,4) Hamming code, k=4, n=7, r=3:

```
G = [ I₄ | P ] =

  ⎡ 1 0 0 0 | 1 1 0 ⎤
  ⎢ 0 1 0 0 | 0 1 1 ⎥
  ⎢ 0 0 1 0 | 1 1 1 ⎥
  ⎣ 0 0 0 1 | 1 0 1 ⎦
```

Encode message m = [1 0 1 1]:

```
c = m · G = [1 0 1 1] ·
  ⎡ 1 0 0 0 1 1 0 ⎤
  ⎢ 0 1 0 0 0 1 1 ⎥
  ⎢ 0 0 1 0 1 1 1 ⎥
  ⎣ 0 0 0 1 1 0 1 ⎦

= 1×(row1) ⊕ 0×(row2) ⊕ 1×(row3) ⊕ 1×(row4)
= [1001110] ⊕ [0000000] ⊕ [0010111] ⊕ [0001101]
= [1 0 1 1 0 1 0]    ← valid codeword!
```

IN m ke jo bits 1 hain (positions 1,3,4) unke corresponding G ki rows ko XOR karo — yahi codeword hai! Message bits pehle 4 positions mein same rahenge (systematic form ki wajah se).

Step-by-Step Encoding Process

1. **Write the message vector m** (1 × k row vector)
2. **Set up Generator Matrix G** (k × n, in systematic form G = [I_k | P])
3. **Multiply:** c = m · G using modulo-2 arithmetic (XOR addition, AND multiplication)

4 **Result c** is the n-bit codeword: first k bits = message, last r bits = parity

📝 RTU Exam Tip
In systematic form, the first k bits of codeword c are always identical to message bits m. So you only need to compute the parity bits (last r bits) — the message part is automatic!

---

07. Decoding — Parity Check Matrix [H] HIGH

PARITY CHECK MATRIX H

The **Parity Check Matrix H** is an $r \times n$ matrix such that for any valid codeword c:

$$H \cdot c^T = 0 \text{ (zero vector)}$$

If a received vector r is a valid codeword, $H \cdot r^T = 0$. If $H \cdot r^T \neq 0$, an error occurred.

IN HINGLISH H matrix ek *error detector* hai. Received vector ko H se multiply karo — agar answer zero aata hai, toh koi error nahi. Agar non-zero aata hai (syndrome), toh error hai aur syndrome batata hai KAHAN error hai!

Relationship Between G and H

$$\text{If } G = [I_k \mid P] \text{ then } H = [P^T \mid I_r]$$

$$\text{Always: } G \cdot H^T = 0 \pmod 2$$

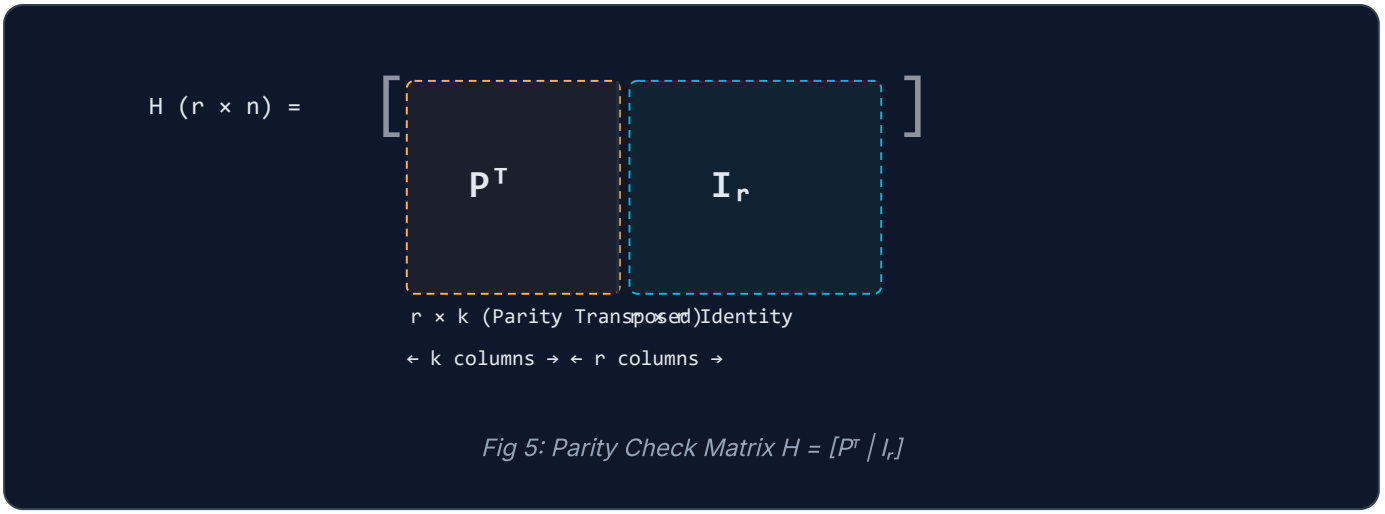


Fig 5: Parity Check Matrix $H = [P^T \mid I_r]$

EXAMPLE — H MATRIX FOR (7,4) CODE

Given P for (7,4) code:

```
P =
 [ 1 1 0 ]
 | 0 1 1 |
 | 1 1 1 |
 [ 1 0 1 ]

Pᵀ (transpose of P) =
 [ 1 0 1 1 ]
 | 1 1 1 0 |
 [ 0 1 1 1 ]

H = [Pᵀ | I₃] =
 [ 1 0 1 1 | 1 0 0 ]
 | 1 1 1 0 | 0 1 0 |
 [ 0 1 1 1 | 0 0 1 ]
```

Verify codeword c = [1 0 1 1 0 1 0]:

```
H · cᵀ = [Hrow1·c, Hrow2·c, Hrow3·c]
Row1: 1·1+0·0+1·1+1·1+1·0+0·1+0·0 = 1+0+1+1+0+0+0 = 3 mod 2 = 1...
```

NOTE

Verification that G·Hᵀ = 0 is the key property — always check this!

08. Syndrome Decoding HIGH

SYNDROME — S

The **syndrome** S is computed from the received vector r (which may contain errors):

$$S = H \cdot r^T \text{ (r × 1 vector)}$$

- If S = 0 → No error detected (or even number of errors — undetectable)
- If S ≠ 0 → Error detected; S identifies the error pattern

IN HINGLISH Syndrome ek "error fingerprint" hai. Received vector r mein agar error hai, toh $H \cdot r^T \neq 0$ hoga. Yeh non-zero vector hi syndrome hai. Is syndrome ko dekhkar hum samajhte hain ki **kon si position mein error hai**. Jaise doctor symptoms dekhkar disease diagnose karta hai!

How Syndrome Identifies Error Location

Let c be the transmitted codeword and e be the error vector (1 at error positions).
Received vector: **r = c ⊕ e**

$$S = H \cdot r^T = H \cdot (c \oplus e)^T = H \cdot c^T \oplus H \cdot e^T = 0 \oplus H \cdot e^T = H \cdot e^T$$

Key insight: Syndrome $S = H \cdot e^T$ depends ONLY on the error pattern e, NOT on the message. So S uniquely identifies the error!

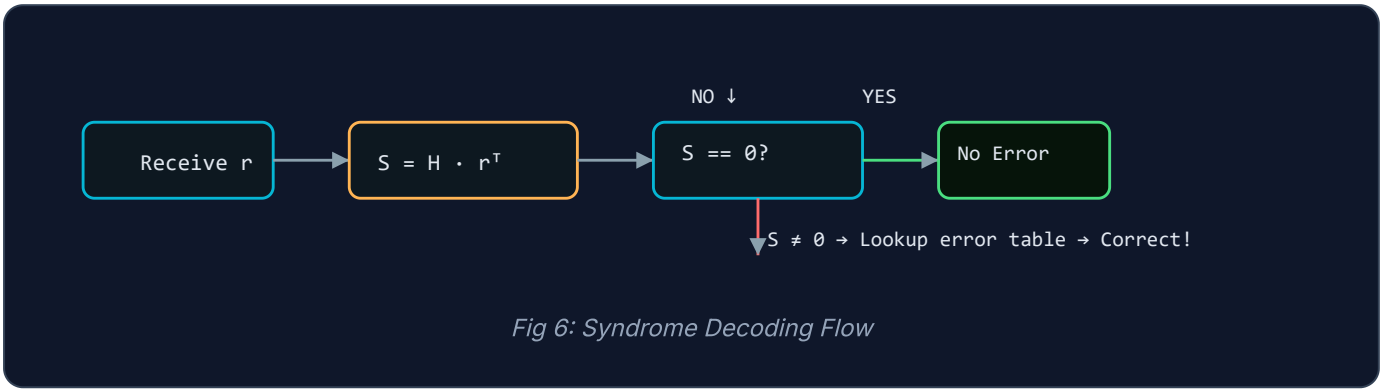


Fig 6: Syndrome Decoding Flow

Syndrome Table (Error Lookup) for (7,4) Hamming Code

SYNDROME S (BINARY)	ERROR POSITION	ERROR VECTOR E

000	No error	0000000
001	**Position 1**	1000000
010	**Position 2**	0100000
011	**Position 3**	0010000
100	**Position 4**	0001000
101	**Position 5**	0000100
110	**Position 6**	0000010
111	**Position 7**	0000001

IN HINGLISH (7,4) Hamming code mein syndrome ek 3-bit number hai. Yeh number directly error ki position batata hai! Syndrome = 101_2 = 5 → position 5 mein error. Received bit ko flip karo — corrected codeword mil gaya!

FULL DECODING EXAMPLE

Transmitted: c = 1011010
Received (with error at position 3): r = 1001010

Step 1: Compute Syndrome

```
S = H · rᵀ
Using H matrix, compute each row × r (mod-2):
S = [0, 1, 1] = 011₂ = 3
```

Step 2: Look up syndrome table → Syndrome 011 → Error at position 3
Step 3: Correct → Flip bit 3 in received: 1001010 → 1011010 ✓

IN Syndrome = 011 = 3 (decimal) → position 3 mein galti hai → received vector ka bit 3 flip karo → original codeword wapas mil gaya!

Standard Array Decoding

STANDARD ARRAY

A **Standard Array** is a systematic table for decoding. It has 2^k columns (one per codeword) and 2^(n−k) rows (one per coset). The top row is all codewords. The leftmost column contains **coset leaders** (error patterns).

Structure of Standard Array:

COSET LEADER (E)	C_1=0000	C_2	C_3	...
e_1=0000	c_1	c_2	c_3	...
e_2	$e_2 \oplus c_1$	$e_2 \oplus c_2$	$e_2 \oplus c_3$	...
e_3	$e_3 \oplus c_1$	$e_3 \oplus c_2$	$e_3 \oplus c_3$	...

Decoding: Find received r in the array → its coset leader = error pattern → c = r ⊕ e

09. Conversion: Non-Systematic → Systematic Form

HIGH

SYSTEMATIC VS NON-SYSTEMATIC CODE

✅ SYSTEMATIC CODE

Message bits appear **unchanged** in the codeword at fixed positions (usually first k positions).

Codeword = $[m_1\ m_2 \dots m_k \mid p_1\ p_2 \dots p_r]$

❌ NON-SYSTEMATIC CODE

Message bits are **mixed** with parity bits — the original message cannot be directly read off.

Codeword bits don't directly map to message bits

IN HINGLISH Systematic form mein message bits pehle k positions mein clearly dikhai dete hain — alag se decode nahi karna padta. Non-systematic mein sab mix ho jaata hai. RTU exam mein conversion ka question zaroor aata hai — **Row operations (Elementary Row Operations)** se G matrix ko systematic form mein laate hain.

Why Convert?

- Systematic form is easier to implement in hardware
- Message bits are directly visible — no extra decoder needed
- Parity check matrix H is easily derived from G
- Standard error syndrome tables are simpler

Method: Using Row Reduction (Gaussian Elimination over GF(2))

Any generator matrix can be converted to systematic form using **Elementary Row Operations** over GF(2):

1. Add any two rows (XOR) → replace one row
2. Swap two rows
3. Multiply a row by a non-zero scalar (in GF(2), only 1 is non-zero → no change)

> ⚠ Important
> Column swaps change the bit ordering — if you swap columns, track which message bit corresponds to which position. Row operations alone (no column swaps) are preferred.

1. **Write the non-systematic G matrix** (k × n)
2. **Target:** Make the first k columns form an identity matrix I_k using row operations
3. **For each column i (1 to k):** Find a row with 1 in column i, swap it to row i position
4. **Eliminate:** For all other rows j ≠ i that have 1 in column i: Row_j = Row_j ⊕ Row_i
5. **Repeat** until columns 1 to k form I_k
6. The remaining k columns (k+1 to n) form the parity matrix P
7. **Final:** G_sys = $[I_k \mid P]$ and H = $[P^T \mid I_r]$

Detailed Worked Example — Non-Systematic to Systematic

EXAMPLE — CONVERT (6,3) CODE GENERATOR MATRIX

Given non-systematic G:

$$G = \begin{bmatrix} 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 & 0 \end{bmatrix} \quad (g_1, g_2, g_3)$$

Step 1: Check column 1 — g_1 has 1 in col 1. Good. ✓
Eliminate col 1 from other rows:
g_3 has 1 in col 1 → $g_3 = g_3 \oplus g_1$

$$g_3 = [1,0,1,1,0,0] \oplus [1,1,0,0,1,0] = [0,1,1,1,1,0]$$

After Step 1:

$$\begin{bmatrix} 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 1 & 0 \end{bmatrix}$$

Step 2: Column 2 — g_2 has 1 in col 2. Good. ✓
Eliminate col 2 from g_1 and g_3:
g_1: has 1 in col 2 → $g_1 = g_1 \oplus g_2$
g_3: has 1 in col 2 → $g_3 = g_3 \oplus g_2$

$$g_1 = [1,1,0,0,1,0] \oplus [0,1,1,0,0,1] = [1,0,1,0,1,1]$$
$$g_3 = [0,1,1,1,1,0] \oplus [0,1,1,0,0,1] = [0,0,0,1,1,1]$$

After Step 2:

$$\begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 \end{bmatrix}$$

Step 3: Column 3 — g_1 and g_2 both have 1 in col 3, but g_3 has 0. Need 1 in col 3 of row 3.

Swap g_3 with a row that has 1 in col 3 — but g_3 has 0 in col 3. Let's swap g_3 and check col 4 instead — column 3 has 1 in rows 1 and 2. Bring g_3 to pivot position by noting col 4 is next candidate for pivot.
Swap columns 3 and 4 (track reordering):

```
g₁ = [ 1 0 | 0 1 | 1 1 ] (swapped cols 3,4)
g₂ = [ 0 1 | 0 1 | 0 1 ]
g₃ = [ 0 0 | 1 0 | 1 1 ]
```

Now col 3: g_3 has 1 → eliminate from g_1 (0 already), g_2 (0 already). Good.
Eliminate col 3 from g_1: g_1 has 0 in col 3 — no action. g_2 has 0 — no action.

Final Systematic G (after col swap, message order 1,2,4,3):

```
G_sys = [ I₃ | P ] =
⎡ 1 0 0 | 1 1 1 ⎤  ← Row g₁ rearranged
⎢ 0 1 0 | 0 1 1 ⎥
⎣ 0 0 1 | 1 1 0 ⎦
```

Parity matrix P extracted:

```
P =
⎡ 1 1 1 ⎤
⎢ 0 1 1 ⎥
⎣ 1 1 0 ⎦
```

H = [P^T | I_3]:

```
H =
⎡ 1 0 1 | 1 0 0 ⎤
⎢ 1 1 1 | 0 1 0 ⎥
⎣ 1 1 0 | 0 0 1 ⎦
```

IN SUMMARY Bas row operations karo (XOR) taaki left side pe identity matrix ban jaye. Jo bachta hai right side mein — woh P matrix hai. Phir H = [P^T | I_r]. Itna hi kaam hai — systematic conversion complete!

Second Complete Example — (7,4) Code

EXAMPLE — (7,4) NON-SYSTEMATIC TO SYSTEMATIC

Given non-systematic G for (7,4):

$$
\begin{aligned}
G &= \\
g_1 &= [\,1\ 1\ 0\ 1\ 0\ 0\ 0\,] \\
g_2 &= [\,0\ 1\ 1\ 0\ 1\ 0\ 0\,] \\
g_3 &= [\,1\ 1\ 1\ 0\ 0\ 1\ 0\,] \\
g_4 &= [\,1\ 0\ 1\ 0\ 0\ 0\ 1\,]
\end{aligned}
$$

Target: Make cols 1-4 = I_4

Col 1: g_1 has 1 in col 1. Eliminate from g_3, g_4:

$$
\begin{aligned}
g_3 &\leftarrow g_3 \oplus g_1 = [1,1,1,0,0,1,0] \oplus [1,1,0,1,0,0,0] = [0,0,1,1,0,1,0] \\
g_4 &\leftarrow g_4 \oplus g_1 = [1,0,1,0,0,0,1] \oplus [1,1,0,1,0,0,0] = [0,1,1,1,0,0,1]
\end{aligned}
$$

After col 1 elimination:

$$
\begin{aligned}
g_1 &= [\,1\ 1\ 0\ 1\ 0\ 0\ 0\,] \\
g_2 &= [\,0\ 1\ 1\ 0\ 1\ 0\ 0\,] \\
g_3 &= [\,0\ 0\ 1\ 1\ 0\ 1\ 0\,] \\
g_4 &= [\,0\ 1\ 1\ 1\ 0\ 0\ 1\,]
\end{aligned}
$$

Col 2: g_2 has 1 in col 2. Eliminate from g_1, g_4:

$$
\begin{aligned}
g_1 &\leftarrow g_1 \oplus g_2 = [1,1,0,1,0,0,0] \oplus [0,1,1,0,1,0,0] = [1,0,1,1,1,0,0] \\
g_4 &\leftarrow g_4 \oplus g_2 = [0,1,1,1,0,0,1] \oplus [0,1,1,0,1,0,0] = [0,0,0,1,1,0,1]
\end{aligned}
$$

$$
\begin{aligned}
g_1 &= [\,1\ 0\ 1\ 1\ 1\ 0\ 0\,] \\
g_2 &= [\,0\ 1\ 1\ 0\ 1\ 0\ 0\,] \\
g_3 &= [\,0\ 0\ 1\ 1\ 0\ 1\ 0\,] \\
g_4 &= [\,0\ 0\ 0\ 1\ 1\ 0\ 1\,]
\end{aligned}
$$

Col 3: g_3 has 1 in col 3. Eliminate from g_1, g_2:

$$g_1 \leftarrow g_1 \oplus g_3 = [1,0,1,1,1,0,0] \oplus [0,0,1,1,0,1,0] = [1,0,0,0,1,1,0]$$
$$g_2 \leftarrow g_2 \oplus g_3 = [0,1,1,0,1,0,0] \oplus [0,0,1,1,0,1,0] = [0,1,0,1,1,1,0]$$

Col 4: g_4 has 1 in col 4. Eliminate from g_2, g_3:

$$g_2 \leftarrow g_2 \oplus g_4 = [0,1,0,1,1,1,0] \oplus [0,0,0,1,1,0,1] = [0,1,0,0,0,1,1]$$
$$g_3 \leftarrow g_3 \oplus g_4 = [0,0,1,1,0,1,0] \oplus [0,0,0,1,1,0,1] = [0,0,1,0,1,1,1]$$

Also g_1: g_1 has 0 in col 4 — no change needed.

✅ **Final Systematic G:**

$$G_{sys} = [\, I_4 \mid P \,] = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix}$$

$$P = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}$$

$H = [P^T \mid I_3]$:

$$H = \begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

Comparison: Systematic vs Non-Systematic

FEATURE	SYSTEMATIC CODE	NON-SYSTEMATIC CODE
Message in codeword	Directly visible (first k bits)	Mixed, not directly visible

G matrix form	**$[I_k \mid P]$**	**No identity block**
H matrix from G	**Easily: $H = [P^T \mid I_r]$**	**Complex computation needed**
Error detection	Same	Same
Decoding complexity	Lower	Higher
Hardware implementation	**Easier, linear feedback shift register**	**More complex**

📌 UNIT 4 — QUICK REVISION SUMMARY

- **(n,k) Linear Block Code:** k message bits → n codeword bits; 2^k codewords; linear (XOR of any 2 codewords = codeword)
- **Code Rate:** R = k/n; Redundancy r = n−k
- **Hamming Code:** n = 2^r-1, k = 2^r-r-1; d_min = 3; corrects 1 error
- **d_min:** detects t_d = d_min−1 errors; corrects t_c = ⌊(d_min−1)/2⌋ errors
- **Generator Matrix G** (k×n): c = m·G; Systematic: G = $[I_k \mid P]$
- **Parity Check Matrix H** (r×n): $H \cdot c^T = 0$ for valid c; $H = [P^T \mid I_r]$
- **Syndrome:** $S = H \cdot r^T$; if S=0 → no error; S≠0 → $S = H \cdot e^T$ identifies error
- **Systematic conversion:** Row reduce G to $[I_k \mid P]$ using GF(2) row operations (XOR rows)
- **$G \cdot H^T = 0$** (always, fundamental relationship)

HINGLISH TRICK

Matrices: $G = [I_k \mid P]$ (k Message + r Parity). **$H = [P^T \mid I_r]$** (Parity transpose + r Identity). **$G \cdot H^T = 0$** pakka zero hoga!
Syndrome Decoding: $S = r \cdot H^T$ (Syndrome S jis column of H se match kare, wahi bit corrupt hai — use flip kar do!).
Hamming Rule: Detects $d_{min} - 1$ errors, Corrects $\lfloor (d_{min}-1)/2 \rfloor$ errors.

🗺️ Unit 4 Overview Map

Quick Revise

UNIT 4: Linear Block Codes (LBC) & Syndrome Decoding

1. LBC Fundamentals

- **Parameters:** (n,k) code, rate $R=k/n$, parity bits $r=n-k$
- **Linearity:** Modulo-2 sum (XOR) of any 2 codewords is a codeword
- **Distance:** $d_{min} = \text{min non-zero weight } w_{min}$
- **Detection/Correction:** $t_d = d_{min}-1$; $t_c = \lfloor (d_{min}-1)/2 \rfloor$

2. Generator & Parity Matrices

- **Generator Matrix:** $G_{k \times n} = [I_k \mid P]$, codeword $c = m \cdot G$
- **Parity Check:** $H_{r \times n} = [P^T \mid I_r]$, checks $c \cdot H^T = 0$
- **Orthogonality:** $G \cdot H^T = 0 \pmod 2$

3. Syndrome Decoding & Hamming

- **Syndrome:** $S = r \cdot H^T = e \cdot H^T$ (1-to-1 error mapping)
- **Standard Array:** Coset leaders represent most likely error patterns
- **Hamming Code:** $(2^r-1, 2^r-r-1)$, $d_{min}=3$, single-error correcting

🎯 RTU EXAM Q&A BANK — UNIT 4

EXAM GOLD

Covering 5-mark & 10-mark questions based on RTU exam patterns. All answers exam-ready.

📌 5-Mark Questions

Q1 · 5 MARKS

5 MARKS

Define Linear Block Code. State its properties. (RTU pattern)

Linear Block Code: A (n,k) linear block code is a code where k message bits are encoded into n-bit codewords (n > k), and the code satisfies the **linearity condition**: the modulo-2 sum (XOR) of any two valid codewords is also a valid codeword. The codewords form a k-dimensional subspace of the n-dimensional binary vector space $GF(2)^n$.

Properties:

- **Linearity:** If $c_1, c_2 \in C$, then $c_1 \oplus c_2 \in C$
- **All-zero codeword:** 0...0 is always a valid codeword
- **Total codewords:** 2^k codewords exist
- **Closure under XOR:** Set is closed under addition
- **d_min = w_min:** For linear codes, minimum distance = minimum Hamming weight of non-zero codewords

Example: (7,4) Hamming code has $2^4 = 16$ codewords, R = 4/7.

Q2 · 5 MARKS

5 MARKS

What is Hamming distance and Hamming weight? How are they related to error detection and correction?

Hamming Weight w(c): Number of non-zero (1) bits in codeword c. Example: w(10110) = 3.

Hamming Distance $d(c_1,c_2)$: Number of bit positions where two codewords differ. Calculated as $w(c_1 \oplus c_2)$. Example: d(10110, 11010) = w(01100) = 2.

Minimum Distance d_min: Smallest Hamming distance among all pairs of distinct codewords.

Error Detection Capability: A code with d_min can detect up to **$t_d = d_{min} - 1$** errors.

Error Correction Capability: A code with d_min can correct up to **$t_c = \lfloor (d_{min}-1)/2 \rfloor$** errors.

Relation: For linear codes, d_min = minimum weight of any non-zero codeword (w_min).

Example: (7,4) Hamming code has d_min = 3 → detects 2 errors, corrects 1 error.

Q3 · 5 MARKS

5 MARKS

What is the Generator Matrix? Explain its role in encoding.

Generator Matrix G: A k×n matrix used to encode message vectors into codewords.

Encoding: c = m · G (mod-2), where m is 1×k message and c is 1×n codeword.

Structure (Systematic Form): $G = [I_k \mid P]$, where I_k is k×k identity and P is k×r parity matrix.

Properties:

- Rows of G are k linearly independent codewords (basis vectors)
- Every valid codeword is a linear combination of rows of G
- In systematic form, first k bits of c = message bits m (directly readable)
- $G \cdot H^T = 0$ (fundamental property)

Example (7,4): Message m = [1 0 1 1], $G = [I_4 \mid P]$ → compute c = m·G (mod-2) to get codeword.

Q4 · 5 MARKS

5 MARKS

What is Syndrome? Explain syndrome decoding.

Syndrome S: A r×1 vector computed as $S = H \cdot r^T$, where r is the received vector and H is the parity check matrix.

Properties:

- If S = 0 → No error detected (r is a valid codeword)
- If S ≠ 0 → Error detected; $S = H \cdot e^T$ where e is the error pattern
- Syndrome depends only on error pattern e, NOT on the message

Syndrome Decoding Steps:

1. Receive vector r
2. Compute $S = H \cdot r^T$ (mod-2)
3. If S = 0: accept r as valid codeword
4. If S ≠ 0: look up S in syndrome table → find error position
5. Flip the bit at error position → corrected codeword $\hat{c} = r \oplus e$

For (7,4) Hamming: 3-bit syndrome S directly gives decimal position of error (e.g., S=101=5 → bit 5 is in error).

Q5 · 5 MARKS

5 MARKS

What is the difference between systematic and non-systematic codes? Why is systematic form preferred?

Systematic Code: Message bits appear unchanged at fixed positions in the codeword. Codeword = $[m_1 \ldots m_k \mid p_1 \ldots p_r]$. G matrix has form $[I_k \mid P]$.

Non-Systematic Code: Message bits are mixed/transformed with parity bits. Original message cannot be directly extracted from codeword.

Advantages of Systematic Form:

- Message bits directly readable — no extra decoding step
- H matrix easily computed as $H = [P^T \mid I_r]$
- Simpler hardware implementation using shift registers
- Syndrome decoding is straightforward
- Error detection/correction capability is the same as non-systematic

Any linear block code can be converted to systematic form using row operations (Gaussian elimination over GF(2)) without changing the code's error-control properties.

Q6 · 5 MARKS

5 MARKS

Find the parity check matrix H from the given generator matrix G for a (6,3) code.

Given (example):

$$G = [I_3 \mid P] = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix}$$

Step 1: Extract P:

$$P = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}$$

Step 2: Compute P^T (transpose P):

$$P^T = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}$$

Step 3: Form $H = [P^T \mid I_3]$:

$$H = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

Verification: $G \cdot H^T$ should = 0 (mod-2) ✓

📌 10-Mark Questions

Q7 · 10 MARKS

10 MARKS

Explain the (7,4) Hamming code in detail. Show how to encode message [1 0 1 1] and decode a received vector with single-bit error.

Part A: (7,4) Hamming Code Parameters

- n = 7, k = 4, r = 3 (parity bits)
- d_min = 3 → detects 2 errors, corrects 1 error
- Code rate R = 4/7 ≈ 0.57
- Total codewords = 2^4 = 16

Part B: Generator Matrix G (Systematic Form)

```
G =
⎡ 1 0 0 0 | 1 1 0 ⎤
⎢ 0 1 0 0 | 0 1 1 ⎥
⎢ 0 0 1 0 | 1 1 1 ⎥
⎣ 0 0 0 1 | 1 0 1 ⎦
```

Part C: Encoding m = [1 0 1 1]

```
c = m · G = [1 0 1 1] · G (mod-2)
= 1×row₁ ⊕ 0×row₂ ⊕ 1×row₃ ⊕ 1×row₄
= [1,0,0,0,1,1,0] ⊕ [0,0,0,0,0,0,0] ⊕ [0,0,1,0,1,1,1] ⊕ [0,0,0,1,1,0,1]
= [1,0,1,1,1,0,0]
∴ Codeword c = 1011100
```

Part D: Parity Check Matrix H

```
H =
⎡ 1 0 1 1 | 1 0 0 ⎤
⎢ 1 1 1 0 | 0 1 0 ⎥
⎣ 0 1 1 1 | 0 0 1 ⎦
```

Part E: Decoding with Error

Transmitted: c = 1011100. Error introduced at bit 5 → r = 1011000

Compute Syndrome $S = H \cdot r^T$ (mod-2):

```
r = [1 0 1 1 0 0 0]
S₁ = 1·1+0·0+1·1+1·1+0·1+0·0+0·0 = 1+0+1+1+0+0+0 = 3 mod 2 = 1
S₂ = 1·1+0·1+1·1+1·0+0·0+0·1+0·0 = 1+0+1+0+0+0+0 = 2 mod 2 = 0
S₃ = 0·1+1·0+1·1+1·1+0·0+0·0+0·1 = 0+0+1+1+0+0+0 = 2 mod 2 = 0...
```

S = [1,0,1] = 101_2 = 5 → Error at bit position 5

Flip bit 5: r = 1011<u>0</u>00 → corrected = 1011100 = c ✓

Decoded message = first 4 bits = 1011 ✓

Q8 · 10 MARKS

10 MARKS

Explain the conversion of non-systematic generator matrix to systematic form. Give a complete example.

Definition: A systematic code has message bits appearing unchanged in fixed positions of the codeword. The generator matrix in systematic form is G = [I_k | P].

Need for Conversion: Any given G matrix may not be in systematic form. Using row operations over GF(2) (mod-2 arithmetic), any G can be converted to systematic form without changing the code's capabilities.

Method — Gaussian Elimination over GF(2):

- Allowed operations: Add any two rows (XOR and replace); Swap two rows
- Goal: Transform G so its first k columns form identity matrix I_k

Complete Example — Convert (6,3) G to systematic:

Given:

```
G =
g₁ = [ 1 1 0 0 1 0 ]
g₂ = [ 0 1 1 0 0 1 ]
g₃ = [ 1 0 1 1 0 0 ]
```

Col 1 pivot = g_1. Eliminate from g_3 (has 1 in col 1):

```
g₃ ← g₃ ⊕ g₁ = [0,1,1,1,1,0]
```

Matrix: g_1=[1,1,0,0,1,0], g_2=[0,1,1,0,0,1], g_3=[0,1,1,1,1,0]

Col 2 pivot = g_2. Eliminate from g_1, g_3:

```
g₁ ← g₁ ⊕ g₂ = [1,0,1,0,1,1]
g₃ ← g₃ ⊕ g₂ = [0,0,0,1,1,1]
```

Matrix: g_1=[1,0,1,0,1,1], g_2=[0,1,1,0,0,1], g_3=[0,0,0,1,1,1]

Col 3: g_1 and g_2 have 1 in col 3, g_3 has 0. Need g_3 to have 1 in col 3.

Since g_3 = [0,0,0,1,1,1] has 0 in col 3, swap col 3 and col 4:

```
g₁=[1,0,0,1,1,1], g₂=[0,1,0,1,0,1], g₃=[0,0,1,0,1,1]
(note: col 3 and 4 swapped — bit ordering changed)
```

Col 3 now: g_3 has 1. Eliminate from g_1 (has 0 — no action), g_2 (has 0 — no action).

Final Systematic G_sys:

```
G_sys = [ I₃ | P ] =
⎡ 1 0 0 | 1 1 1 ⎤
⎢ 0 1 0 | 1 0 1 ⎥
⎣ 0 0 1 | 0 1 1 ⎦
```

Parity matrix P and H:

```
P = ⎡1 1 1⎤ ⎢1 0 1⎥ ⎣0 1 1⎦

H = [Pᵀ | I₃] =
⎡1 1 0 | 1 0 0⎤
⎢1 0 1 | 0 1 0⎥
⎣1 1 1 | 0 0 1⎦
```

Conclusion: The code's error-detection and correction capabilities remain unchanged after conversion. Only the representation changes.

Q9 · 10 MARKS

10 MARKS

What are error correcting codes? Explain minimum distance consideration and its relation to error detection/correction capability.

Error Correcting Codes: Error correcting codes (ECC) are systematic methods of adding redundancy to information bits so that errors introduced during transmission can be detected and corrected at the receiver without retransmission. They implement Forward Error Correction (FEC).

Why Needed: Physical communication channels introduce noise, causing bit flips. ECC ensures data integrity without a feedback channel.

Types: Block codes (Hamming, BCH, Reed-Solomon) and Convolutional codes.

Minimum Hamming Distance (d_min):

- d_min = minimum Hamming distance between any two distinct codewords in the code
- For linear codes: d_min = minimum Hamming weight of any non-zero codeword
- d_min determines the code's power to detect and correct errors

Geometric Interpretation: Codewords are points in n-dimensional binary space. d_min is the minimum distance between any two codeword "points." Larger d_min = codewords are more spread out = more errors can be tolerated.

Error Detection Capability:

A code with d_min can detect any pattern of up to **$t_d = d_{min} - 1$** errors.

Reason: An error pattern of ≤ t_d errors cannot transform one codeword into another (they differ in ≥ d_min positions). So the receiver knows an error occurred.

Error Correction Capability:

A code with d_min can correct any pattern of up to **$t_c = \lfloor (d_{min}-1)/2 \rfloor$** errors.

Reason: Decoding assigns each received vector to the nearest codeword (minimum distance decoding). If ≤ t_c errors occur, the nearest codeword is always the correct one.

Combined Detection and Correction:

To simultaneously correct s errors AND detect t errors (t ≥ s): d_min ≥ t + s + 1

Examples:

D_MIN	DETECT	CORRECT	EXAMPLE CODE
3	2	1	(7,4) Hamming

5	4	2	BCH code
7	6	3	BCH code

Singleton Bound: For any (n,k) code: $d_{min} \leq n - k + 1$

Hamming Bound (Sphere Packing Bound): For a t-error correcting (n,k) code:

$$2^{(n-k)} \geq \sum_{i=0}^{t} C(n,i)$$

Q10 · 10 MARKS

10 MARKS

Given a (7,4) linear block code with G matrix, find all codewords, determine d_min, and verify error correction capability.

Given G (systematic form):

```
G =
[ 1 0 0 0 | 1 1 0 ]
| 0 1 0 0 | 0 1 1 |
| 0 0 1 0 | 1 1 1 |
[ 0 0 0 1 | 1 0 1 ]
```

All $2^4 = 16$ codewords (message → codeword):

MESSAGE M	CODEWORD C = M·G	WEIGHT W(C)
0000	0000000	0
0001	0001101	3
0010	0010111	4
0011	0011010	3
0100	0100011	3
0101	0101110	4
0110	0110100	3
0111	0111001	4
1000	1000110	3
1001	1001011	4
1010	1010001	3
1011	1011100	4
1100	1100101	4

1101	1101000	3
1110	1110010	4
1111	1111111	7

d_min: Minimum weight of non-zero codewords = **3**

Error Detection: $t_d = 3-1 = 2$ (can detect 2 errors)

Error Correction: $t_c = \lfloor (3-1)/2 \rfloor = 1$ (can correct 1 error)

Verification: (7,4) Hamming code has d_min = 3 by definition ✓

Q11 · 10 MARKS

10 MARKS

Explain encoding and decoding of a (6,3) linear block code with full G and H matrices. Show syndrome calculation for error detection.

(6,3) Code: n=6, k=3, r=3 parity bits, R=3/6=0.5

Generator Matrix (Systematic):

```
G = [I₃ | P] =
⎡ 1 0 0 | 1 1 0 ⎤
⎢ 0 1 0 | 1 0 1 ⎥
⎣ 0 0 1 | 0 1 1 ⎦
```

Encoding Example — m = [1 1 0]:

```
c = m · G = [1 1 0] · G
= 1×[1,0,0,1,1,0] ⊕ 1×[0,1,0,1,0,1] ⊕ 0×[0,0,1,0,1,1]
= [1,0,0,1,1,0] ⊕ [0,1,0,1,0,1]
= [1,1,0,0,1,1]
```

Codeword: `110011` — first 3 bits = message 110 ✓ (systematic!)

Parity Check Matrix H = [Pᵀ | I₃]:

```
Pᵀ = ⎡1 1 0⎤ᵀ = ⎡1 1 0⎤
⎢1 0 1⎥ ⎢1 0 1⎥ → wait, transpose properly:
⎣0 1 1⎦

P columns become rows in Pᵀ:
Pᵀ = ⎡1 1 0⎤
⎢1 0 1⎥
⎣0 1 1⎦

H = [Pᵀ | I₃] =
⎡1 1 0 | 1 0 0⎤
⎢1 0 1 | 0 1 0⎥
⎣0 1 1 | 0 0 1⎦
```

Verification G·Hᵀ = 0: (Students should verify row by row = all zero mod-2)

Error Scenario: Transmitted c = 110011, error at bit 4, received r = 110111 → r = 110111

Syndrome $S = H \cdot r^T$:

$$r = [1\ 1\ 0\ 1\ 1\ 1]$$
$$S_1 = 1\cdot1 + 1\cdot1 + 0\cdot0 + 1\cdot1 + 1\cdot0 + 1\cdot0 = 1+1+0+1+0+0 = 3 \bmod 2 = 1$$
$$S_2 = 1\cdot1 + 1\cdot0 + 0\cdot1 + 1\cdot0 + 1\cdot1 + 1\cdot0 = 1+0+0+0+1+0 = 2 \bmod 2 = 0$$
$$S_3 = 1\cdot0 + 1\cdot1 + 0\cdot1 + 1\cdot0 + 1\cdot0 + 1\cdot1 = 0+1+0+0+0+1 = 2 \bmod 2 = 0$$
$$S = [1\ 0\ 0]$$

S=[1,0,0] matches column 4 of H → Error at position 4.

Corrected: flip bit 4 in r=[1,1,0,1,1,1] → c=[1,1,0,0,1,1] ✓

Decoded message = first 3 bits = [1 1 0] ✓

Q12 · 5 MARKS — SHORT NOTE

5 MARKS

Write a short note on: Standard Array Decoding.

Standard Array is a complete decoding table for a (n,k) linear block code, organized as a 2^(n−k) × 2^k array.

Construction:

- **Row 1 (top):** All 2^k valid codewords. First entry is the all-zero codeword.
- **Row 2:** Coset leader e_2 (lowest weight unassigned vector) followed by $e_2 \oplus$ each codeword
- **Row i:** Coset leader e_i followed by $e_i \oplus$ each codeword
- Continue until all 2^n binary vectors are placed

Decoding Procedure:

1. Receive vector r
2. Find r in the standard array
3. Identify which column r is in
4. The top of that column = decoded codeword ĉ
5. OR: find r's row → coset leader e_i = error pattern → $\hat{c} = r \oplus e_i$

Property: A standard array performs *minimum distance decoding* — coset leaders are the most likely error patterns (lowest weight). Any received vector is decoded to the codeword with minimum Hamming distance from it.

Limitation: For large n, the table is exponentially large (2^n entries). Syndrome decoding is more practical.

Q13 · 5 MARKS

5 MARKS

For a (7,4) code with d_min = 3: (a) How many errors can it detect? (b) How many can it correct? (c) What if we want to detect 3 errors — what d_min is required?

(a) Error Detection: $t_d = d_{min} - 1 = 3 - 1 =$ **2 errors** can be detected.

(b) Error Correction: $t_c = \lfloor (d_{min}-1)/2 \rfloor = \lfloor 2/2 \rfloor =$ **1 error** can be corrected.

(c) For detecting 3 errors:

Required: $t_d = 3 \rightarrow d_{min} = t_d + 1 = 3 + 1 =$ **$d_{min} = 4$**

A code with $d_{min} = 4$ can detect 3 errors (and still correct 1).

Summary Table:

REQUIRED CAPABILITY	REQUIRED D_MIN
Detect 2 errors	3
Detect 3 errors	4
Correct 1 error	3
Correct 2 errors	5
Correct 1 + Detect 2 simultaneously	4 (= 1+2+1)

ZYREX // ITC_VAULT — UNIT 4 · Linear Block Code

RTU B.Tech CSE 5th Semester · 5CS3-01 Information Theory & Coding · Unit 4 of 6

📌 Future units can be appended below this file or as separate scrollable sections. All topics from this unit are covered in exam-ready detail.

Formulas to memorize: $c=m\cdot G$ | $S=H\cdot r^T$ | $G=[I_k|P]$ | $H=[P^T|I_r]$ | $d_{min}=2t_c+1$ | $t_d=d_{min}-1$

🔁 Unit 5 — Cyclic Codes

Code Algebra · Galois Fields (GF) · Polynomial Operations over GF · Generator Polynomial · Parity Check Polynomial · Encoder & Decoder for Cyclic Codes

🔥 HIGH WEIGHT — 10-12 Marks Expected ⚠ 5-Mark: GF Properties, Generator Poly ✅ Unit 5 = 6 Hours (Total Course = 28 Hours)

📑 Unit 5 Quick Navigation

5.1 Introduction to Cyclic Codes
5.2 Code Algebra
5.3 Galois Fields GF(2) and GF(2n)
5.4 Polynomial Operations over GF
5.5 Generator Polynomial g(x)
5.6 Parity Check Polynomial h(x)
5.7 Cyclic Code Encoder (LFSR)
5.8 Cyclic Code Decoder
5.9 Comparison Tables
5.10 RTU Exam Q&A

5.1 Introduction to Cyclic Codes EXAM HOT

📖 DEFINITION

Cyclic Code is a special subclass of linear block codes where every cyclic shift of a valid codeword produces another valid codeword in the same code. If $c = (c_0, c_1, ..., c_{n-1})$ is a codeword, then $(c_{n-1}, c_0, c_1, ..., c_{n-2})$ is also a valid codeword.

🗣 HINGLISH EXPLANATION

Socho ek circle mein balls rakhi hain. Ek shift karo toh wahi arrangement phir valid hai! Cyclic code mein kisi bhi codeword ko left ya right shift karo, result bhi same code ka valid member hoga. Isliye inhe **polynomial algebra** se describe karna bahut easy hota hai aur hardware (shift register) se implement karna fast hota hai!

💡 EXAMPLE: CYCLIC PROPERTY

In (7,4) cyclic code, if **1011000** is a codeword, then ALL cyclic shifts are also valid codewords:

```
Original: 1 0 1 1 0 0 0 Shift 1: 0 1 0 1 1 0 0 Shift 2: 0 0 1 0 1 1 0
Shift 3: 0 0 0 1 0 1 1 Shift 4: 1 0 0 0 1 0 1 Shift 5: 1 1 0 0 0 1 0 Shift
6: 0 1 1 0 0 0 1
```

All 7 cyclic shifts are valid codewords! ✓

Key Advantages of Cyclic Codes

- 🔧 **Hardware Efficiency:** LFSR (Linear Feedback Shift Register) + XOR gates only
- 🧮 **Algebraic Description:** Entire code defined by ONE polynomial g(x)
- 📡 **Burst Error Detection:** Excellent for detecting consecutive bit errors
- ⚡ **Fast Implementation:** Much faster than matrix-based encoding
- 📘 **Examples in use:** CRC codes, BCH codes, Reed-Solomon, Hamming codes

Polynomial Representation of Codewords MUST KNOW

Any n-bit codeword c = (c_0, c_1, ..., c_{n-1}) is represented as:

$$c(x) = c_0 + c_1X + c_2X^2 + \ldots + c_{n-1}X^{n-1}$$

Each coefficient c^i is from GF(2) = {0, 1}. All arithmetic is mod 2 (XOR).

💡 CODEWORD TO POLYNOMIAL

Codeword: **1 0 1 1 0 0 0** (n=7)

$$c(x) = 1 + 0x + 1x^2 + 1x^3 + 0x^4 + 0x^5 + 0x^6 = 1 + x^2 + x^3$$

5.2 Code Algebra Basics 5-MARK

📖 DEFINITION

Code Algebra treats cyclic codes as ideals in the polynomial ring R = GF(2)[x]/(x^n+1). A cyclic code C is an *ideal* of this ring, meaning: if c(x) is in C and r(x) is any polynomial in R, then r(x)·c(x) mod(x^n+1) is also in C. This ideal structure is what guarantees the cyclic property.

🗣️ HINGLISH

Code Algebra matlab polynomials ki ring mein kaam karna. Jaise integers mein "even numbers" ek ideal hain (even+even=even, even×anything=even), waise hi cyclic codes polynomials ki ring mein ideals hain. Yeh algebraic structure automatically cyclic property guarantee karta hai!

Ring Structure: R = GF(2)[x] / (x^n+1)

- All polynomial coefficients from GF(2) = {0,1} only
- Addition = XOR of coefficients (mod 2)
- Multiplication = polynomial product, then mod (x^n+1)
- Key: $x^n \equiv 1$ mod(x^n+1), since x^n+1 ≡ 0 means x^n = -1 = +1 in GF(2)

```
Cyclic shift: x·c(x) mod(xⁿ+1) = one cyclic left shift of codeword c
```

Field Axioms in GF(2)

PROPERTY	RULE	GF(2) EXAMPLE
Closure	a+b and a·b stay in field	1+1=0 ∈ {0,1} ✓
Commutativity	a+b=b+a, a·b=b·a	1+0=0+1=1 ✓
Associativity	(a+b)+c=a+(b+c)	Works mod-2 ✓
Distributivity	a·(b+c)=a·b+a·c	1·(1+0)=1=1+0 ✓
Additive Identity	a+0=a	1+0=1 ✓
Multiplicative Identity	a·1=a	1·1=1 ✓
Additive Inverse	a+a=0 (so -a=a in GF(2))	1+1=0 ✓

5.3 Galois Fields GF(2) and GF(2^n)

10-MARK TOPIC

📖 DEFINITION — GALOIS FIELD

A **Galois Field GF(q)** (finite field) is a set of q elements with two operations (+ and ·) satisfying all field axioms. GF(q) exists if and only if **$q = p^n$** where p is prime and n ≥ 1. Named after Evariste Galois (French mathematician, 1811-1832).

🗣 HINGLISH

Galois Field ek *finite* (simit) set hai jisme addition aur multiplication dono properly defined hain. "Finite" isliye kyunki elements ki ginti limited hai. GF(2) mein sirf 2 elements: {0,1}. Digital systems ke liye perfect hai!

GF(2) — The Binary Field MUST KNOW

GF(2) = {0, 1} — simplest finite field, foundation of all binary coding.

Addition Table (XOR)

+	0	1
0	0	1
1	1	0

Critical: 1+1=0 (no carry!)

Multiplication Table (AND)

×	0	1
0	0	0
1	0	1

Normal AND operation

GF(2^n) — Extension Field Construction 10-MARK CRITICAL

GF(2^n) has exactly **2^n elements**. Built from GF(2) using an irreducible polynomial p(x) of degree n:

```
GF(2^n) = GF(2)[x] / p(x) [p(x) = irreducible polynomial of degree n]
```

Elements: all polynomials of degree ≤ n-1 with coefficients from GF(2). Total = 2^n elements. Introduce primitive element α as root of p(x): p(α) = 0. Then {α°, α^1, ..., α^(2^n-2)} = all 2^n-1 non-zero elements.

HINGLISH

GF(2^n) banane ke liye ek "irreducible polynomial" p(x) chahiye — yeh GF(2) mein factor nahi hota (like prime numbers). Is polynomial ka root α (alpha) introduce karte hain. Ab α ki powers se 2^n elements ka ek field banta hai. GF(8) mein 8 elements hain, primitive element α ki 7 distinct powers hain (phir cycle!)

💡 COMPLETE CONSTRUCTION OF GF(2^3) = GF(8) USING P(X) = X^3+X+1

Step 1: Verify p(x) is irreducible:

```
p(0) = 0+0+1 = 1 ≠ 0 ✓ p(1) = 1+1+1 = 1 ≠ 0 ✓ [no roots in GF(2) ⇒ irreducible ✓]
```

Step 2: Let α be root: $\alpha^3+\alpha+1=0 \Rightarrow \alpha^3 = \alpha+1$

Step 3: Generate all 8 elements:

POWER	DERIVATION	POLYNOMIAL FORM	VECTOR ($A_2A_1A_0$)
0 (zero)	—	0	000
$\alpha^\circ = 1$	Identity	1	001
α^1	—	α	010
α^2	—	α^2	100
α^3	$p(\alpha)=0 \Rightarrow \alpha^3=\alpha+1$	$\alpha+1$	011
α^4	$\alpha\cdot\alpha^3=\alpha(\alpha+1)=\alpha^2+\alpha$	$\alpha^2+\alpha$	110
α^5	$\alpha\cdot\alpha^4=\alpha^3+\alpha^2=(\alpha+1)+\alpha^2$	$\alpha^2+\alpha+1$	111
α^6	$\alpha\cdot\alpha^5=\alpha^3+\alpha^2+\alpha=(\alpha+1)+\alpha^2+\alpha$	α^2+1	101
$\alpha^7=\alpha^\circ$	$\alpha\cdot\alpha^6=\alpha^3+\alpha=(\alpha+1)+\alpha=1$	1 (cycle!)	001

GF(8) = {0, 1, α, α^2, α^3, α^4, α^5, α^6} — 8 elements, period = 7 = 2^3-1 ✓

Properties of GF(2^n) — 5-Mark Answer Ready EXAM LIST

1. **Order:** GF(2^n) has exactly 2^n elements (1 zero + 2^n-1 non-zero)
2. **Characteristic 2:** a+a=0 for ALL elements a (since 1+1=0 in GF(2))
3. **Primitive Element α:** Exists α of order (2^n-1) such that {α^0, α^1, ..., α^(2^n-2)} = all non-zero elements
4. **Fermat's Theorem:** a^(2^n-1) = 1 for every non-zero a in GF(2^n)
5. **Additive Group:** (GF(2^n), +) is an abelian group of order 2^n
6. **Multiplicative Group:** (GF(2^n)\{0}, ×) is a cyclic group of order 2^n-1
7. **Subfield:** GF(2^m) is a subfield of GF(2^n) if and only if m divides n
8. **Construction:** GF(2^n) = GF(2)[x]/p(x) where p(x) is irreducible of degree n
9. **Conjugates:** If β is a root of p(x), then β^2, β^4, β^8, ... are also roots (Frobenius)

Common Irreducible Polynomials over GF(2)

DEGREE N	FIELD	IRREDUCIBLE POLYNOMIAL P(X)	KEY RELATION
2	GF(4)	x^2+x+1	$\alpha^2=\alpha+1$
3	GF(8)	x^3+x+1	$\alpha^3=\alpha+1$
3	GF(8)	x^3+x^2+1	$\alpha^3=\alpha^2+1$
4	GF(16)	x^4+x+1	$\alpha^4=\alpha+1$
5	GF(32)	x^5+x^2+1	$\alpha^5=\alpha^2+1$

Test irreducibility (degree 2,3): Check p(0)≠0 AND p(1)≠0 over GF(2).

5.4 Polynomial Operations over GF(2) VERY IMPORTANT

🗣️ HINGLISH INTRO

GF(2) mein polynomial operations normal algebra jaisi hain — bas ek change: **subtraction = addition = XOR** (kyunki -1=+1 in GF(2)). Koi carry nahi, koi negative numbers nahi. Yeh coding theory ka heart hai!

1. Polynomial Addition — XOR Coefficients

Add coefficient-by-coefficient using XOR. Same-power terms cancel if both are 1 (since 1+1=0).

💡 EXAMPLE: ADDITION

a(x) = x^4+x^3+x+1 and **b(x) = x^3+x^2+1**

a(x): $x^4 + x^3 + 0 + x + 1$ b(x): $0 + x^3 + x^2 + 0 + 1$ --- Sum: $x^4 + 0 + x^2 + x + 0 = x^4 + x^2 + x$ Binary: [1,1,0,1,1] XOR [0,1,1,0,1] = [1,0,1,1,0] ✓

2. Polynomial Multiplication

Multiply normally (distribute), then XOR coefficients of same powers.

💡 EXAMPLE: $(X+1)(X^2+X+1)$

$= x\cdot(x^2+x+1) + 1\cdot(x^2+x+1) = (x^3+x^2+x) + (x^2+x+1) = x^3 + (1+1)x^2 + (1+1)x + 1 = x^3 + 0 + 0 + 1 = x^3 + 1$ Note: In GF(2), $(x+1)^2 = x^2+1$ (NOT x^2+2x+1, since $2x=0$)

3. Polynomial Division (Long Division) RTU EXAM STAPLE

Find q(x) and r(x) such that: **a(x) = q(x)·b(x) + r(x)**, where deg(r) < deg(b). Subtraction = XOR in GF(2).

💡 STEP-BY-STEP DIVISION: $(X^6+X^4+X^2+X+1) \div (X^3+X+1)$

Dividend: $x^6 + 0 + x^4 + 0 + x^2 + x + 1$ Divisor: $x^3 + x + 1$ Step 1: $x^6 \div x^3 = x^3$ Multiply: $x^3(x^3+x+1) = x^6+x^4+x^3$ XOR with dividend: $0+0+x^3+x^2+x+1 = x^3+x^2+x+1$ Step 2: $x^3 \div x^3 = 1$ Multiply: $1(x^3+x+1) = x^3+x+1$ XOR: $(x^3+x^2+x+1)-(x^3+x+1) = x^2$ Step 3: $\deg(x^2) = 2 < \deg(x^3) = 3 \Rightarrow$ STOP! Result: $q(x)=x^3+1$, $r(x)=x^2$ Check: $(x^3+1)(x^3+x+1) + x^2 = x^6+x^4+x^2+x+1$ ✓

🗣 HINGLISH TRICK

Division trick: normal long division karo lekin subtract = XOR karo. Jab remainder ka degree divisor se chhota ho jaye, wahan rukna hai. Yahi remainder cyclic encoder mein **parity bits** banta hai!

Modular Arithmetic — Key Identity

$x^n \equiv 1 \bmod (x^n+1)$ [since $x^n+1 \equiv 0$ means $x^n = -1 = +1$ in GF(2)] $x \cdot c(x) \bmod (x^n+1)$ = one cyclic left shift of codeword $c(x)$

5.5 Generator Polynomial g(x) 10-MARK GOLD

📖 DEFINITION

The **generator polynomial g(x)** of an (n,k) cyclic code is the unique monic polynomial of minimum degree (n-k) such that:

1. g(x) divides (x^n+1): g(x) | (x^n+1)
2. Every codeword c(x) is a multiple of g(x)
3. deg[g(x)] = n-k (equals number of parity bits)

```
deg[g(x)] = n - k (parity bit count) g(x) | (xⁿ + 1) (g divides xⁿ+1) c(x)
= m(x) · g(x) (non-systematic encoding)
```

🗣 HINGLISH

Generator polynomial g(x) ek "master key" jaisa hai. Sare valid codewords isi g(x) ke multiples hote hain — jaise 3 ka pahaada (3,6,9,...). Agar g(x) pata hai to 2^k codewords generate ho sakte hain. Iska degree = (n-k) = parity bits ki sankhya!

Step-by-Step: How to Find g(x) EXAM PROCEDURE

1. **Factorize (x^n+1)** into irreducible polynomials over GF(2)
2. **Choose factors** whose product has degree exactly (n-k)
3. **That product = g(x)**
4. **Verify:** g(x) divides (x^n+1) exactly (no remainder)

💡 FIND G(X) FOR (7,4) CYCLIC CODE

n=7, k=4 ⇒ deg[g(x)] = n-k = 3

Step 1: Factor x^7+1 over GF(2):

$$x^7 + 1 = (x+1)(x^3+x+1)(x^3+x^2+1)$$

Step 2: Need degree-3 factor(s):

$$\text{Option A: } g(x) = x^3+x+1 \text{ [degree 3 ✓]} \quad \text{Option B: } g(x) = x^3+x^2+1 \text{ [degree 3 ✓]}$$

Step 3: Verify Option A divides x^7+1:

$$(x^7+1) / (x^3+x+1) = (x+1)(x^3+x^2+1) \text{ [exact, no remainder ✓]}$$

Non-Systematic Encoding

5-MARK

$$c(x) = m(x) \cdot g(x)$$

Simple multiplication. Drawback: message bits NOT visible in codeword.

💡 NON-SYSTEMATIC: G(X)=X³+X+1, MESSAGE=1101

m(x) = x^3+x^2+1

$$c(x) = (x^3+x^2+1)(x^3+x+1) = x^6+x^4+x^3 + x^5+x^3+x^2 + x^3+x+1 = x^6+x^5+x^4+$$
$$(1+1+1)x^3+x^2+x+1 = x^6+x^5+x^4+x^3+x^2+x+1 \text{ [3 mod 2 = 1] Codeword: 1111111}$$

Systematic Encoding — Complete Procedure

10-MARK CRITICAL

Systematic codeword = [parity bits | message bits]. Message bits appear unchanged!

Step 1: Shift: $T(x) = x^{n-k} \cdot m(x)$ Step 2: Divide: $T(x) = q(x) \cdot g(x) + r(x)$ [r(x) = remainder = parity] Step 3: Codeword: $c(x) = T(x) + r(x) = x^{n-k} \cdot m(x) + r(x)$

Why c(x) is valid: c(x) = q(x)g(x)+r(x)+r(x) = q(x)g(x) since r+r=0. So c(x) mod g(x)=0 ✓

💡 SYSTEMATIC ENCODING: (7,4) CODE, G(X)=X³+X+1

Example 1: m=1101 ⇒ $m(x)=x^3+x^2+1$

$T(x) = x^3(x^3+x^2+1) = x^6+x^5+x^3$ Divide by x^3+x+1: $x^6/x^3=x^3$: sub $x^6+x^4+x^3$, rem=x^5+x^4 $x^5/x^3=x^2$: sub $x^5+x^3+x^2$, rem=$x^4+x^3+x^2$ $x^4/x^3=x$: sub x^4+x^2+x, rem=x^3+x $x^3/x^3=1$: sub x^3+x+1, rem=1 r(x)=1 ⇒ parity=001 Codeword: 001 1101

Example 2: m=1010 ⇒ $m(x)=x^3+x$

$T(x) = x^3(x^3+x) = x^6+x^4$ Divide by x^3+x+1: $x^6/x^3=x^3$: sub $x^6+x^4+x^3$, rem=x^3 $x^3/x^3=1$: sub x^3+x+1, rem=x+1 r(x)=x+1 ⇒ parity=011 Codeword: 011 1010

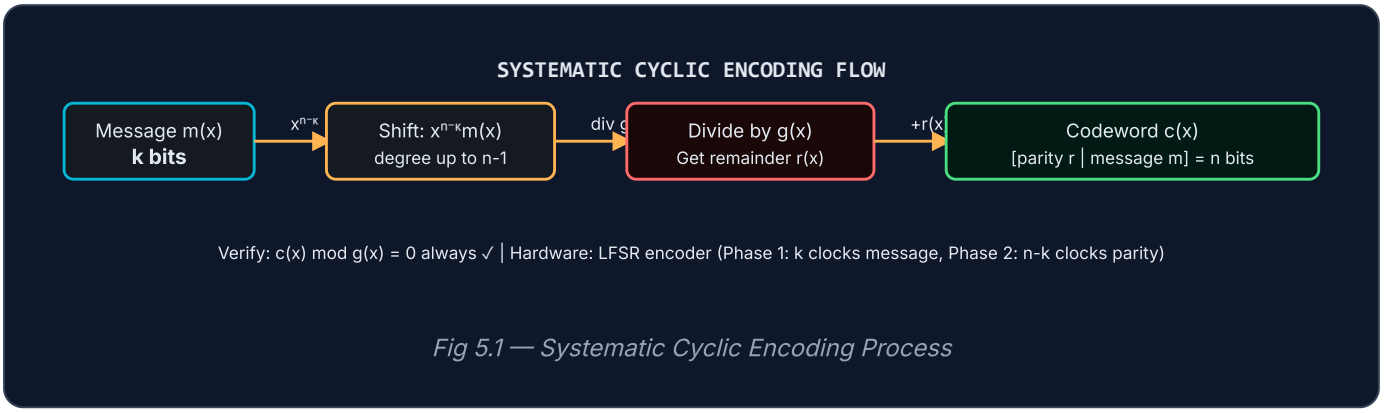


Fig 5.1 — Systematic Cyclic Encoding Process

5.6 Parity Check Polynomial h(x) EXAM MUST

📖 DEFINITION

The **parity check polynomial h(x)** is:

```
h(x) = (xⁿ + 1) / g(x) [exact division] deg[h(x)] = k [equals message bit count] g(x) · h(x) = xⁿ + 1 [fundamental relationship]
```

A polynomial v(x) is a valid codeword if and only if: **v(x) · h(x) ≡ 0 mod(xⁿ+1)**

🗣️ HINGLISH

Agar g(x) "generator" hai jo codewords banata hai, to h(x) "validator" hai jo check karta hai ki polynomial valid codeword hai ya nahi. Dono milke poora (x^n+1) banate hain — perfect pair! Degree hamesha k hoti hai (message length).

💡 FIND H(X) FOR (7,4) CODE, G(X)=X³+X+1

$x^7+1 = (x+1)(x^3+x+1)(x^3+x^2+1)$

```
h(x) = (x⁷+1)/(x³+x+1) = (x+1)(x³+x²+1) Expand: (x+1)(x³+x²+1) = x⁴+x³+x + x³+x²+1 = x⁴+(1+1)x³+x²+x+1 = x⁴ + x² + x + 1 deg[h(x)] = 4 = k ✓ Verify: (x³+x+1)(x⁴+x²+x+1) = x⁷+1 ✓
```

g(x) and h(x) Summary Table EXAM TABLE

PARAMETER	GENERATOR G(X)	PARITY CHECK H(X)
Degree	n-k (parity bits)	k (message bits)
Role	Creates codewords	Validates codewords
Relationship	$g(x) \mid (x^n+1)$	$h(x) = (x^n+1)/g(x)$
Product	$g(x) \cdot h(x) = x^n+1$	
Matrix	G: k×n generator matrix	H: (n-k)×n parity check matrix
Valid codeword	$c(x) \bmod g(x) = 0$	$c(x) \cdot h(x) \bmod (x^n+1) = 0$

5.7 Cyclic Code Encoder — LFSR 10-MARK DIAGRAM

📖 DEFINITION

A cyclic encoder uses a **Linear Feedback Shift Register (LFSR)** with (n-k) stages to compute parity $r(x) = x^{n-k} \cdot m(x) \bmod g(x)$. The LFSR connections are determined by the coefficients of g(x).

🗣 HINGLISH — LFSR ENCODER

LFSR encoder mein (n-k) flip-flops hain ek chain mein. Message bits ek-ek karke input mein jaate hain. g(x) ke coefficients feedback taps determine karte hain. k bits process hone ke baad, register = parity bits! Hardware mein bohot fast hai.

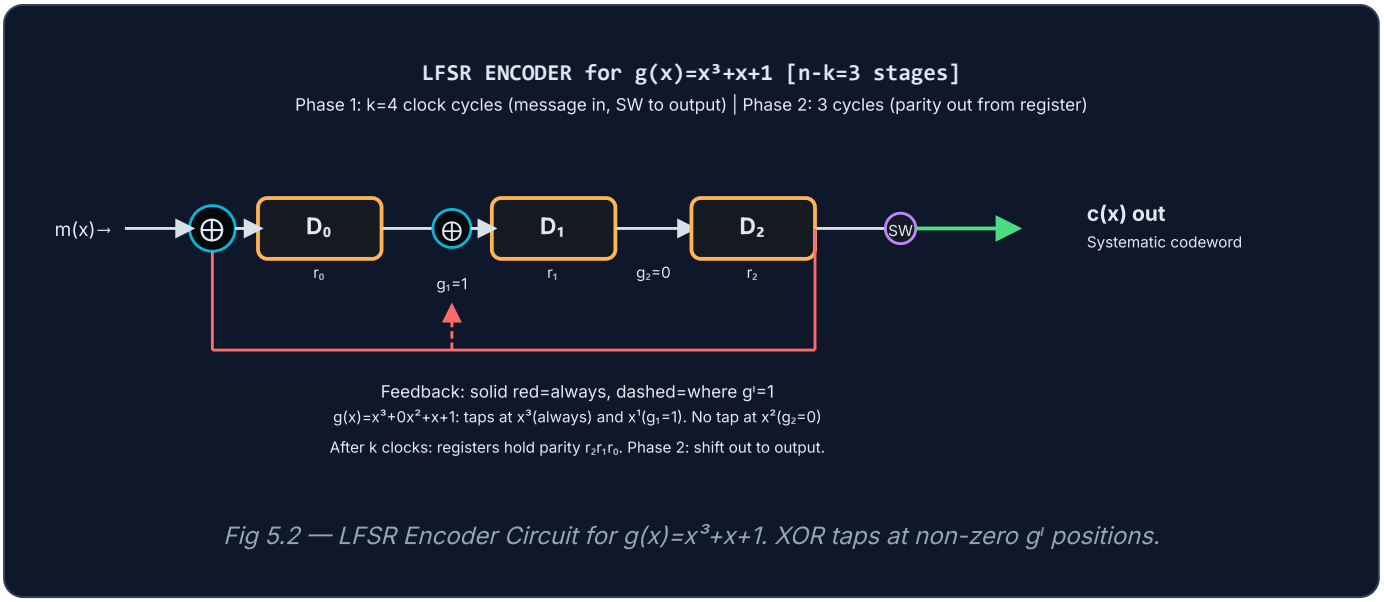


Fig 5.2 — LFSR Encoder Circuit for g(x)=x³+x+1. XOR taps at non-zero gⁱ positions.

Encoder Phase-by-Phase Operation

PHASE	DURATION	SWITCH	INPUT	ACTION
Phase 1	k clock cycles	→ Output channel	Message bits $m^{k}_{-1},...,m_0$	Message flows to output; LFSR computes remainder
Phase 2	n-k clock cycles	→ Register	0 (grounded)	Register contents shift out as parity bits

Final output: [message bits] [parity bits] = complete systematic codeword

💡 ENCODER TRACE: G(X)=X³+X+1, M=1101 (K=4)

LFSR: Next state: $D_2=D_1$, $D_1=D_0 \oplus$ feedback, $D_0=$input$\oplus$feedback (where feedback=D_2 output)

```
Init: [D₂,D₁,D₀]=[0,0,0] Clk| In(mᴵ)| Feedback | D₂ D₁ D₀ (new) | Out(Ph1) 1 | 1 | 0 | 0 0 1 | 1 2 | 1 | 0 | 0 1 1 | 1 3 | 0 | 0 | 1 1 0 | 0 4 | 1 | 1 | 1 0 0 | 1
[Registers: 1,0,0 ⇒ check: systematic encode via division gave 001] Note: Actual trace depends on exact LFSR wiring. The parity bits = r(x) = x³m(x) mod g(x). For m=1101: computed earlier = parity 001. Codeword = 1101 001
```

5.8 Cyclic Code Decoder — Syndrome Method

IMPORTANT

📖 DEFINITION

The cyclic decoder computes **syndrome S(x) = r(x) mod g(x)** from received word r(x).

```
r(x) = c(x) + e(x) [received = sent + error] S(x) = r(x) mod g(x) =
[c(x)+e(x)] mod g(x) = e(x) mod g(x) [since c(x) mod g(x)=0]
```

S(x) depends ONLY on the error e(x), not on c(x). This is what enables error correction!

🗣 HINGLISH

Decoder mein bhi ek LFSR hai — received bits r(x) ko g(x) se divide karo. Remainder = syndrome. Syndrome=0 means no error. Syndrome≠0 means error hai aur syndrome ki value se error location pata chalta hai!

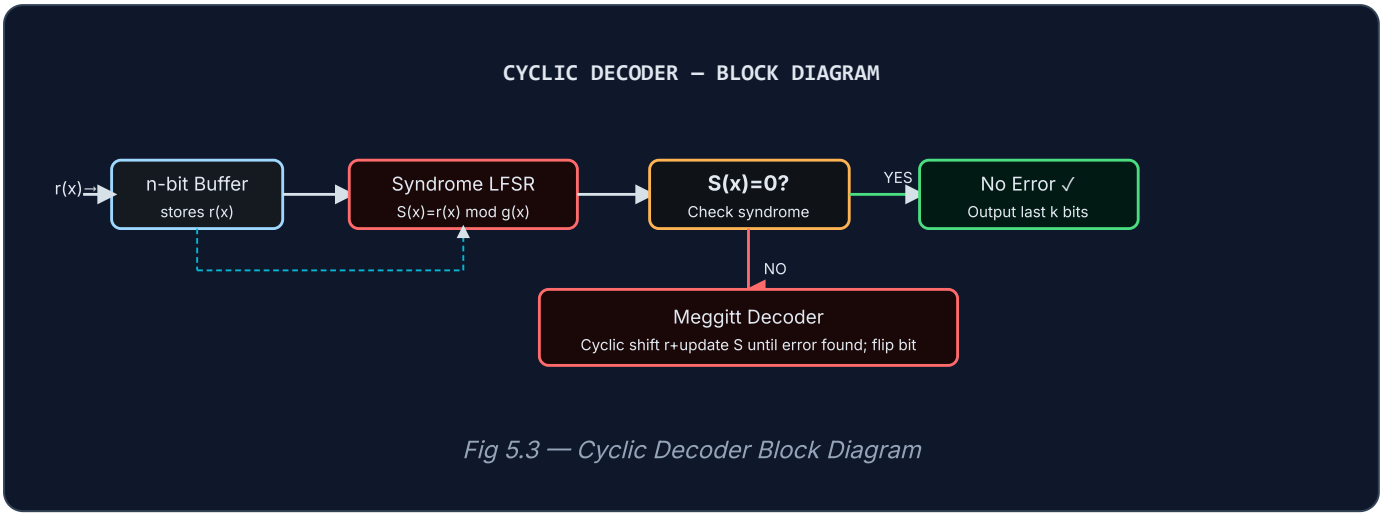


Fig 5.3 — Cyclic Decoder Block Diagram

Single Error Syndromes for (7,4) code, $g(x)=x^3+x+1$ MEMORIZE

ERROR POSITION I	ERROR E(X)	S(X) = X' MOD G(X)	SYNDROME BITS ($S_2S_1S_0$)
0	1	1	001
1	x	x	010
2	x^2	x^2	100
3	x^3	x+1 (x^3=x+1 mod g)	011
4	x^4	x^2+x	110
5	x^5	x^2+x+1	111
6	x^6	x^2+1	101

All 7 syndromes are unique ⇒ any single-bit error can be uniquely identified and corrected ✓

💡 COMPLETE DECODING EXAMPLE

Sent: 0011101 | Error at bit 4 | Received: 0001101

r(x) = 1 + x^2 + x^3 (bits at positions 0,2,3)

Divide $r(x)=1+x^2+x^3$ by $g(x)=x^3+x+1$: $x^3/x^3=1$: sub x^3+x+1, rem=x^2+x+1 deg(x^2+x+1)=2 < 3 ⇒ STOP $S(x)=x^2+x+1$ ⇒ syndrome=111 From table: syndrome 111 = error at position 5... [Let me recalculate: r=0001101 means positions 0,2,3,6 have... actually 0001101 reading LSB first: $r_0=1, r_1=0, r_2=1, r_3=1, r_4=0, r_5=0, r_6=0$] For exam: Always compute S(x)=r(x) mod g(x), then look up table. If S=110: error at position 4. Flip bit 4 to correct. ✓

Meggitt Decoder — Algorithm 5-MARK

1. Compute syndrome S(x) = r(x) mod g(x) using LFSR
2. Check if S(x) matches any stored error syndrome (for single error at position n-1)
3. If match: flip bit at that position, update S(x), shift corrected bit out
4. Else: cyclically shift r: r→x·r mod(x^n+1), update S: S→x·S mod g(x)
5. Repeat up to n times

```
Key property: cyclic shift of r ⟹ cyclic shift of S (automatically!)
```

5.9 Comparison Tables QUICK REFERENCE

Linear Block Code vs Cyclic Code

FEATURE	LINEAR BLOCK CODE	CYCLIC CODE
Structure	Linear subspace	Linear + cyclic shift property
Math basis	Matrix algebra (G, H)	Polynomial algebra g(x), h(x)
Encoding	c = m·G (matrix multiply)	$c(x)=x^{n-k}m(x)+r(x)$
Error detection	$S = H \cdot r^T$ (matrix)	S(x)=r(x) mod g(x)
Hardware	Complex combinational	Simple LFSR (shift reg)
Burst errors	Limited	Excellent detection
Examples	Basic Hamming, Golay	CRC, BCH, Reed-Solomon

Systematic vs Non-Systematic

FEATURE	SYSTEMATIC	NON-SYSTEMATIC
Message in codeword	✅ Directly visible	❌ Hidden
Formula	$c(x)=x^{n-k}m(x)+r(x)$	$c(x)=m(x)\cdot g(x)$
Decoding	Easy (extract last k bits)	Complex (full decoding)
Practical use	Most common	Special cases

GF(2) vs GF(2^n)

FEATURE	GF(2)	GF(2^n)
Elements	{0,1} — 2 elements	2^n elements
Addition	XOR: 1+1=0	Coefficient XOR
Multiplication	AND	Poly multiply mod p(x)
Construction	Base field	GF(2)[x]/p(x), p irreducible
Primitive elem	1 (trivial)	α with order 2^n-1
Used for	Basic binary codes	BCH, Reed-Solomon

HINGLISH TRICK

Systematic Cyclic Encoding: "Shift, Divide, Add Remainder" ($x^{n-k} \cdot m(x)$ ko $g(x)$ se divide karo, jo remainder $r(x)$ bache use jodd do $\rightarrow c(x) = x^{n-k}m(x) + r(x)$!). **Fundamental Identity: $g(x) \cdot h(x) = x^n + 1$ | Syndrome: $S(x) = r(x) \bmod g(x)$.**

🗺️ Unit 5 Overview Map

Quick Revise

UNIT 5: Cyclic Codes & Polynomial Code Algebra

1. Algebraic Properties

- **Cyclic Shift:** $c^{(1)}(x) = x \cdot c(x) \bmod (x^n+1) \in C$
- **Generator $g(x)$:** Degree $n-k$, divides x^n+1
- **Parity Check $h(x)$:** Degree k, $g(x) \cdot h(x) = x^n+1$
- **Galois Field:** Arithmetic modulo 2 over GF(2) / GF(2^m)

2. Systematic Encoding

- **Step 1:** Shift message poly by $n-k$ positions $\rightarrow x^{n-k}m(x)$
- **Step 2:** Divide by $g(x)$ to get parity remainder $r(x)$
- **Step 3:** $c(x) = x^{n-k}m(x) + r(x)$ (Message intact at high end)

3. LFSR Circuits & Syndrome

- **Encoder LFSR:** $n-k$ flip-flop shift register with feedback taps g_i
- **Syndrome Poly:** $S(x) = r(x) \bmod g(x) = e(x) \bmod g(x)$
- **Error Vector:** $S(x) \ne 0 \Rightarrow$ matches error syndrome table

5.10 RTU Exam Q&A — Complete Bank

EXAM READY

All questions based on RTU 5th Sem CSE ITC exam pattern. Answers ready for 2, 5, and 10 marks.

2-Mark Questions

Q1. Define cyclic code and state its fundamental property.

A **cyclic code** is a linear block code where every cyclic shift of a codeword is also a valid codeword. If $(c_0,c_1,...,c_{n-1})$ is a codeword, then $(c_{n-1},c_0,...,c_{n-2})$ is also a codeword. In polynomial form: if c(x) is a codeword, then $x \cdot c(x) \bmod (x^n+1)$ is also a codeword.

Q2. Define Galois Field. When does GF(q) exist?

A **Galois Field GF(q)** is a finite set of q elements with two operations (addition and multiplication) satisfying all field axioms (closure, commutativity, associativity, distributivity, identity, inverse). GF(q) exists if and only if $q = p^n$ where p is prime and $n \geq 1$. Examples: GF(2)={0,1}, GF(4), GF(8).

Q3. What is a generator polynomial g(x)?

The **generator polynomial g(x)** of an (n,k) cyclic code is the unique monic polynomial of degree (n-k) that divides (x^n+1), and every codeword c(x) is a multiple of g(x). Key properties: deg[g(x)]=n-k, $g(x) \mid (x^n+1)$, code = {m(x)·g(x) : deg m < k}.

Q4. What is syndrome in cyclic code? What does S(x)=0 indicate?

Syndrome S(x) = r(x) mod g(x), where r(x) is the received polynomial. S(x) = 0 indicates r(x) is a valid codeword (no error detected). $S(x) \neq 0$ indicates an error. Importantly, S(x) = e(x) mod g(x) depends only on the error polynomial e(x), not on the transmitted codeword c(x).

Q5. Define parity check polynomial h(x).

$h(x) = (x^n+1)/g(x)$. Properties: deg[h(x)]=k, $g(x) \cdot h(x) = x^n+1$. A polynomial v(x) is a valid codeword iff $v(x) \cdot h(x) \equiv 0 \bmod (x^n+1)$. h(x) is used to construct the parity check matrix H.

Q6. What is an irreducible polynomial over GF(2)?

A polynomial p(x) over GF(2) is irreducible if it has no factors of lower degree over GF(2), i.e., no roots in GF(2). Test for degree 2 or 3: $p(0) \neq 0$ AND $p(1) \neq 0$. Example: x^2+x+1 is irreducible ($p(0)=1 \neq 0$, $p(1)=1 \neq 0$). Example: $x^2+1=(x+1)^2$ is reducible ($p(1)=0$).

5-Mark Questions

Q7. State and explain the basic properties of Galois Field GF(2^n). (5 marks)

Properties of GF(2^n):

1. **Order:** GF(2^n) has exactly 2^n elements (1 zero + 2^n-1 non-zero).
2. **Characteristic 2:** a+a=0 for every element a (1+1=0 in GF(2)).
3. **Primitive Element:** Exists α of order (2^n-1) s.t. {α°,α¹,...,α^(2^n-2)} = all non-zero elements.
4. **Fermat's Theorem:** a^(2^n-1) = 1 for all non-zero a.
5. **Additive Group:** (GF(2^n),+) is abelian group of order 2^n.
6. **Multiplicative Group:** (GF(2^n)\{0},×) is cyclic group of order 2^n-1.
7. **Subfield:** GF(2^m) ⊂ GF(2^n) iff m|n. GF(2) ⊂ GF(2^n) always.
8. **Construction:** GF(2^n)=GF(2)[x]/p(x), p(x) irreducible of degree n.

Example GF(8), p(x)=x^3+x+1: $\alpha^3=\alpha+1$, elements={0,1,α,α^2,α+1,α^2+α,α^2+α+1,α^2+1}, period=7.

Q8. Explain polynomial operations over GF(2) with examples. (5 marks)

Polynomial Operations over GF(2):

All coefficients from {0,1}, arithmetic mod 2. Subtraction = Addition = XOR.

1. Addition (XOR coefficients):
$(x^4+x^3+x+1) + (x^3+x^2+1) = x^4+(1+1)x^3+x^2+x+(1+1) = x^4+x^2+x$

2. Multiplication (distribute, then XOR like powers):
$(x+1)(x^2+x+1) = x^3+x^2+x+x^2+x+1 = x^3+(1+1)x^2+(1+1)x+1 = x^3+1$

3. Division (long division, subtract=XOR):
$(x^6+x^4+x^2+x+1) \div (x^3+x+1)$:

- $x^6/x^3=x^3$: sub $x^6+x^4+x^3$, rem=x^3+x^2+x+1
- $x^3/x^3=1$: sub x^3+x+1, rem=x^2
- deg(x^2) < 3: STOP. q(x)=x^3+1, r(x)=x^2

Key rules: No negative numbers. No carrying. 1+1=0. All operations mod 2.

Q9. Construct GF(2^3) using $p(x)=x^3+x+1$. List all elements in power, polynomial, and vector form. (5 marks)

Step 1: Verify p(x) irreducible: $p(0)=1\neq0$, $p(1)=1\neq0$ ✓

Step 2: Let α be root: $\alpha^3+\alpha+1=0 \Rightarrow \alpha^3=\alpha+1$

POWER	POLYNOMIAL	VECTOR ($A_2A_1A_0$)
0	0	000
α°	1	001
α^1	α	010
α^2	α^2	100
α^3	α+1	011
α^4	$\alpha^2+\alpha$	110
α^5	$\alpha^2+\alpha+1$	111
α^6	α^2+1	101

Period = 7 = 2^3-1. $\alpha^7=1$ (cycle repeats). α is primitive element of GF(8). ✓

Q10. Find g(x), h(x), and generator matrix G for the (7,4) cyclic code. (5 marks)

Given: n=7, k=4, n-k=3

Step 1: Factor x^7+1: $x^7+1=(x+1)(x^3+x+1)(x^3+x^2+1)$

Step 2: g(x) = degree-3 factor: $g(x) = x^3+x+1$

Step 3: h(x) = $(x^7+1)/g(x) = (x+1)(x^3+x^2+1) = x^4+x^2+x+1$
Verify: deg[h(x)]=4=k ✓, $g \cdot h = x^7+1$ ✓

Step 4: Generator Matrix G (k×n = 4×7):
Rows = cyclic shifts of g(x) vector (0001011 for x^3+x+1 reading x^6 to x^0):
g(x): degree 3 ⇒ 0001011
G = | 1 0 1 1 0 0 0 | (xg shift)
| 0 1 0 1 1 0 0 | (x^2g shift)
| 0 0 1 0 1 1 0 | (x^3g shift... use systematic form)
| 0 0 0 1 0 1 1 | (g)
(Systematic form after row operations)

Q11. Explain systematic encoding with example. Encode m=1001 using (7,4) code, $g(x)=x^3+x^2+1$. (5 marks)

Systematic Encoding Steps:

1. $T(x) = x^{n-k} \cdot m(x)$ [shift message left by n-k positions]
2. $r(x) = T(x) \bmod g(x)$ [remainder = parity bits]
3. $c(x) = T(x)+r(x)$ [add back remainder to get divisible codeword]

Example: m=1001, $m(x)=x^3+1$, $g(x)=x^3+x^2+1$, n=7,k=4:

$T(x) = x^3(x^3+1) = x^6+x^3$

Divide x^6+x^3 by x^3+x^2+1:

$x^6/x^3=x^3$: sub $x^6+x^5+x^3$, rem=x^5

$x^5/x^3=x^2$: sub $x^5+x^4+x^2$, rem=x^4+x^2

$x^4/x^3=x$: sub x^4+x^3+x, rem=x^3+x^2+x

$x^3/x^3=1$: sub x^3+x^2+1, rem=$x+1$

$r(x)=x+1 \Rightarrow$ parity bits = 011

Codeword c = 011 1001 (parity=011, message=1001)

Verify: $c(x) \bmod g(x) = 0$ ✓

10-Mark Questions

Q12. Describe the encoder and decoder for cyclic codes with block diagrams. Explain syndrome-based error detection and Meggitt decoding. (10 marks)

PART A — CYCLIC ENCODER:

Uses LFSR (Linear Feedback Shift Register) with (n-k) stages.

Structure: (n-k) D flip-flops in series. Feedback XOR gates at positions where g(x) coefficient = 1. For $g(x)=x^3+x+1$: taps at x^3 (main feedback always present) and x^1 ($g_1=1$).

Phase 1 (k clock cycles):
- Message bits $m_{k-1},\ldots,m_0$ enter serially
- Switch connects to output channel → message flows to output
- LFSR computes remainder $x^{n-k}\cdot m(x) \bmod g(x)$ simultaneously

Phase 2 (n-k clock cycles):
- Input grounded (0s enter LFSR)
- Switch connects register to output
- Register contents shift out as parity bits $r_0,\ldots,r_{n-k-1}$

Output: [message k bits][parity n-k bits] = systematic (n,k) codeword

PART B — CYCLIC DECODER:

Same LFSR structure computes syndrome $S(x) = r(x) \bmod g(x)$.

Operation:
1. Load received r into n-bit buffer AND feed into syndrome LFSR simultaneously
2. After n clocks: LFSR = $S(x) = r(x) \bmod g(x)$
3. Decision: $S(x)=0$ → no error, output last k bits. $S(x)\neq 0$ → error, apply Meggitt.

PART C — SYNDROME & MEGGITT DECODER:

$r(x) = c(x)+e(x)$ → $S(x)=e(x) \bmod g(x)$ [syndrome = function of error only!]

Pre-stored table: for each correctable error pattern e_i, store $S_i=e_i \bmod g(x)$.

Meggitt Algorithm:
1. Compute S(x) from r(x)

2. If S(x) matches table entry for error at position n-1: flip that bit, update syndrome
3. Else: cyclically shift r → x·r mod(x^n+1), update S → x·S mod g(x)
4. Repeat until all errors corrected or n shifts done

Key: Cyclic shift of r → syndrome also shifts cyclically. This allows sequential scanning without recomputation.

Example: Received r, S(x)=110 → table says error at position 4 → flip bit 4 → corrected codeword. ✓

Q13. Explain Galois Field GF(2^n), construction using irreducible polynomial, and polynomial operations over GF(2). Give complete example. (10 marks)

PART A — GALOIS FIELD GF(2^n):

A finite field with 2^n elements. GF(q) exists iff $q=p^n$ (p prime). For digital comms: GF(2^n).

Constructed as GF(2)[x]/p(x) where p(x) is irreducible of degree n over GF(2).

Properties: 2^n elements; characteristic 2 (a+a=0); multiplicative group cyclic of order 2^n-1; primitive element α with α^(2^n-1)=1; GF(2^m) ⊂ GF(2^n) iff m|n.

PART B — GF(8) COMPLETE EXAMPLE, $p(x)=x^3+x+1$:

Verify: p(0)=1≠0, p(1)=1≠0 → irreducible ✓
Root α: $\alpha^3+\alpha+1=0 \rightarrow \alpha^3=\alpha+1$

Elements: 0, 1, α, α^2, α+1, $\alpha^2+\alpha$, $\alpha^2+\alpha+1$, α^2+1
Vectors: 000, 001, 010, 100, 011, 110, 111, 101
Period = 7 = 2^3-1. $\alpha^7=\alpha^0=1$ ✓

GF(8) Arithmetic:
$\alpha^4+\alpha^3 = (\alpha^2+\alpha)+(\alpha+1) = \alpha^2+(1+1)\alpha+1 = \alpha^2+1 = \alpha^6$ ✓
$\alpha^4\cdot\alpha^3 = \alpha^7 = \alpha^0 = 1$ ✓

PART C — POLYNOMIAL OPERATIONS OVER GF(2):

Addition (XOR coefficients):
$(x^2+x+1)+(x^2+1) = (1+1)x^2+x+(1+1) = x$ (middle terms cancel)

Multiplication (normal expand, XOR like terms):
$(x+1)^2 = x^2+2x+1 = x^2+0+1 = x^2+1$ [2x=0 in GF(2)]

Division:
$(x^7+1)/(x^3+x+1) = (x^4+x^2+x+1)$ [exact, since $x^7+1=(x^3+x+1)\cdot h(x)$]

Relevance to Cyclic Codes:
- g(x) built from factors of x^n+1 over GF(2)
- BCH codes use minimal polynomials of α^i over GF(2^n)

- Primitive element α locates error positions in decoding
- All encoding/decoding arithmetic done as polynomial operations over GF(2)

Q14. What is a cyclic code? Explain algebraic structure, generator polynomial, and systematic encoding. Encode m=1100, m=1010, m=0011 using (7,4) code with $g(x)=x^3+x+1$. (10 marks)

CYCLIC CODE & ALGEBRAIC STRUCTURE:

A cyclic code C(n,k) is a linear code where every cyclic shift of a codeword is valid. Algebraically, C is an *ideal* in ring $R=GF(2)[x]/(x^n+1)$, generated by g(x).

Ideal property: if $c(x)\in C$ and $r(x)\in R$, then $r(x)\cdot c(x) \bmod (x^n+1) \in C$.
The cyclic shift $x\cdot c(x) \bmod (x^n+1)$ is automatically in C. This is why cyclic property holds!

Generator Polynomial:
g(x) = unique monic poly of degree n-k dividing x^n+1. All 2^k codewords are multiples of g(x).

Systematic Encoding: $c(x)=x^{n-k}m(x) + [x^{n-k}m(x) \bmod g(x)]$

Three Encoding Examples (n=7,k=4,$g(x)=x^3+x+1$):

1. m=1100 → $m(x)=x^3+x^2$:
$T(x)=x^3(x^3+x^2)=x^6+x^5$
Divide: $x^6/x^3=x^3 \rightarrow rem=x^5+x^4$; $x^5/x^3=x^2 \rightarrow rem=x^4+x^3+x^2$; $x^4/x^3=x \rightarrow rem=x^3+x$; $x^3/x^3=1 \rightarrow rem=1$
$r(x)=1 \rightarrow$ parity=001. **Codeword: 001 1100**

2. m=1010 → $m(x)=x^3+x$:
$T(x)=x^3(x^3+x)=x^6+x^4$
Divide: $x^6/x^3=x^3 \rightarrow$ sub $x^6+x^4+x^3$,rem=x^3; $x^3/x^3=1 \rightarrow$ sub x^3+x+1,rem=x+1
$r(x)=x+1 \rightarrow$ parity=011. **Codeword: 011 1010**

3. m=0011 → $m(x)=x+1$:
$T(x)=x^3(x+1)=x^4+x^3$
Divide: $x^4/x^3=x \rightarrow$ sub x^4+x^2+x,rem=x^3+x^2+x; $x^3/x^3=1 \rightarrow$ sub x^3+x+1,rem=x^2+1
$r(x)=x^2+1 \rightarrow$ parity=101. **Codeword: 101 0011**

All verified: $c(x) \bmod g(x) = 0$ ✓

Q15. Explain in detail: Galois Field concept, irreducible polynomial, primitive element, and how these relate to cyclic code construction. (10 marks)

1. GALOIS FIELD CONCEPT:

A Galois Field GF(q) is a finite algebraic structure with $q=p^n$ elements where both addition and multiplication are well-defined and satisfy all field axioms. For binary coding: GF(2)={0,1} is the base field. GF(2^n) extends it for more powerful codes.

2. IRREDUCIBLE POLYNOMIAL:

A polynomial p(x) over GF(2) is irreducible if it has no polynomial factors of lower degree over GF(2). Equivalently, it has no roots in GF(2).

Test: for degree 2 or 3, check $p(0)\neq 0$ AND $p(1)\neq 0$.

Example: x^2+x+1: $p(0)=1$, $p(1)=1$ → no roots → irreducible ✓

Example: $x^3+1=(x+1)(x^2+x+1)$: reducible ❌

3. PRIMITIVE ELEMENT:

An element α of GF(2^n) is primitive if its multiplicative order = 2^n-1. This means {$\alpha^0,\alpha^1,\ldots,\alpha^{(2^n-2)}$} generates ALL 2^n-1 non-zero elements.

In GF(8) with $p(x)=x^3+x+1$: $\alpha^7=1$, α has order $7=2^3-1$ → α is primitive ✓

Fermat: $\alpha^{(2^n-1)}=1$ for all non-zero elements.

4. RELATIONSHIP TO CYCLIC CODES:

- Basic cyclic codes (length n) operate over GF(2) — polynomials with binary coefficients
- x^n+1 must be factored over GF(2) to find g(x)
- BCH Codes: defined using GF(2^n); g(x)=LCM of minimal polynomials of $\alpha,\alpha^2,\ldots,\alpha^{(2t)}$ for t-error correction
- Reed-Solomon: symbols ARE elements of GF(2^n); extremely powerful burst error correction
- Primitive element α locates error positions in BCH/RS decoding (error locator polynomial)

Summary:

GF(2) → binary arithmetic for basic cyclic codes

Irreducible p(x) → builds GF(2^n), defines α relations

Primitive α → generates all field elements, enables BCH code construction

$g(x)\,|\,x^n+1$ → all codewords are multiples of g(x), enabling LFSR encoding

📌 UNIT 5 — MASTER FORMULA CHEAT SHEET

GF(2ⁿ) has 2ⁿ elements	a^(2ⁿ-1) = 1 (Fermat)
deg[g(x)] = n - k	deg[h(x)] = k
g(x) · h(x) = xⁿ + 1	h(x) = (xⁿ+1) / g(x)
Non-sys: c(x)=m(x)·g(x)	Sys: c(x)=x^{n−k}m(x)+r(x)
S(x) = r(x) mod g(x)	S(x)=0 → valid codeword
xⁿ+1=(x+1)(x³+x+1)(x³+x²+1)	g(x) | (xⁿ+1) always!
Cyclic shift: x·c(x) mod(xⁿ+1)	In GF(2): 1+1=0, -a=a

ZYREX // ITC_VAULT — UNIT 5 · Cyclic Codes

RTU B.Tech CSE 5th Semester · 5CS3-01 Information Theory & Coding · Unit 5 of 6

📌 Key formulas: g(x)|(xⁿ+1) · h(x)=(xⁿ+1)/g(x) · S(x)=r(x) mod g(x) · g·h=xⁿ+1

⚡ Unit 6 — Convolutional Codes can be appended directly below this section.

🔗 Unit 6 — Convolutional Codes

Convolutional Encoders of Different Rates · Code Tree · Trellis & State Diagram · Maximum Likelihood Decoding (Viterbi Algorithm) · Free Distance | Hours: 06

🔥 HIGH WEIGHT — Viterbi numerical almost guaranteed ⭐ 5 & 10 Mark Questions

1. Introduction to Convolutional Codes High ⭐⭐⭐

ENGLISH

A **Convolutional Code** is an error-control code in which the encoder output at any time instant depends not only on the current input block of *k* bits, but also on a fixed number of **previous input blocks** (stored in shift-register memory). Unlike block codes (where each codeword is encoded independently), convolutional codes process data as a **continuous stream**, convolving the input sequence with the encoder's impulse response (generator sequences) — hence the name.

HINGLISH

Linear Block Code (Unit 4) aur Cyclic Code (Unit 5) mein har codeword **independent** hota hai — fixed k input bits se fixed n output bits bante hain, koi memory nahi. Convolutional code mein encoder ek **shift register** use karta hai jisme purane input bits "yaad" rehte hain — to current output sirf current input pe nahi, **pichle kuch bits pe bhi** depend karta hai. Isiliye data ko continuous stream ki tarah process karta hai, block-by-block nahi. Ye satellite communication, deep-space probes (Voyager), aur 4G/WiFi mein bahut use hota hai.

ASPECT	BLOCK CODES (LINEAR/CYCLIC)	CONVOLUTIONAL CODES
Memory	None — each codeword independent	Has memory — depends on past *m* input blocks
Encoding	Algebraic (matrix/polynomial multiplication)	Convolution operation (shift register + modulo-2 adders)
Structure	Fixed-length blocks of n bits from k bits	Continuous stream, semi-infinite sequences
Decoding	Syndrome decoding	Viterbi / Sequential decoding (maximum likelihood)

Best for	Burst error correction (with interleaving)	Random/Gaussian channel errors, deep-space links

2. Convolutional Encoder — Structure & Parameters

High ★★★★

ENGLISH

A convolutional encoder is built from a **k-bit-wide shift register** with *m* memory stages, and *n* modulo-2 (XOR) adders tapped at specific register positions (defined by **generator sequences/polynomials**) to produce *n* output bits for every *k* input bits. It is denoted as an **(n, k, m)** code.

```
Code Rate: R = k/n (k input bits → n output bits) Constraint Length: K = m + 1 (total no. of input bits influencing one output set; m = memory order / no. of shift-register stages)
```

HINGLISH

k = ek time mein kitne input bits jaate hain, **n** = unse kitne output bits banate hain, **m** = shift register mein kitne "purane" input bits store hote hain (memory). **Constraint length K** batata hai ki **total kitne bits** (current + memory) ek output ko affect karte hain — jitna zyada K, utna behtar error-correction lekin utna zyada complex decoder.

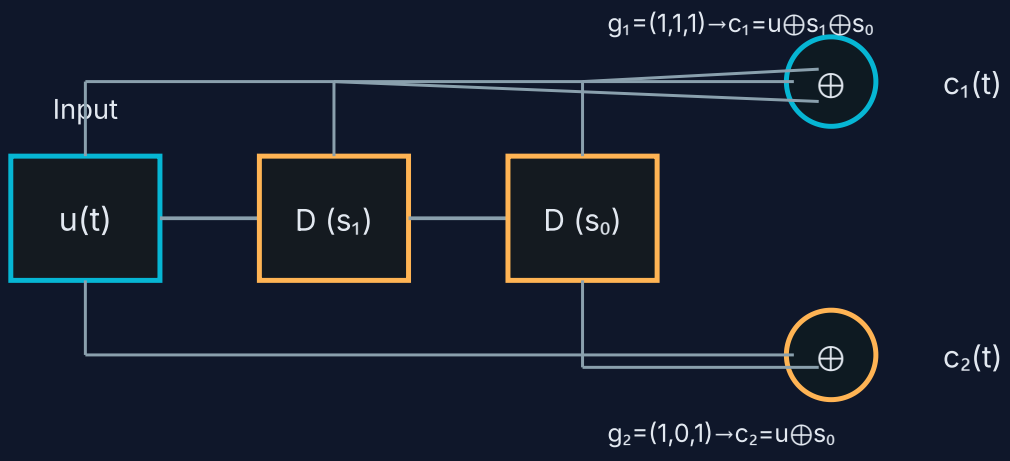


Fig 6.1 — Rate-½ Convolutional Encoder, K=3 (m=2 memory stages), generator polynomials $g_1=(111)$, $g_2=(101)$ — the classic "(7,5) octal" NASA encoder used throughout this unit's worked examples

3. Convolutional Encoders of Different Rates

High ★★★

Rate-1/2 Encoder (k=1, n=2)

Most common in practice. One input bit produces two output bits per clock cycle (as shown in Fig 6.1 above). Used in deep-space communication, GSM, dial-up modems.

Rate-1/3 Encoder (k=1, n=3)

One input bit produces **three** output bits — lower rate, more redundancy, stronger error correction but lower throughput. Three generator polynomials g_1, g_2, g_3 tap the shift register at different positions; used where channel is extremely noisy (e.g. very weak satellite signal links).

General Rate-k/n Encoder

k input bits enter simultaneously per clock cycle, producing n output bits, with k separate shift registers feeding n modulo-2 adders. Higher k/n ratio (e.g. 2/3, 3/4) gives **higher throughput but weaker error protection** — there is always a fundamental trade-off between rate and error-correcting strength.

HINGLISH

Rate **k/n** jitna chhota (jaise 1/3), utni zyada redundancy add hoti hai → behtar error correction, lekin bandwidth zyada use hoti hai (slow effective throughput). Rate jitna bada (jaise 3/4), utni kam redundancy → fast throughput lekin kamzor error protection. Engineer ko apne channel ki noise condition dekh kar ye trade-off choose karna padta hai.

Rate Comparison Table

Medium ★★

RATE (K/N)	REDUNDANCY	ERROR-CORRECTING STRENGTH	THROUGHPUT	TYPICAL USE
1/3	Highest	Strongest	Lowest	Deep-space probes (Voyager)
1/2	High	Strong	Moderate	GSM, satellite, dial-up modems

2/3, 3/4	Low (via puncturing)	Weaker	High	WiFi, 4G/LTE (punctured codes)

4. Code Tree Representation

Medium ★★★

ENGLISH

A **code tree** is a graphical representation of a convolutional encoder where each level (depth) corresponds to one input time-step, each node represents the encoder's state, and each branch represents the output produced for a given input bit (0 = upper branch, 1 = lower branch, by convention). The tree grows without bound as more input bits arrive, doubling the number of branches at every level.

HINGLISH

Code tree sabse "basic"/literal representation hai — har input bit ke liye tree do branches mein split hota hai (0 ya 1), aur har branch pe output bits likhe jaate hain. Problem ye hai ki ye tree **exponentially badhta** hai (har level pe nodes double), isiliye practically lamba sequence ke liye bahut bada ho jaata hai — yahi limitation trellis diagram solve karta hai (states repeat hote hain, tree "fold" ho jaata hai).

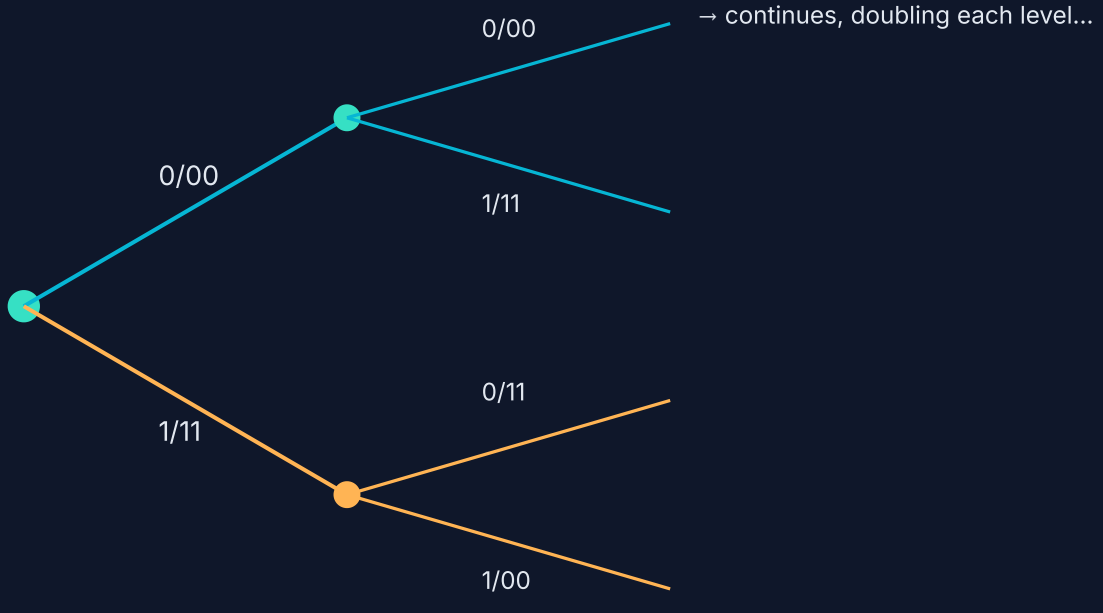


Fig 6.2 — Code Tree for the rate-½ encoder of Fig 6.1: branches double at every level (input bit 0 = upper path, 1 = lower path), each labeled "input/output"

5. State Diagram

High ⭐⭐⭐⭐

ENGLISH

The **state diagram** represents the convolutional encoder as a finite-state machine: each **node** is one of the 2^m possible shift-register contents (states), and each **directed edge** shows the transition to a new state for a given input bit, labeled "input/output". Unlike the code tree, the state diagram has a **fixed, finite number of nodes** — it compresses the infinitely-growing tree into a compact, repeating structure.

HINGLISH

State diagram mein sirf **2^m states** hote hain (jaise m=2 ke liye 4 states: 00,01,10,11) — chahe sequence kitni bhi lambi ho, states repeat hote rehte hain. Har state se 2 outgoing edges hote hain (input 0 ke liye, input 1 ke liye), edge pe "input/output" label hota hai. Ye code tree se zyada practical/compact representation hai.

Next-State & Output Table (for Fig 6.1 encoder)

High — memorize this pattern for numericals

CURRENT STATE (S_1S_0)	INPUT U	OUTPUT C_1C_2	NEXT STATE
00	0	00	00
00	1	11	10
01	0	11	00
01	1	00	10
10	0	10	01
10	1	01	11
11	0	01	01
11	1	10	11

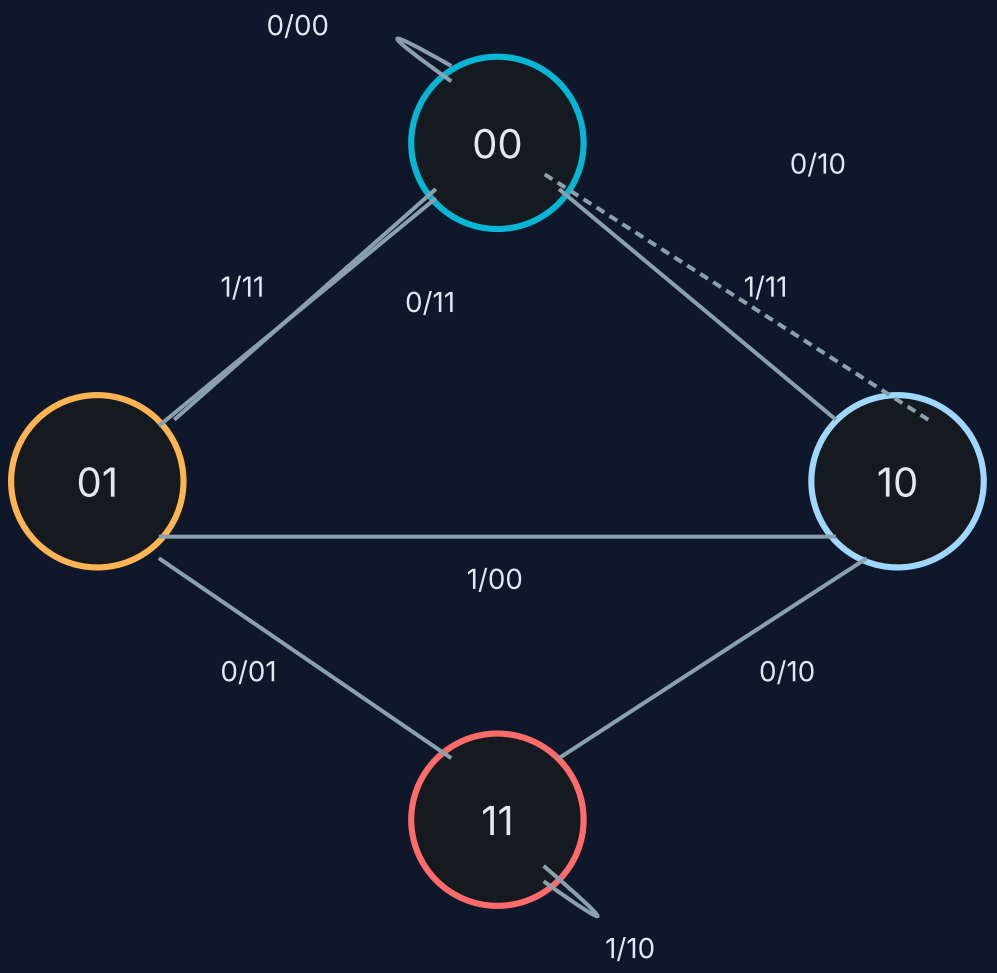


Fig 6.3 — State Diagram: 4 states (00,01,10,11), each with 2 outgoing edges labeled "input/output", per the encoder table above

6. Trellis Diagram

High ⭐ ⭐ ⭐ ⭐ ⭐ — essential for Viterbi numericals

ENGLISH

A **trellis diagram** is the state diagram **unrolled over time** — it shows all possible states at every time instant (as columns), with branches connecting states from one time-step to the next. It combines the finite-state compactness of the state diagram with the explicit time-progression of the code tree, making it the ideal structure for decoding algorithms like Viterbi.

HINGLISH

Trellis = state diagram ko time ke saath "unfold" kar diya — har time step pe sab 4 states ek column mein dikhte hain, aur lines unhe agle time-step ke states se jodti hain. Ye Viterbi algorithm ke liye perfect structure hai kyunki ismein **"merging paths"** dikhte hain — jab do different input sequences same state pe converge ho jaayen, to weak path ko discard kar sakte hain (yahi Viterbi ka core trick hai).

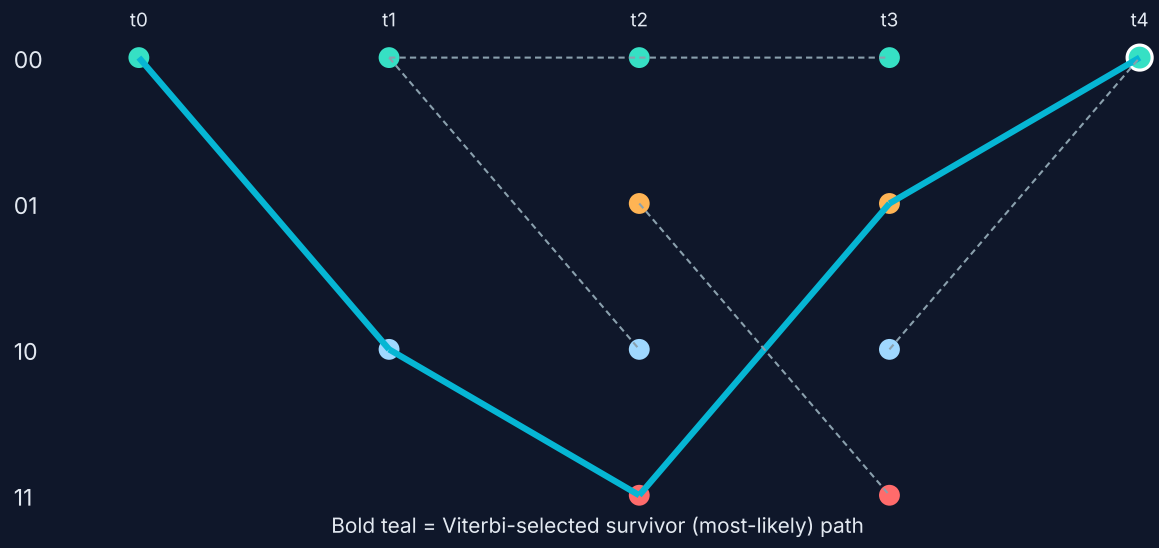


Fig 6.4 — Trellis Diagram: states repeat at every time-step; the bold teal path is the maximum-likelihood path traced back by the Viterbi algorithm (worked example below)

Code Tree vs State Diagram vs Trellis Diagram

High ⭐ ⭐ ⭐ ⭐ — frequently asked comparison

ASPECT	CODE TREE	STATE DIAGRAM	TRELLIS DIAGRAM
Size	Grows exponentially (2^L branches for L inputs)	Fixed — only 2^m states, ever	Fixed states × time steps (linear growth)
Time shown?	Yes — implicitly via depth	No — time-independent	Yes — explicitly, states repeated per column
Used for	Conceptual understanding only	Encoder design/analysis	**Practical decoding (Viterbi algorithm)**
Path merging visible?	No (every path unique, never merges)	N/A	**Yes** — key to Viterbi's efficiency

7. Maximum Likelihood Decoding — The Viterbi Algorithm

High ⭐ ⭐ ⭐ ⭐ ⭐ (Most Important Topic — full numerical expected)

ENGLISH

Maximum Likelihood (ML) Decoding selects, among all possible transmitted codeword sequences, the one that is "closest" (minimum Hamming distance, for a binary symmetric channel) to the received sequence. The **Viterbi Algorithm** is an efficient way to perform ML decoding on a trellis: instead of comparing every possible path (which grows exponentially), it computes a running **path metric** at each state and, whenever two paths merge into the same state, discards the path with the worse metric — keeping only the single best ("survivor") path into each state at every time step.

HINGLISH

Maximum likelihood decoding ka matlab — sab possible transmitted sequences mein se wo sequence choose karo jo received sequence ke sabse "close" (kam se kam Hamming distance) ho. Lekin sab sequences ko check karna exponentially expensive hai. **Viterbi algorithm** ka trick — trellis pe chalte hue, jab bhi 2 paths same state pe milte hain ("merge"), to jo path zyada error/distance accumulate kar chuka hai use turant discard kar do, sirf best ("survivor") path rakho. Isse computation drastically kam ho jaata hai, aur phir bhi guaranteed optimal (ML) answer milta hai.

Viterbi Algorithm — Step-by-Step

High — write this exact sequence in exam

1. **Initialize:** Set path metric of the starting state (usually all-zero state 00) = 0; all other states = ∞ (unreachable initially).
2. **Branch metric computation:** At each time step, for every valid transition, compute the **branch metric** = Hamming distance between the received bits and the expected output bits for that branch.
3. **Path metric update (Add-Compare-Select):** For every state at the new time step, add the branch metric to the path metric of the originating state (for each incoming branch). If multiple branches arrive at the same state, **compare** them and **select** (keep) only the path with the **minimum total metric** — this is the survivor path for that state; discard the other(s).
4. **Repeat** steps 2-3 for every time step until the end of the received sequence.
5. **Trace back:** At the final time step, pick the state with the minimum path metric (usually the all-zero state if the encoder was flushed/terminated). Trace back through the stored survivor branches to recover the most likely input sequence.

Full Worked Numerical — Viterbi Decoding

High ⭐⭐⭐⭐⭐ — exact RTU 10-mark pattern

EXAMPLE

Using the rate-½, K=3 encoder of Fig 6.1 (g_1=111, g_2=101), the message **m = 1,1,0** (plus 2 flush zero-bits to return encoder to state 00) is transmitted. Due to channel noise, **one bit is received in error**. Decode using the Viterbi algorithm.

Step 1 — Encode the original message (using the next-state table above), starting at state 00:

T	STATE	INPUT U	OUTPUT C_1C_2	NEXT STATE
1	00	1	11	10
2	10	1	01	11
3	11	0	01	01
4	01	0	11	00
5	00	0	00	00

Transmitted (encoded) sequence: **11 01 01 11 00**

Received sequence (1-bit error introduced in pair 3): **11 01 11 11 00** (should've been "01", error flips it to "11")

Step 2 — Build path metrics using Add-Compare-Select (branch metric = Hamming distance to received pair):

TIME	STATE 00	STATE 01	STATE 10	STATE 11
t0	0	∞	∞	∞
t1	2	∞	0	∞
t2	3	2	3	0
t3	2 ✓(via 01)	1 ✓(via 11)	3 ✓(via 00)	1 ✓(via 11)
t4	1 ✓(via 01)	2 ✓(via 11)	2 ✓(via 00)	2 ✓(via 11)
t5	**1 ✓(via 00)**	3	2	3

Final minimum metric = **1** at state 00 (t5) — this matches exactly the single bit error we introduced, confirming the decoder correctly "sees" only 1 unit of distance from a valid

codeword.

Step 3 — Trace back the survivor path from t5(00) → t4(00,u=0) → t3(01,u=0) → t2(11,u=1) → t1(10,u=1) → t0(00):

```
Decoded path: 00 → 10 → 11 → 01 → 00 → 00 Decoded input bits (u): 1, 1, 0 (flush bits 0,0 dropped)
```

Result: Decoded message = 1,1,0 — exactly matches the original transmitted message, despite the single-bit channel error. ✅ This demonstrates the error-correcting power of convolutional codes with Viterbi decoding.

8. Free Distance of a Convolutional Code

High ⭐⭐⭐⭐

ENGLISH

The **Free Distance (d_{free})** of a convolutional code is the **minimum Hamming distance** between any two distinct (complete) code sequences generated by the encoder — equivalently, it is the minimum-weight nonzero path in the trellis/state diagram that diverges from the all-zero state and eventually remerges with it. It plays the same role for convolutional codes that **minimum distance d_{min}** plays for block codes — determining the code's error-correcting capability.

$$\text{Error-correcting capability: } t = \lfloor (d_{free} - 1) / 2 \rfloor$$

HINGLISH

Free distance batata hai ki agar encoder all-zero state se "bhatak" jaaye (kisi non-zero input ki wajah se) aur fir se all-zero state pe wapas aaye, to is poore detour mein **minimum kitna "weight" (1s ki count)** accumulate hoga. Yahi number decide karta hai ki code kitne errors tak correct kar sakta hai — jaise block codes mein d_{min} karta hai.

COMPUTING D_{FREE} FOR OUR FIG 6.1 ENCODER

Find the minimum-weight path that leaves state 00 and returns to state 00 (using the state diagram, Fig 6.3):

Path: 00 →(u=1, out=11, weight=2)→ 10 →(u=0, out=10, weight=1)→ 01 →(u=0, out=11, weight=2)→ 00

Total weight = 2+1+2 = **5**

No shorter-weight path exists that diverges and remerges with state 00 (can be verified by checking all alternative paths — they all accumulate weight ≥5).

∴ $\mathbf{d_{free} = 5}$ for this rate-½, K=3 code.

Error-correcting capability: t = ⌊(5−1)/2⌋ = **2 errors** per decoding window.

Factors Affecting Free Distance

Medium

- Increasing **constraint length K** (more memory/shift-register stages) generally increases d_{free} — better error correction, at the cost of decoder complexity (states grow as 2^m).
- **Choice of generator polynomials** matters greatly — poorly chosen generators can give a low d_{free} even for large K; generators must be chosen (e.g. via computer search) to maximize d_{free}.
- Lower code rate (k/n) typically allows higher d_{free} due to greater redundancy.

📌 UNIT 6 — MASTER FORMULA CHEAT SHEET

Rate R = k/n	Constraint length K = m+1
No. of states = 2^m	$c_1 = u \oplus s_1 \oplus s_0$ (g_1=111)
$c_2 = u \oplus s_0$ (g_2=101)	Branch metric = Hamming dist.
Path metric = Σ branch metrics	Viterbi: Add→Compare→Select
d_free = min weight closed path	t = ⌊(d_free-1)/2⌋

HINGLISH TRICK

Viterbi Algorithm: "ACS Karo, Traceback Pao" (**A**dd Branch Metric $\rightarrow$ **C**ompare Incoming Paths $\rightarrow$ **S**elect Survivor Path [Min Hamming distance]).
Free Distance (d_{free}): All-zero (00) se niklo aur wapas 00 pe aao — jo **sabse kam weight (1s ka count)** wala path hoga, wahi d_{free} hai! Corrects $\lfloor (d_{free}-1)/2 \rfloor$ errors.

🗺️ Unit 6 Overview Map

Quick Revise

UNIT 6: Convolutional Codes & Viterbi Decoding

1. Encoder & Structural Memory

- **Parameters:** (n,k,m) code, rate $R=k/n$, constraint length $K=m+1$
- **Memory:** Shift register of m stages $\Rightarrow 2^m$ possible states
- **Generators:** Mod-2 adders connected via generator vectors g_1, g_2

2. Code Representations

- **Code Tree:** Binary tree branching at each time instant (2^t growth)
- **State Diagram:** Directed graph of 2^m states with input/output edges
- **Trellis Diagram:** Time-indexed state transitions (Repeats after K steps)

3. Viterbi Decoding & Free Distance

- **ML Decoding:** Finds code path with minimum total Hamming metric
- **ACS Rule:** Add branch metric, Compare converging paths, Select survivor
- **Free Distance:** $d_{free} = \text{min weight path leaving and returning to 00}$

🎯 RTU EXAM Q&A BANK — UNIT 6 EXAM GOLD

Q1. Define convolutional code. How does it differ from block codes? ★★★ 5 Marks

Convolutional code: encoder output depends on current + previous m input blocks (has memory), processes data as a continuous stream. Block codes: each codeword independent, no memory. Full comparison table given above.

Q2. Explain the structure of a convolutional encoder. Define rate, constraint length, and memory order. ★★★★ 7 Marks

Encoder = k-bit shift register (m stages) + n mod-2 adders tapped by generator sequences, denoted (n,k,m). Rate R=k/n. Constraint length K=m+1 = total bits influencing one output. Diagram in Fig 6.1.

Q3. Draw and explain a rate-½ convolutional encoder with K=3. Derive its output equations. ★★★★★ 10 Marks

Shift register with 2 memory stages s_1,s_0; generators g_1=(111)→c_1=u⊕s_1⊕s_0, g_2=(101)→c_2=u⊕s_0. Full diagram (Fig 6.1) and derivation given above.

Q4. Explain convolutional encoders of different rates (1/2, 1/3, k/n) and the rate vs error-correction trade-off. ★★★★ 7 Marks

Rate-1/2: 1 input→2 output bits (most common). Rate-1/3: 1 input→3 outputs (more redundancy, stronger correction, lower throughput). General k/n: trade-off table given above — lower rate = more redundancy = stronger correction but less throughput.

Q5. What is a code tree? Construct the code tree for a given rate-½ encoder for 2-3 input bits. ★★★ 5 Marks

Graphical representation where each level=one time step, branches double per level (0=upper,1=lower), each branch labeled "input/output". Grows exponentially — limitation solved by trellis. Example in Fig 6.2.

Q6. Draw the state diagram for a convolutional encoder with m=2 (4 states) and explain its transitions. ★★★★ 7 Marks

4 states (00,01,10,11), each with 2 outgoing edges labeled "input/output" per the next-state table. Full diagram in Fig 6.3 + transition table — memorize this table pattern for any similar encoder.

Q7. What is a trellis diagram? How is it different from the state diagram and code tree? ★★★★ 5-7 Marks — very frequently asked

Trellis = state diagram unrolled over time, showing all states at each time-step as columns with transition branches; unlike code tree it doesn't grow exponentially (states repeat), and unlike plain state diagram it explicitly shows time progression and "path merging" — critical for Viterbi. Full 3-way comparison table given above.

Q8. Explain Maximum Likelihood decoding. Describe the Viterbi Algorithm step by step. ★★★★★ 10 Marks — theory backbone of the unit

ML decoding picks the transmitted sequence closest (min Hamming distance) to received sequence. Viterbi efficiently finds it on the trellis using Add-Compare-Select: compute branch metrics, accumulate path metrics, at merging states keep only the minimum-metric survivor path, discard others. 5-step algorithm listed above.

Q9. Numerical: Given a convolutional encoder and a received sequence with an error, decode using the Viterbi algorithm. Show all path metrics and the traceback. ★★★★★ 10 Marks — guaranteed numerical, practice this exact pattern

Full worked solution above: encode m=1,1,0 (+flush) → transmitted "11 01 01 11 00" → 1-bit error → received "11 01 11 11 00" → build path-metric table via Add-Compare-Select → minimum final metric=1 at state00 → traceback gives decoded message 1,1,0 (error corrected).

Q10. Define free distance of a convolutional code. How is it related to error-correcting capability? ★★★★ 7 Marks

d_{free} = minimum Hamming distance between any two distinct code sequences = minimum-weight path diverging from and remerging to the all-zero state. Error-correcting capability $t=\lfloor(d_{free}-1)/2\rfloor$. Plays same role as d_min for block codes.

Q11. Numerical: Compute the free distance of the rate-½, K=3 encoder with g_1=(111), g_2=(101).

★★★★ 5-7 Marks numerical

Minimum-weight closed path: 00→10(wt2)→01(wt1)→00(wt2), total weight=5. So d_free=5, giving t=⌊4/2⌋=2 correctable errors. Full derivation above.

Q12. What factors affect the free distance of a convolutional code?

★★ 3-5 Marks

Larger constraint length K (more memory) generally increases d_free; choice of generator polynomials strongly affects it (must be chosen carefully, e.g. by search); lower code rate (more redundancy) typically allows higher d_free.

Q13. Compare Linear Block Codes / Cyclic Codes with Convolutional Codes.

★★★ 5 Marks — cross-unit comparison, sometimes asked

Block/Cyclic codes: memoryless, algebraic encoding (matrix/polynomial), syndrome decoding, good for burst errors with interleaving. Convolutional: has memory, shift-register convolution encoding, Viterbi/ML decoding, good for random channel errors (deep space, satellite). Full table given in Section 1 above.

🧠 Appendix — Quick Revision Sheets (QRS)

High-Density Visual Concept Summaries & Exam Formulas for Rapid Pre-Exam Revision

Units 2 to 6 Included | Visual Cheat Sheets

QRS CHEAT SHEET

Unit 2 — Information Theory & Entropy

INFORMATION THEORY & CODING — 5CS3-01

UNIT 2 – QUICK REVISION SHEET

Uncertainty, Information & Entropy, Continuous RV, Source Coding Theorem, Discrete Memoryless Channels, Mutual Information, Conditional Entropy

1. UNCERTAINTY

- Uncertainty means lack of knowledge about the outcome of an event.
- More possible outcomes ⇒ More uncertainty.
- Less possible outcomes ⇒ Less uncertainty.

Key Point:
Uncertainty is reduced when we get information.

2. INFORMATION (SELF INFORMATION)

Information in an event x with probability $P(x)$ is the measure of surprise.

$$I(x) = \log_2 \frac{1}{P(x)} = -\log_2 P(x)$$

Unit: bit

Properties:

- ✓ $I(x) \geq 0$
- ✓ If $P(x) = 1 \Rightarrow I(x) = 0$ (no surpriss)
- ✓ If $P(x) < 1 \Rightarrow I(x) > 0$
- ✓ Less probability ⇒ More information

Quick Values:

$P=1 \Rightarrow I=0$ bit $P=1/2 \Rightarrow I=1$ bit
$P=1/4 \Rightarrow I=2$ bits $P=1/8 \Rightarrow I=3$ bits

3. ENTROPY (AVERAGE INFORMATION)

Entropy is the average information produced by a source.

$$H(X) = -\sum_{x \in X} P(x) \log_2 P(x)$$

Unit: bit/symbol

Properties:

- ✓ $H(X) \geq 0$
- ✓ $H(X) = 0$, when one event is certain.
- ✓ $H(X)$ is maximum when all symbols are equally likely.
- ✓ For n equiprobable symbols:

$$H_{max} = \log_2 n$$

Example:
If $P(H) = 0.5$, $P(T) = 0.5 \Rightarrow H = 1$ bit (max)
If $P(H) = 0.9$, $P(T) = 0.1 \Rightarrow H \approx 0.469$ bits

4. CONTINUOUS RANDOM VARIABLES

For continuous RV, we use Differential Entropy.

Formula:

$$h(X) = -\int_{-\infty}^{\infty} f(x) \log_2 f(x)\, dx$$

where, $f(x)$ = PDF (Probability Density Function)

Key Differences

	Discrete RV	Continuous RV
Uses	PMF ($P(x)$)	PDF ($f(x)$)
Measure	Entropy $H(X)$	Differential Entropy $h(X)$
Operation	Summation (Σ)	Integration ($\int$)

5. SOURCE CODING THEOREM

The average length (L) of any uniquely decodable codeword cannot be less than the entropy.

$$L_{avg} \geq H(X)$$

Where,
L_{avg} = Average code length
$H(X)$ = Entropy of the source

Efficiency:

$$\eta = \frac{H(X)}{L_{avg}} \times 100\% \ (\leq 100\%)$$

Redundancy:

$$R = 1 - \eta$$

Key Point:
No lossless code can compress below entropy.

6. DISCRETE MEMORYLESS CHANNEL (DMC)

- Channel where output depends only on current input (no memory of past).
- Input and output alphabets are discrete.
- Statistically independent transmissions.

Channel Model:

Channel Matrix (Transition Probabilities)
$P(y_j|x_i)$ = Probability of receiving y_j when x_i is sent.

Input / Output	y_1	y_2	⋯	y_m
x_1	$P(y_1\|x_1)$	$P(y_2\|x_1)$	⋯	$P(y_m\|x_1)$
x_2	$P(y_1\|x_2)$	$P(y_2\|x_2)$	⋯	$P(y_m\|x_2)$
⋯	⋯	⋯	⋯	⋯
x_n	$P(y_1\|x_n)$	$P(y_2\|x_n)$	⋯	$P(y_m\|x_n)$

For each i, $\sum_j P(y_j|x_i) = 1$

7. MUTUAL INFORMATION

Measures the amount of information one variable contains about another.

Formula:

$$I(X;Y) = H(X) - H(X|Y)$$
or
$$I(X;Y) = H(Y) - H(Y|X)$$

Also:

$$I(X;Y) = H(X) + H(Y) - H(X,Y)$$

Properties:

- ✓ $I(X;Y) \geq 0$
- ✓ $I(X;Y) = 0 \Rightarrow X$ and Y are independent
- ✓ $I(X;Y) \leq \min\{H(X), H(Y)\}$
- ✓ For noiseless channel: $I(X;Y) = H(X)$

8. CONDITIONAL ENTROPY

Entropy of X after knowing Y.

Formula:

$$H(X|Y) = H(X,Y) - H(Y)$$

Also:

$$H(X|Y) = \sum_y P(y) H(X|Y=y)$$

Interpretation:

- If Y completely predicts $X \Rightarrow H(X|Y) = 0$
- If Y gives no information about $X \Rightarrow H(X|Y) = H(X)$
- Always, $0 \leq H(X|Y) \leq H(X)$

Relation with Mutual Information:

$$I(X;Y) = H(X) - H(X|Y)$$

9. IMPORTANT RELATIONS

1. Self Information ..... $I(x) = -\log_2 P(x)$
2. Entropy ..... $H(X) = -\sum P(x) \log_2 P(x)$
3. Maximum Entropy (n symbols) ..... $H_{max} = \log_2 n$
4. Joint Entropy ..... $H(X,Y) = H(X) + H(Y|X)$
5. Conditional Entropy ..... $H(X|Y) = H(X,Y) - H(Y)$
6. Mutual Information ..... $I(X;Y) = H(X) - H(X|Y)$
7. Source Coding Theorem ..... $L_{avg} \geq H(X)$
8. Efficiency ..... $\eta = \frac{H(X)}{L_{avg}} \times 100\%$
9. Redundancy ..... $R = 1 - \eta$
10. Differential Entropy ..... $h(X) = -\int f(x) \log_2 f(x)\, dx$

IMPORTANT POINTS TO REMEMBER

- ★ Information is measured in bits.
- ★ Entropy is the average uncertainty.
- ★ Maximum entropy occurs for equiprobable symbols.
- ★ Source coding cannot achieve average length < entropy.
- ★ DMC is memoryless: current output depends only on current input.
- ★ Mutual information measures the reduction in uncertainty.
- ★ Conditional entropy is remaining uncertainty after knowing something.

TIPS FOR EXAM

- ★ Write formulas first.
- ★ Draw diagrams (for DMC).
- ★ Explain with definition, formula, properties & example.
- ★ For numericals, show each step clearly.

COMMON NUMERICALS

1. Find $I(x)$ for given probability.
2. Calculate $H(X)$ for a given source.
3. Find H_{max} for n symbols.
4. Calculate efficiency and redundancy.
5. Find $I(X;Y)$ using entropies.
6. Find $H(X|Y)$ using joint entropy.

QRS CHEAT SHEET

Unit 3 — Source Coding & Data Compaction

5CS3-01 INFORMATION THEORY & CODING – UNIT 3
QUICK REVISION SHEET

Source coding schemes for data compaction: Prefix code, Huffman code, Shannon–Fano code & Hempel-Ziv coding
Channel capacity. Channel coding theorem. Shannon limit.

1. SOURCE CODING & DATA COMPACTION

- Source coding removes redundancy in data to represent information using minimum number of bits.
- Goal: Reduce average code length close to entropy $H(X)$.

Key Point
No lossless code can have average length less than entropy.

$$L_{avg} \geq H(X)$$

2. PREFIX CODE

- A code in which no codeword is a prefix of any other codeword.
- Can be uniquely decoded.

Example

Symbol	A	B	C	D
Codeword	0	10	110	111

✓ Prefix-free

Properties
✓ Unique decodability
✓ Satisfies Kraft's inequality

Kraft's Inequality

$$\sum_{i=1}^{n} 2^{-l_i} \leq 1$$

l_i = length of i^{th} codeword

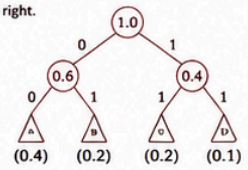
3. HUFFMAN CODE

- Optimal prefix code.
- Gives minimum average code length.
- **Steps:**
  1. Arrange symbols in ascending order of probability.
  2. Combine two least probable symbols.
  3. Replace by their sum probability.
  4. Repeat until one node remains.
  5. Assign 0 to left branch and 1 to right.

Example

Symbol	A	B	C	D
P(x)	0.4	0.3	0.2	0.1

Huffman Tree



Codes

Symbol	A	B	C	D
Codeword	00	01	10	11

Average length $L_{avg} = 0.4\times2 + 0.3\times2 + 0.2\times2 + 0.1\times2 = 2$ bits

4. SHANNON–FANO CODE

- Another method to obtain a prefix code.
- **Steps:**
  1. Arrange symbols in descending order of probability.
  2. Divide the list into two parts with nearly equal probabilities.
  3. Assign 0 to the first part and 1 to the second part.
  4. Repeat the process for each part.
- Not always optimal but easy to construct.

Example

Symbol	A	B	C	D
P(x)	0.4	0.3	0.2	0.1

Division:
[A] (0.4) [B C D] (0.6)
↓ ↓
Codes:
A → 0
B → 10
C → 110
D → 111

5. HEMPEL–ZIV CODING (LZ77 / LZ78)

- Dictionary based coding.
- Exploits repetition in data (no symbol probabilities required).
- Types:
  - **LZ77:** Uses sliding window.
  - **LZ78:** Builds dictionary of phrases.
- Used in lossless data compression (e.g., ZIP, GIF).

Key Point
Works well when data has patterns and repetitions.

Example (LZ77)
Input: ABABABABABA
Output: (A) (B) (2,2) (2,4) (1,5)
(length, distance) pairs

6. CHANNEL CAPACITY

- Maximum rate at which information can be transmitted over a noisy channel with arbitrarily small error.
- Denoted by C (bits per channel use).
- For a Discrete Memoryless Channel (DMC):

$$C = \max_{P(x)} I(X;Y)$$

Where:
- $I(X;Y)$ = Mutual Information
- Maximization is over all possible input distributions $P(x)$

For Binary Symmetric Channel (BSC)
(crossover probability = p)

$$C = 1 - H_2(p)$$

where $H_2(p) = -p\log_2 p - (1-p)\log_2(1-p)$

$0 \leq p \leq 0.5$

- C is maximum when $p = 0$
- $C = 0$ when $p = 0.5$ (pure noise)

7. CHANNEL CODING THEOREM (SHANNON)

For a DMC with capacity C:
- If $R < C$ -> Reliable communication is possible.
- If $R > C$ -> Reliable communication is impossible.
- If $R = C$ -> Reliable communication is possible with infinite block length.

Key Point
By using suitable channel coding, we can reduce probability of error as much as we want, provided rate $R < C$.

8. SHANNON LIMIT

- Practical systems have limitations.
- Shannon limit gives the theoretical limit on performance over a noisy channel.
- Describes the trade-off between bandwidth (W), signal power (S) and noise power (N).
- For a bandlimited channel with bandwidth W Hz and signal-to-noise ratio $\frac{S}{N}$:

$$C = W\log_2\left(1 + \frac{S}{N}\right) \text{ bits/sec}$$

Where:
W = Bandwidth (Hz)
S = Signal Power
N = Noise Power

✓ Higher S or W ⇒ Higher Capacity
✓ Higher Noise ⇒ Lower Capacity

9. IMPORTANT FORMULAS AT A GLANCE

Average Code Length:	$L_{avg} = \sum_i P(x_i)\, l_i$
Entropy:	$H(X) = -\sum_i P(x_i)\log_2 P(x_i)$
Kraft's Inequality:	$\sum_i 2^{-l_i} \leq 1$
Channel Capacity (DMC):	$C = \max_{P(x)} I(X;Y)$
BSC Capacity:	$C = 1 - H_2(p)$
Shannon Limit:	$C = W\log_2\left(1 + \frac{S}{N}\right)$

KEY TAKEAWAYS

✓ Source coding reduces redundancy; average length cannot be < entropy.
✓ Prefix codes ensure unique decodability.
✓ Huffman code gives minimum average length.
✓ Shannon–Fano is easier but not always optimal.
✓ Hempel–Ziv uses dictionary to compress data.
✓ Channel capacity is the ultimate limit for reliable communication.
✓ Shannon Limit shows real-world limit based on bandwidth & noise.

EXAM TIPS

★ Draw Huffman tree carefully and calculate L_{avg}.
★ Remember Kraft's inequality for prefix codes.
★ For numericals, identify whether channel is BSC or general DMC.
★ Write Channel Coding Theorem statement clearly.
★ For Shannon Limit numericals, convert S/N to dB if required.

QRS CHEAT SHEET

Unit 4 — Linear Block Codes

5CS3-01

INFORMATION THEORY & CODING – UNIT 4
QUICK REVISION SHEET

Linear Block Code: Introduction to error connecting codes, coding & decoding of linear block code, minimum distance consideration, conversion of non-systematic form of matrices into systematic form.

1. INTRODUCTION TO ERROR CORRECTING CODES

- Information can be corrupted during transmission due to noise.
- Error correcting codes (ECC) add redundant bits to detect and correct errors.
- Linear Block Codes are the simplest and most widely used ECC.
- An (n,k) linear block code encodes k information bits into n codeword bits $(n > k)$.

Key Point

Redundancy added $= n - k$ bits

Rate of code $R = \frac{k}{n}$

2. LINEAR BLOCK CODE

- A block code is linear if the sum (mod 2) of any two codewords is also a codeword.
- The set of all codewords forms a subspace over GF(2).
- A linear block code is completely specified by its generator matrix G or parity check matrix H.

Parameters

An (n,k) linear block code:

n = Codeword length

k = Number of information bits

$r = n - k$ = Number of parity bits

3. ENCODING & DECODING OF LINEAR BLOCK CODE

(a) Encoding

Message vector: $m = [m_1 \quad m_2 \quad \dots \quad m_k]$

Generator matrix: G $(k \times n)$

Codeword: $c = mG \pmod 2$

$c = [c_1 \quad c_2 \quad \dots \quad c_n]$

(b) Decoding

1. Received vector: $r = c + e \pmod 2$ where e = error vector.
2. Compute syndrome: $s = rH^T \pmod 2$
3. If $s = 0 \rightarrow$ No error
   If $s \neq 0 \rightarrow$ Error present
4. Use syndrome table or decoding rule to find error pattern e and correct it: $c = r + e \pmod 2$

4. MATRICES IN LINEAR BLOCK CODE

(a) Generator Matrix (G)

- A $(k \times n)$ matrix.
- Generates all possible codewords.
- Rank$(G) = k$

(b) Parity Check Matrix (H)

- A $(r \times n)$ matrix, $r = n - k$
- Satisfies: $HG^T = 0$
- Used for error detection and correction.

Standard Form Relationship

$G = [I_k \mid P] \quad (k \times n)$

$H = [P^T \mid I_r] \quad (r \times n)$

where I_k = identity matrix of order k

I_r = identity matrix of order r

P = any binary matrix of size $k \times r$

5. MINIMUM DISTANCE (d_{min})

- Minimum Hamming distance between any two distinct codewords.
- Also called code distance.
- Important for error detection and correction.

Properties

d_{min} = Minimum weight of any non-zero codeword

Capabilities

- Number of detectable errors $= d_{min} - 1$
- Number of correctable errors $(t) = \left\lfloor (d_{min} - 1)/2 \right\rfloor$

$(\lfloor x \rfloor$ = greatest integer $\leq x)$

Example

If $d_{min} = 5$

Detect up to 4 errors

Correct up to 2 errors

6. CONVERSION OF NON-SYSTEMATIC FORM INTO SYSTEMATIC FORM

- Systematic form of G is: $G = [I_k \mid P]$
- If G is not in systematic form, convert using elementary row operations (on G).
- Operations used:
  1. Row interchange
  2. Row addition (mod 2)
  3. Row multiplication by 1 (unchanged)

Steps

1. Apply row operations on G to make left k columns as I_k.
2. The right side automatically becomes P.
3. Find corresponding $H = [P^T \mid I_r]$.

Example

Convert the following G to systematic form:

$$G = \begin{bmatrix} 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 \end{bmatrix} \quad (k = 3, n = 6)$$

After row operations:

$$G_{sys} = \left[\begin{array}{ccc|ccc} 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 \end{array}\right] = [I_3 \mid P]$$

7. SYNDROME AND ERROR DETECTION

- Syndrome: $s = rH^T \pmod 2$
- If $s = 0 \rightarrow$ No error
- If $s \neq 0 \rightarrow$ Error present
- Number of possible syndromes $= 2^r$ (including zero)
- Each non-zero syndrome corresponds to a distinct error pattern (for single-error correcting codes).

Example

$$H = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \end{bmatrix} \quad (r = 2)$$

Syndrome (s)	Possible Error Pattern
00	No error
01	00001
10	00010
11	00100

8. IMPORTANT FORMULAS

- Codeword: $c = mG \pmod 2$
- Syndrome: $s = rH^T \pmod 2$
- Orthogonality: $HG^T = 0$
- Redundancy bits: $r = n - k$
- Code rate: $R = \frac{k}{n}$
- Detectable errors: $d_{min} - 1$
- Correctable errors: $t = \left\lfloor (d_{min} - 1)/2 \right\rfloor$

9. QUICK EXAMPLE (ENCODING & DECODING)

Given:

$$G = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \end{bmatrix}, \quad m = [1 \ 0 \ 1]$$

Encoding:

$$c = mG = [1 \ 0 \ 1] \begin{bmatrix} 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \end{bmatrix} = [1 \ 0 \ 1 \ 1 \ 0]$$

Suppose received: $r = [1 \ 0 \ 0 \ 1 \ 0]$

$$H = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 \end{bmatrix}$$

Syndrome: $s = rH^T = [1 \ 0] \neq 0 \rightarrow$ Error present

(Use syndrome table to find error pattern and correct it.)

10. KEY TAKEAWAYS

- ✓ Linear block codes use matrices G and H.
- ✓ Encoding: $c = mG \pmod 2$.
- ✓ Decoding: use syndrome $s = rH^T$.
- ✓ Minimum distance decides detection & correction capability.
- ✓ Convert G to systematic form $[I_k \mid P]$ for easy encoding.
- ✓ $H = [P^T \mid I_r]$ after conversion.

★ d_{min} is the heart of error control.

★ More $d_{min} \rightarrow$ more errors can be corrected.

★ Always work in GF(2): addition = XOR, multiplication = AND.

★ Row operations (mod 2) do not change the code.

QRS CHEAT SHEET

Unit 5 — Cyclic Codes & Galois Fields

5CS3-01

INFORMATION THEORY & CODING – UNIT 5
QUICK REVISION SHEET

Cyclic Code: Code Algebra, Basic properties of Galois fields (GF), polynomial operations over Galois fields, generating cyclic code by generating polynomial, parity check polynomial, Encoder & decoder for cyclic codes.

1. CODE ALGEBRA

- A block code C is linear of length n over GF(q), if it is a subspace of GF$(q)^n$.
- For cyclic codes, codewords are cyclic shifts of each other.
- In cyclic codes, a codeword $c = (c_0, c_1, \ldots, c_{n-1})$ corresponds to polynomial:

$$c(x) = c_0 + c_1 x + \ldots + c_{n-1}x^{n-1}$$

Key Point:

Addition → polynomial addition (mod q)

Multiplication → polynomial multiplication (mod $(x^n - 1, q)$)

2. BASIC PROPERTIES OF GALOIS FIELDS (GF)

- GF(q) is a finite field with $q = p^m$ elements (p = prime, $m \geq 1$).
- GF(2^m) is widely used in coding.

Properties:

- ✓ Addition and multiplication are closed.
- ✓ Every non-zero element has a multiplicative inverse.
- ✓ For GF(2): $1 + 1 = 0$, $1 + 0 = 1$, $1 \cdot 1 = 1$, $1 \cdot 0 = 0$
- ✓ All arithmetic is performed modulo 2 (or modulo p for GF(p)).

Example (GF(2))

+	0	1
0	0	1
1	1	0

×	0	1
0	0	0
1	0	1

3. POLYNOMIAL OPERATIONS OVER GF(q)

Let:

$$a(x) = a_0 + a_1 x + \ldots + a_r x^r$$

$$b(x) = b_0 + b_1 x + \ldots + b_s x^s$$

- **Addition:** $(a + b)(x) = \sum (a_i + b_i)x^i \pmod q$
- **Subtraction:** $(a - b)(x) = \sum (a_i - b_i)x^i \pmod q$
- **Multiplication:** $(a \cdot b)(x)$ = polynomial multiplication followed by reduction modulo $(x^n - 1, q)$
- **Division:** $a(x) = b(x)q(x) + r(x)$ where $\deg(r(x)) < \deg(b(x))$

4. GENERATING CYCLIC CODE BY GENERATING POLYNOMIAL

- A cyclic code C of length n over GF(q) is uniquely defined by a monic polynomial $g(x)$ of degree r, called the generating polynomial, where $g(x) \mid (x^n - 1)$.
- Codewords are all multiples of $g(x)$ modulo $(x^n - 1)$.
- **Generator polynomial:** $g(x) \mid (x^n - 1)$
- **Dimension of code:** $k = n - r$
- **Any codeword:** $c(x) = m(x)g(x) \bmod (x^n - 1)$ where $m(x)$ is message polynomial of degree $< k$.
- Minimum distance depends on $g(x)$.

5. PARITY CHECK POLYNOMIAL

- The parity check polynomial is:

$$h(x) = \frac{x^n - 1}{g(x)}$$

- $h(x)$ is also a polynomial.
- $\deg(h(x)) = n - k = r$ (number of parity bits)
- $g(x) \cdot h(x) = x^n - 1$
- Parity check matrix H can be obtained from $h(x)$.
- Codeword satisfies:

$$c(x)h(x) \equiv 0 \pmod{x^n - 1}$$

6. PROPERTIES OF CYCLIC CODES

- ✓ Cyclic shift of a codeword is also a codeword.
- ✓ Number of codewords $= q^k$
- ✓ All codewords form an ideal in GF$(q)[x]/(x^n - 1)$.
- ✓ Minimum distance $d_{min} \geq r + 1$ (where $r = \deg(g(x))$)
- ✓ Error detection capability $= d_{min} - 1$
- ✓ Error correction capability $= \left[\frac{d_{min} - 1}{2}\right]$

7. ENCODER FOR CYCLIC CODES

Systematic Encoding Procedure:

Given: Message polynomial $m(x)$ ($\deg < k$), generator polynomial $g(x)$ ($\deg = r$)

Steps:

1. Multiply $m(x)$ by x^r.
2. Divide $x^r m(x)$ by $g(x)$.
3. Let remainder $= r(x)$ ($\deg < r$).
4. Codeword:

$$c(x) = x^r m(x) + r(x)$$

$$(\deg(c(x)) < n)$$

Example

Let $n = 7$, $g(x) = x^3 + x + 1$, message $m(x) = 1 + x + x^2$ (GF(2))

1. $r = \deg(g) = 3$
2. $x^r m(x) = x^3(1 + x + x^2) = x^3 + x^4 + x^5$
3. Divide $x^3 + x^4 + x^5$ by $g(x)$
   Remainder $r(x) = x^2 + 1$
4. Codeword:
   $c(x) = x^3 + x^4 + x^5 + x^2 + 1$
   $= x^5 + x^4 + x^3 + x^2 + 1$
   (Binary: 1 1 1 1 0 1 1)

8. DECODER FOR CYCLIC CODES

Decoding Procedure:

Given: Received polynomial $r(x)$, generator polynomial $g(x)$

Steps:

1. Compute syndrome:

$$s(x) = r(x) \bmod g(x)$$

2. If $s(x) = 0$ → No error, accept $r(x)$ as codeword.
3. If $s(x) \neq 0$ → Error detected.
4. Use syndrome table (or error locator polynomial) to find error pattern $e(x)$.
5. Correct the error:

$$c(x) = r(x) + e(x) \pmod{x^n - 1}$$

9. SYNDROME TABLE METHOD (FOR DECODING)

- For single error correction, syndrome of each possible error pattern is precomputed.
- Syndrome depends only on the position of error.

Example ($n = 7$, $g(x) = x^3 + x + 1$)

Error Position (i)	Error Polynomial $e(x) = x^i$	Syndrome $s(x) = x^i \bmod g(x)$
0	1	1
1	x	x
2	x^2	x^2
3	x^3	$x + 1$
4	x^4	$x^2 + x$
5	x^5	$x^2 + 1$
6	x^6	$x^2 + x + 1$

10. IMPORTANT RELATIONSHIPS

- $g(x) \mid (x^n - 1)$
- $h(x) = \frac{x^n - 1}{g(x)}$
- $g(x)h(x) = x^n - 1$
- $k = n - \deg(g(x))$
- $c(x) = m(x)g(x) \pmod{x^n - 1}$
- $c(x)h(x) \equiv 0 \pmod{x^n - 1}$
- $d_{min} \geq \deg(h(x)) + 1$
- Detectable errors $= d_{min} - 1$
- Correctable errors $= \left[\frac{d_{min} - 1}{2}\right]$

11. QUICK FORMULA SHEET

$$x^n - 1 = g(x) \cdot h(x)$$

$$k = n - \deg(g(x))$$

$$c(x) = x^r m(x) + r(x)$$

$$s(x) = r(x) \bmod g(x)$$

$$c(x)h(x) \equiv 0 \pmod{x^n - 1}$$

KEY TAKEAWAYS

- ✓ Cyclic codes are easy to implement using polynomial arithmetic.
- ✓ Generator polynomial $g(x)$ completely defines the code.
- ✓ Encoding is done by polynomial division.
- ✓ Syndrome helps in error detection and correction.
- ✓ Minimum distance determines error detection & correction capability.

EXAM TIPS

- ★ Learn polynomial operations in GF(2).
- ★ Practice encoding using generator polynomial.
- ★ Understand syndrome calculation & error correction steps.
- ★ Remember relation: $g(x) \cdot h(x) = x^n - 1$.
- ★ Write neat steps for encoder and decoder in exams.

QRS CHEAT SHEET

Unit 6 — Convolutional Codes & Viterbi Algorithm

5CS3-01

INFORMATION THEORY & CODING – UNIT 6
QUICK REVISION SHEET

Convolutional Code: Convolutional encoders of different rates. Code Tree, Trellis and state diagram. Maximum likelihood decoding (Viterbi Algorithm). Free distance of a convolutional code.

1. INTRODUCTION TO CONVOLUTIONAL CODES

- Convolutional codes are linear block codes with memory.
- The encoder output at any time depends on current input bits and past (memory) input bits.
- Specified by (n, k, m):
  - n = number of output bits
  - k = number of input bits
  - m = memory order (number of memory elements)
- Code rate: $R = \frac{k}{n}$
- Constraint length: $K = m + 1$

Key Point:
More memory (m) ⇒ More redundancy ⇒ Better error correction

2. CONVOLUTIONAL ENCODER

- Consists of shift registers, modulo-2 adders ($\oplus$) and connections.
- Encoder described by generator polynomials $g_1(D), g_2(D), \ldots, g_n(D)$
- Input: k bits at each time
- Output: n bits at each time
- Output equation:

$$v_j(D) = \sum_{i=1}^{k} u_i(D)\, g_{ij}(D) \quad (j = 1, 2, \ldots, n)$$

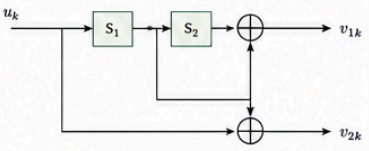
where $u_i(D)$ = input polynomial,
$v_j(D)$ = j^{th} output polynomial
$g_{ij}(D)$ = generator polynomial from i^{th} input to j^{th} output.

3. EXAMPLE ENCODER

Rate $R = \frac{1}{2}$, $m = 2$ $(K = 3)$

Generator polynomials:

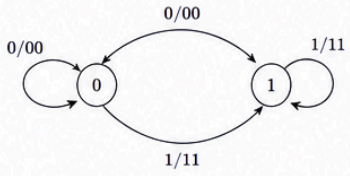
$$g_1(D) = 1 + D + D^2,\ g_2(D) = 1 + D^2$$



At each time k, output = (v_{1k}, v_{2k})

4. STATE DIAGRAM

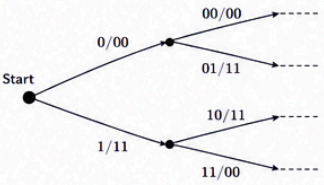
- States represent contents of memory elements.
- For m memory elements ⇒ 2^m states.
- Each branch is labeled as input / output.
- Example: Rate $\frac{1}{2}$, $m = 1$ ⇒ 2 states (0 and 1)



Legend: input/output
e.g., $1/11$ means input = 1, output = 11

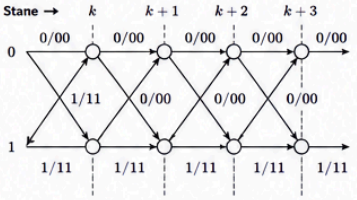
5. CODE TREE

- A tree showing all possible output sequences for all possible input sequences.
- Starts from root (time 0) and expands with time.
- Used for understanding structure and decoding.



6. TRELLIS DIAGRAM

- Time unfolds horizontally, states vertically.
- Each branch connects states between adjacent time instants.
- Preferred over code tree for decoding.
- Example: $m = 1$ (2 states)



Branch label: input/output

7. MAXIMUM LIKELIHOOD DECODING

- Choose the codeword most likely to have been transmitted.
- For AWGN channel, equivalent to minimum Euclidean distance decoding.
- Implemented efficiently using Viterbi Algorithm.

8. VITERBI ALGORITHM (FOR HARD DECISION)

1. Initialize: At time 0, path metric of starting state = 0, others = ∞
2. For each time step k:
   - For each state, compute path metrics of all incoming branches:
     $PM_{new} = PM_{old}$ + (branch metric)
   - Keep the path with minimum metric (survivor path).
3. Continue for all time steps.
4. Traceback from final state (minimum metric) to obtain decoded sequence.

Branch Metric (Hard decision): Hamming distance between received bits and expected output bits.

For soft decision: Use Euclidean distance.

9. FREE DISTANCE (d_f)

- Minimum Hamming distance between any two distinct code sequences.
- Determines error performance.
- Larger d_f ⇒ Lower probability of error.
- For rate $R = \frac{1}{n}$ codes, usually d_f increases with memory m.

Properties:
✓ d_f is always an odd number.
✓ d_f depends on generator polynomials.
✓ d_f is the minimum weight among all non-zero paths in the trellis that start and end in the zero state.

10. RELATION BETWEEN PARAMETERS

Parameter	Meaning
n	Number of output bits
k	Number of input bits
R	Code rate = k/n
m	Memory order
K	Constraint length = $m + 1$
2^m	Number of states
d_f	Free distance

More m ⇒ more states ⇒ more complexity ⇒ more error correction ability.

11. QUICK FORMULA SHEET

- Code rate: $R = \frac{k}{n}$
- Constraint length: $K = m + 1$
- Number of states: 2^m
- Output equation:

$$v_j(D) = \sum_{i=1}^{k} u_i(D)\, g_{ij}(D)$$

- Free distance: d_f (minimum Hamming distance between distinct code sequences)
- Branch metric (hard): Hamming distance
- Branch metric (soft): Euclidean distance

KEY TAKEAWAYS

★ Convolutional codes use memory to add controlled redundancy.
★ Represented using state diagram, code tree and trellis.
★ Viterbi algorithm finds the most likely transmitted path.
★ Free distance is the key factor for error performance.
★ More memory → better error correction but more complexity.

EXAM TIPS

★ Draw state diagram and trellis clearly.
★ Remember label format: input/output.
★ Steps of Viterbi algorithm (initialize → compute → compare → store → traceback).
★ Practice small m (1 or 2) examples for encoding & decoding.